

Introduction to the SLGP Package

Athénaïs Gautier

2026-09-24

This vignette provides a practical introduction to Spatial Logistic Gaussian Process (SLGP) modeling, demonstrating its implementation and application on the classical Fiji-Tonga earthquake catalogue.

Versioning note. This vignette was built with **SLGP** version 2.0.0, on **R** version 4.4.1 (2024-06-14 ucrt) (2026-09-24). Please report the version alongside any issue.

1 Session information

For reproducibility, the exact package versions used to build this vignette:

```
R> utils::packageVersion("SLGP")
[1] '2.0.0'
R> sessionInfo()
R version 4.4.1 (2024-06-14 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 22631)

Matrix products: default

locale:
 [1] LC_COLLATE=French_France.utf8  LC_CTYPE=French_France.utf8
 [3] LC_MONETARY=French_France.utf8 LC_NUMERIC=C
 [5] LC_TIME=French_France.utf8

time zone: Europe/Paris
tzcode source: internal

attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods   base

other attached packages:
[1] forcats_1.0.0    viridis_0.6.5    viridisLite_0.4.2 ggpubr_0.6.0
[5] ggplot2_4.0.0    dplyr_1.1.4      tidyr_1.3.1      SLGP_2.0.0

loaded via a namespace (and not attached):
 [1] gtable_0.3.6      xfun_0.49         QuickJSR_1.4.0
 [4] rstatix_0.7.2     inline_0.3.19     lattice_0.22-6
 [7] vctrs_0.6.5       tools_4.4.1       generics_0.1.3
[10] goftest_1.2-3     stats4_4.4.1      parallel_4.4.1
[13] tibble_3.2.1      fansi_1.0.6       pkgconfig_2.0.3
[16] Matrix_1.7-0      RColorBrewer_1.1-3 S7_0.2.0
[19] RcppParallel_5.1.9 lifecycle_1.0.4    compiler_4.4.1
```

[22]	farver_2.1.2	MatrixModels_0.5-3	codetools_0.2-20
[25]	carData_3.0-5	SparseM_1.84-2	DiceDesign_1.10
[28]	quantreg_6.1	htmltools_0.5.8.1	yaml_2.3.10
[31]	Formula_1.2-5	pillar_1.9.0	car_3.1-3
[34]	MASS_7.3-60.2	StanHeaders_2.32.10	abind_1.4-8
[37]	rstan_2.32.6	tidyselect_1.2.1	digest_0.6.37
[40]	purrr_1.0.2	bookdown_0.42	splines_4.4.1
[43]	fastmap_1.2.0	grid_4.4.1	colorspace_2.1-1
[46]	cli_3.6.3	magrittr_2.0.3	loo_2.8.0
[49]	survival_3.6-4	pkgbuild_1.4.5	utf8_1.2.4
[52]	broom_1.0.7	withr_3.0.2	scales_1.4.0
[55]	backports_1.5.0	rmarkdown_2.29	matrixStats_1.4.1
[58]	gridExtra_2.3	ggsignif_0.6.4	mvnfast_0.2.8
[61]	evaluate_1.0.1	knitr_1.48	rstantools_2.4.0
[64]	rlang_1.1.4	Rcpp_1.1.2	glue_1.8.0
[67]	rstudioapi_0.17.1	R6_2.5.1	

2 Dataset

We illustrate the model’s capabilities using the `quakes` dataset shipped with base R [R Core Team, 2025]: 1000 seismic events of body-wave magnitude above 4.0 recorded since 1964 in the Fiji-Tonga region, originating from the Harvard PRIM-H project. Each event carries its epicentre (`lat`, `long`), its hypocentre `depth` (40-680 km), its magnitude `mag`, and the number of `stations` that detected it.

For this vignette, we focus on modeling the distribution of hypocentre depths (`depth`) along an east-west transect, indexed by longitude (`long`). The region hosts two subduction systems, and the resulting Wadati-Benioff zones [Isacks et al., 1968, Frohlich, 2006] make the conditional distribution of `depth` deform across the transect - from shallow-unimodal near the trench, to bimodal where shallow and deep seismicity coexist, to deep-dominated. Together with the naturally clustered epicentres (few replicates, wildly uneven coverage), this makes the catalogue an ideal stress test for SLGP modeling.

```
R> data("quakes")
R> library(tidyr)
R> library(dplyr)
```

At first, we will use a unidimensional index, to allow for easier visualisations, quantitative as well as qualitative analysis.

3 A uni-dimensional index and continuous response application

We represent the data to visualise the relationship between `depth` and `long`.

```
R> # Bin data together for visualisation purpose
R> df <- quakes %>%
+   mutate(long_bin = cut(long, breaks = seq(165, 190, by = 2.5), include.lowest = FALSE)) %>%
+   group_by(long_bin) %>%
+   mutate(long_bin = paste0(long_bin, "\nn=", n())) %>%
+   ungroup() %>%
+   mutate(long_bin = factor(long_bin,
+                             levels = sort(unique(long_bin), decreasing = FALSE))) %>%
+   data.frame()
R>
R> range_response <- c(40, 680) # Can use range(df$depth), or user defined range as we do here
R> range_x <- c(165, 190) # Can use range(df$long), or user defined range as we do here
R>
```

```

R> library(ggplot2)
R> library(ggpubr)
R> library(viridis)
R> # Scatterplot: long vs depth
R> scatter_plot <- ggplot(df, aes(x = long, y = depth)) +
+   geom_point(alpha = 0.5, color = "grey", pch=20) +
+   labs(y = "Hypocentral depth (km)",
+        x = "Longitude (°)",
+        title = "Scatterplot of the data") +
+   theme_bw()+
+   coord_cartesian(xlim=range_x,
+                   ylim=range_response)
R>
R> # Histogram: Distribution of depth by 'long' bin
R> hist_plot <- ggplot(df, aes(x = depth)) +
+   geom_histogram(mapping=aes(y=after_stat(density)),
+                  position = "identity", breaks = seq(40, 680, 40),
+                  fill="darkgrey", col="grey50", lwd=0.2, alpha=0.7) +
+   geom_rug(sides = "b", color = "navy", alpha = 0.5)+
+   facet_wrap(~ long_bin, scales = "free_y", nrow=2) +
+   labs(x = "Hypocentral depth (km)",
+        y = "Probability density",
+        title = "Histogram of 'depth' by 'longitude' group") +
+   theme_bw()+
+   coord_cartesian(xlim=range_response,
+                   ylim=c(0, 0.015))
R> ggarrange(scatter_plot, hist_plot, ncol = 2, nrow = 1,
+            widths = c(0.3, 0.7))

```

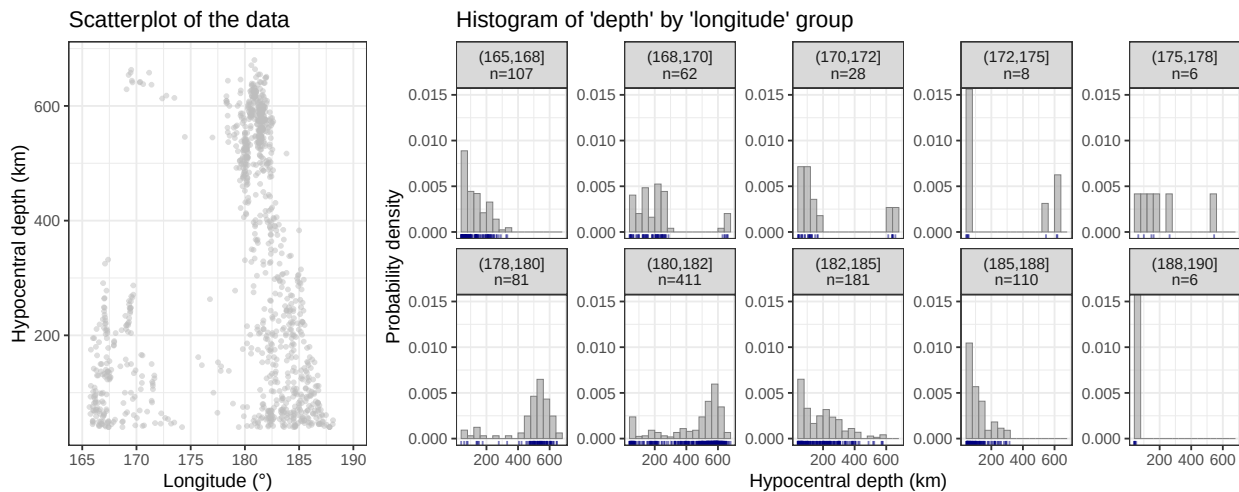


Figure 1: Hypocentre depth against longitude in the Fiji-Tonga ‘quakes’ catalogue (left), and empirical depth distributions within longitude bands of width 2.5 degrees (right). Sample sizes per band vary by two orders of magnitude, and the distribution changes shape across the transect.

```

R> ggsave("../Figures/scatter.pdf", width=10, height=3.5, scale=1.0)

```

The two planes of seismicity are visible to the naked eye: a shallow band near the trenches and a deep band, down to nearly 700 km, along the subducting slab. Between the two, both regimes coexist and the empirical distributions turn bimodal. This dataset provides a demanding test case for SLGP modeling, offering a

compact, one-dimensional covariate space, heterogeneously distributed data, and shifting distributional shapes with a well-understood geophysical ground truth.

To model the distributional changes observed in the earthquake catalogue, we now introduce the Spatial Logistic Gaussian Process (SLGP) model. SLGPs provide a flexible non-parametric framework for modeling spatially dependent probability densities. By transforming a Gaussian Process (GP) through exponentiation and normalization, SLGPs ensure positivity and integration to one, making them well-suited for density estimation. In this section, we specify the SLGP prior, and visualise its behaviour.

3.1 Prior

The prior in an SLGP represents our initial beliefs about the structure of the data before incorporating observations. It defines a distribution over possible density functions, capturing spatial dependencies while allowing sufficient flexibility. The following code chunk sets up an SLGP prior over `depth ~ long`, where `depth` is modeled as a function of `long`:

```
R> library(SLGP)
R>
R> modelPrior <- slgp(depth~long, # Use a formula to specify predictors VS response
+                       # Can use depth~. for all variables,
+                       # Or 'depth' ~ long + var2 + var3 for more variables
+                       data=df,
+                       method="Prior", # Prior, no estimation scheme
+                       basisFunctionsUsed = "RFF",
+                       interpolateBasisFun="WNN", # Will Accelerate inference
+                       hyperparams = list(lengthscale=c(0.15, 0.15),
+                                           # Applied to normalised data
+                                           # So 0.15 is 15% of the range of values
+                                           sigma2=1),
+
+                       # Will be re-selected with sigmaEstimationMethod
+                       sigmaEstimationMethod = "heuristic",
+                       # Set to heuristic for numerical stability
+                       predictorsLower= c(range_x[1]),
+                       predictorsUpper= c(range_x[2]),
+                       responseRange= range_response,
+                       opts_BasisFun = list(nFreq=200,
+                                           MatParam=5/2),
+                       seed=1, opts=list(ndraws = 3))
R>
R> print(modelPrior)
Spatial Logistic Gaussian Process (SLGP) model
  Formula      : depth ~ long
  Response     : depth  in [40, 680]
  Covariate(s) : long
  Estimation   : Prior
  Basis        : RFF  (rank p = 400)
  Length-scales : 0.15, 0.15
  Variance sigma2: 1.124
  Data         : 1000 obs, 605 distinct covariate value(s)
  Fitting time  : 1.04s
  Coefficient draws: 3
R> summary(modelPrior)
Summary of a Spatial Logistic Gaussian Process (SLGP) model
  Formula      : depth ~ long
  Response     : depth  in [40, 680]
```

```

Covariate(s) : long
Estimation   : Prior
Basis        : RFF (rank p = 400)
Length-scales : 0.15, 0.15
Variance sigma2: 1.124
Data         : 1000 obs, 605 distinct covariate value(s)

```

Estimation diagnostics

```

Fitting time   : 1.04s (setup 1.04s, estimation 0s)
Coefficient draws : 3
Log-posterior  : not available for method 'Prior'

```

Here:

- The model uses Random Fourier Features (RFF) for the finite-rank latent GP.
- The lengthscale (15% of the normalized range) controls the smoothness of variation. This is selected following our proposed heuristic.
- The heuristic sigma estimation ensures numerical stability.

3.1.1 Looking at several draws of the prior

To understand how the SLGP prior behaves, we generate and visualize random draws from the prior distribution over probability densities of `depth` at different `long` values. We first use the standard `plot()` method for SLGP objects.

```
R> plot(modelPrior, draw = c(1:3), panels = TRUE)
```

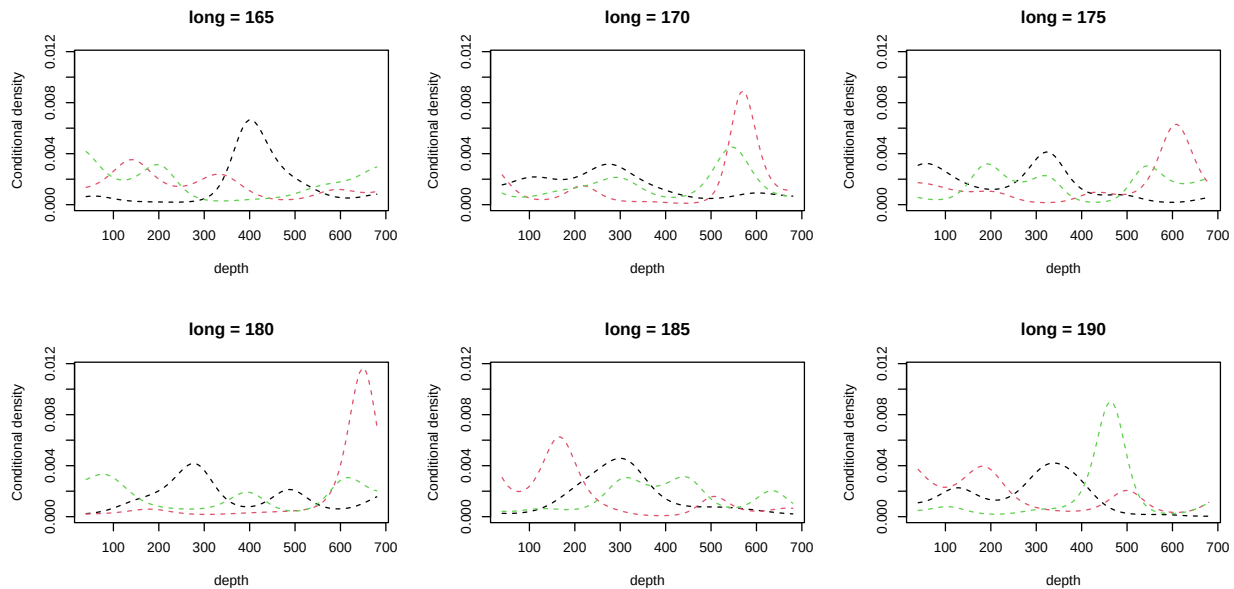


Figure 2: Three draws from the SLGP prior, shown as conditional densities of `depth` at six longitudes, from the base `plot()` method.

```

R> dev.copy(pdf, "../Figures/ribbonsPriorBase.pdf", width = 10, height = 3.5)
pdf
3
R> dev.off()

```

pdf
2

We can also use the `predict()` method for SLGP objects and make more advanced visuals.

```
R>
R> dfGrid <- data.frame(expand.grid(seq(range_x[1], range_x[2], 1),
+                               seq(range_response[1], range_response[2],, 101)))
R> colnames(dfGrid) <- c("long", "depth")
R>
R> predPrior <- predict(modelPrior, newdata = dfGrid, type="density")
R>
R> colnames(predPrior) <- c("long", "depth", paste0("Draw from the prior n°", seq(3)))
R>
R> predPrior <- predPrior %>% pivot_longer(-c("long", "depth"))
R> scale_factor <- 300
R>
R> ggplot() +
+   labs(title = "Prior SLGP samples of conditional depth densities, visualised across slices",
+         x = "Hypocentral depth (km)",
+         y = "Longitude (°)") +
+   theme_bw() +
+   geom_ribbon(data=predPrior,
+             mapping=aes(x=depth, ymax=scale_factor*value+long,
+                       ymin=long, group=-long, fill=long),
+             col="grey", alpha=0.9) +
+   scale_fill_viridis(option = "plasma",
+                     guide = guide_colorbar(title = "Indexing variable: longitude",
+                                           barheight = unit(2, units = "mm"),
+                                           barwidth = unit(55, units = "mm"),
+                                           title.position = 'top',
+                                           label.position = "bottom",
+                                           title.hjust = 0.5)) +
+   theme(legend.position = "bottom") +
+   coord_flip() +
+   facet_grid(.~name)
```

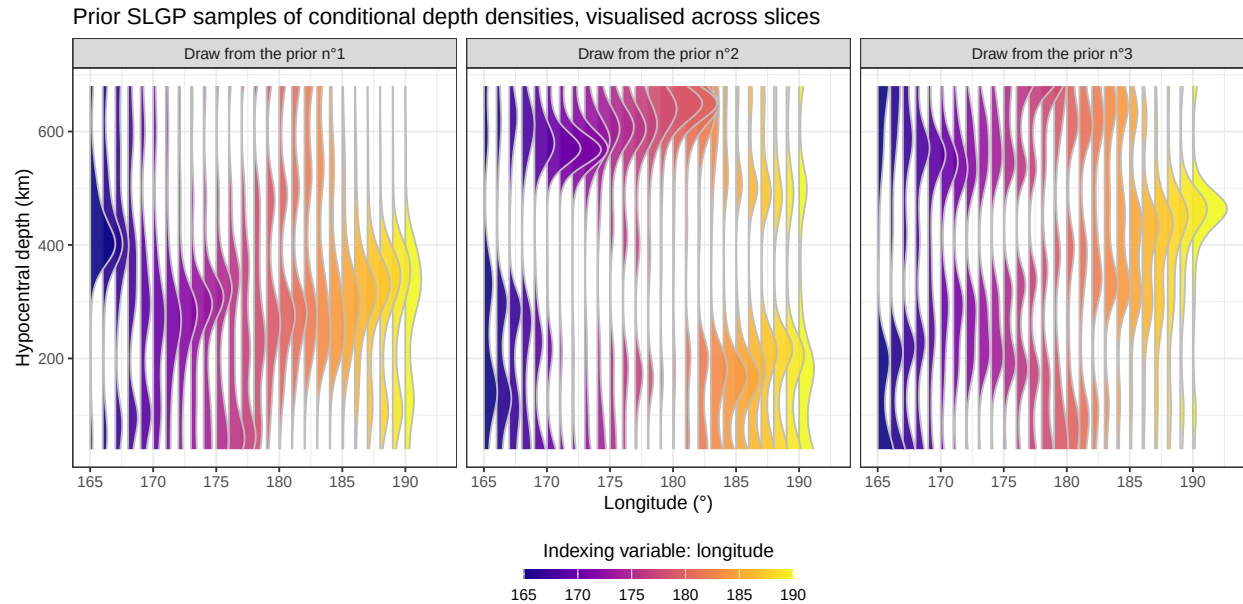


Figure 3: Three draws from the SLGP prior, visualised as conditional densities of depth along slices of longitude. Each draw is a whole field of densities, varying smoothly with longitude.

```
R> ggsave(paste0("./Figures/ribbonsPrior", ".pdf"), width=10, height=4)
```

This figure illustrates how the SLGP prior encodes a distribution over densities for different `long` values. The wide variability indicates the flexibility of SLGPs.

3.1.2 Assessing whether the flexibility matches that of the data

To assess how well the SLGP prior aligns with the actual data distribution, we can also compare it to histograms of `depth` at selected `long` values. This visualisation helps us evaluate whether the prior has enough flexibility to represent the observed variability in the data - including the shallow/deep bimodality.

```
R> selected_values <- c(167, 180, 185)
R> gap <- 0.5
R> df_filtered <- df %>%
+   mutate(interval=findInterval(long, c(0,
+                                   selected_values[1]-gap,
+                                   selected_values[1]+gap,
+                                   selected_values[2]-gap,
+                                   selected_values[2]+gap,
+                                   selected_values[3]-gap,
+                                   selected_values[3]+gap)))%>%
+   filter(interval %in% c(2, 4, 6))%>%
+   group_by(interval)%>%
+   mutate(category = paste0("Longitude close to ", c("", selected_values[1],
+                                                       "", selected_values[2],
+                                                       "", selected_values[3])[interval],
+                             "\nn=", n()))
R>
R> names <- sort(unique(df_filtered$category))
R> dfGrid <- data.frame(expand.grid(selected_values,
+                                   seq(range_response[1], range_response[2], 101)))
R> colnames(dfGrid) <- c("long", "depth")
```

```

R> predPrior <- predict(modelPrior, newdata = dfGrid)
R> colnames(predPrior) <- c("long", "depth", paste0("Draw from the prior n°", seq(3)))
R> predPrior <- predPrior%>%
+   pivot_longer(-c("long", "depth"))
R> predPrior$category <- ifelse(predPrior$long==selected_values[1], names[1],
+                               ifelse(predPrior$long==selected_values[2], names[2], names[3]))
R>
R> ggplot(mapping=aes(x = depth)) +
+   geom_histogram(df_filtered,
+                 mapping=aes(y=after_stat(density)),
+                 position = "identity", breaks = seq(40, 680, 20),
+                 fill="darkgrey", col="grey50", lwd=0.2, alpha=0.7) +
+   geom_rug(data=df_filtered, sides = "b", color = "navy", alpha = 0.5)+
+   geom_line(data=predPrior, mapping=aes(y=value, group=name),
+             color = "black", lwd=0.1, alpha=0.5)+
+   geom_line(data=predPrior, mapping=aes(y=value, group=name, col=name), lwd=1.1)+
+   facet_wrap(~ category, scales = "free_y", nrow=1) +
+   labs(x = "Hypocentral depth (km)",
+        y = "Probability density",
+        title = "Binned 'depth' histograms by 'long' (width = 1) VS three SLGP prior draws at bins centers
+   theme_bw()+
+   theme(legend.position="bottom",
+         legend.direction = "horizontal",
+         legend.title = element_blank())+
+   coord_cartesian(xlim=range_response,
+                   ylim=c(0, 0.02))

```

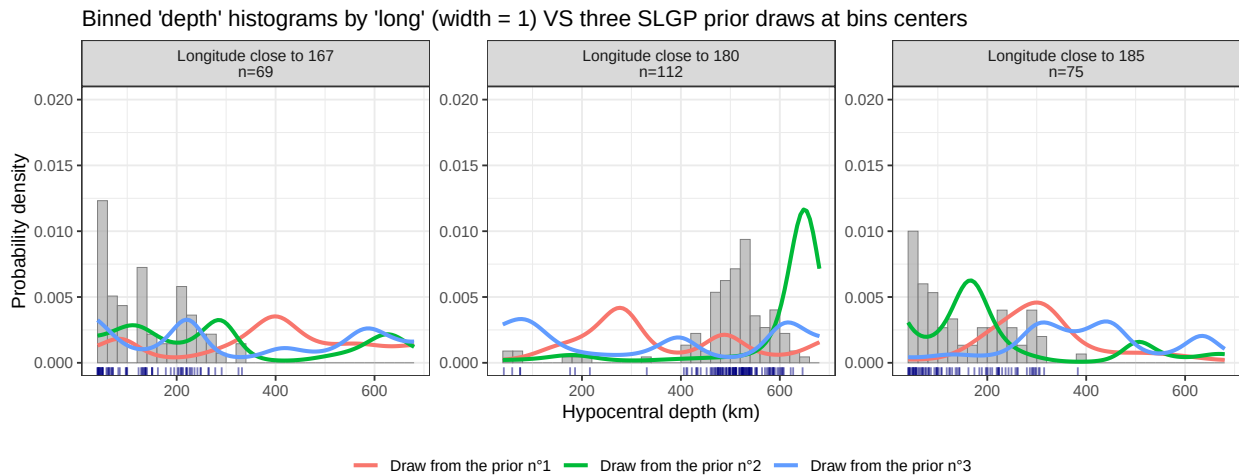


Figure 4: Three SLGP prior draws at three longitudes, against the histogram of the events within 0.5 degree of each. The prior is not fitted to the data; the comparison only checks that its flexibility is of the right order.

```

R> ggsave(paste0("./Figures/histPrior", ".pdf"), width=10, height=3.5)

```

Since this is a prior distribution, it does not yet incorporate information from the actual data. Therefore, we should not expect it to match the empirical histograms. However, the prior samples display a reasonable level of variability and structure, suggesting that the model would be well-suited for density estimation after incorporating data through posterior inference.

3.1.3 Other priors

To explore how different covariance structures influence the SLGP prior, we compare three additional kernel choices:

- Exponential Kernel: A special case of the Matérn class ($\nu = 1/2$) that results in less smooth realizations.
- Matérn 3/2 Kernel: A compromise between flexibility and smoothness.
- Gaussian Kernel: The limiting case, leading to infinitely differentiable functions.

```
R> common_args <- list( formula = depth ~ long, data = df, method = "Prior",
+                       basisFunctionsUsed = "RFF", interpolateBasisFun = "WNN",
+                       hyperparams = list(lengthscale = c(0.15, 0.15), sigma2 = 1),
+                       sigmaEstimationMethod = "heuristic", opts=list(ndraws=1),
+                       predictorsLower = range_x[1], predictorsUpper = range_x[2],
+                       responseRange = range_response, seed = 1)
R>
R> # With Exponential = Matérn 1/2 kernel
R> modelPrior2 <- do.call(slgp, c(common_args,
+                                list(opts_BasisFun = list(nFreq = 200, MatParam = 1/2))))
R> # With Matérn 3/2 kernel
R> modelPrior3 <- do.call(slgp, c(common_args,
+                                list(opts_BasisFun = list(nFreq = 200, MatParam = 3/2))))
R> # With Gaussian = Matérn Inf kernel
R> modelPrior4 <- do.call(slgp, c(common_args,
+                                list(opts_BasisFun = list(nFreq = 200, MatParam = Inf))))
```

The figure below visualizes samples from the priors corresponding to each kernel. While the overall behavior remains similar, the choice of kernel influences the smoothness of the density estimates. The exponential kernel exhibits the most variability, while the Gaussian kernel enforces stronger smoothness constraints.

```
R> dfGrid <- data.frame(expand.grid(seq(range_x[1], range_x[2], 1),
+                                   seq(range_response[1], range_response[2], 101)))
R> colnames(dfGrid) <- c("long", "depth")
R>
R> predPrior <- rbind(predict(modelPrior2, newdata = dfGrid),
+                     predict(modelPrior3, newdata = dfGrid),
+                     predict(modelPrior4, newdata = dfGrid))
R> predPrior$Kernel <- c(apply(seq(3), function(i){rep(i, nrow(dfGrid))}))
R> colnames(predPrior) <- c("long", "depth", "value", "Kernel")
R>
R> predPrior$Kernel <- factor(c("Exponential", "Matérn 3/2", "Gaussian")[predPrior$Kernel],
+                           levels=c("Exponential", "Matérn 3/2", "Gaussian"))
R> scale_factor <- 300
R> ggplot() +
+   labs(x = "Hypocentral depth (km)",
+        y = "Longitude (°)",
+        title = paste("Prior SLGP samples, visualised across slices.",
+                       "SLGPs have varying degrees of smoothness.")) +
+   theme_bw()+
+   geom_ribbon(data=predPrior,
+              mapping=aes(x=depth, ymax=scale_factor*value+long,
+                          ymin=long, group=-long, fill=long),
+              col="grey", alpha=0.9)+
+   scale_fill_viridis(option = "plasma",
+                      guide = guide_colorbar(title = "Indexing variable: Longitude",
+                                              barheight = unit(2, units = "mm"),
```

```

+                                     barwidth = unit(55, units = "mm"),
+                                     title.position = 'top',
+                                     label.position = "bottom",
+                                     title.hjust = 0.5))+
+   theme(legend.position = "bottom")+
+   coord_flip()+
+   facet_grid(.~Kernel)

```

Prior SLGP samples, visualised across slices. SLGPs have varying degrees of smoothness.

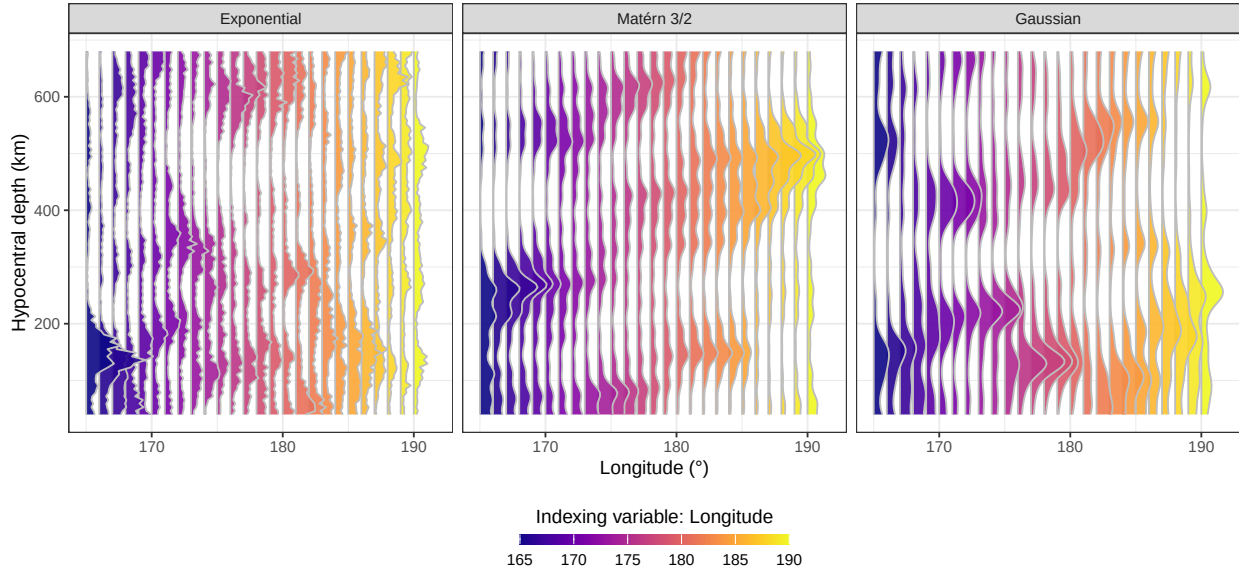


Figure 5: Prior draws under three Matérn kernels of increasing smoothness: exponential, Matérn 3/2 and Gaussian. The regularity of the kernel carries over to the fields of densities.

```
R> ggsave(paste0("./Figures/ribbonsPriorOthers", ".pdf"), width=10, height=4)
```

3.2 Estimation: Maximum a posteriori estimate

For a fast and computationally efficient estimation, we propose using MAP estimation. MAP delivers a single point estimate by maximizing the posterior distribution. It is the fastest estimation scheme we propose, however MAP does not facilitate uncertainty quantification because it yields a non-probabilistic estimate of the underlying density field, focusing instead on identifying the mode of the posterior distribution.

3.2.1 Performing the estimation

We demonstrate two equivalent ways to train an SLGP model using our package:

- Direct initialization: Computes the basis functions and performs the full estimation from scratch.
- Retraining from another model: Reuses pre-computed basis functions from an existing SLGP model and updates only the coefficients for efficiency.

```

R> modelMAP <- slgp(depth~long,
+                   data=df,
+                   method="MAP", #Maximum a posteriori estimation scheme
+                   basisFunctionsUsed = "RFF",
+                   interpolateBasisFun="WNN",
+                   hyperparams = list(lengthscale=c(0.15, 0.15),

```

```

+                               sigma2=1),
+                               sigmaEstimationMethod = "heuristic",
+                               predictorsLower= c(range_x[1]),
+                               predictorsUpper= c(range_x[2]),
+                               responseRange= range_response,
+                               opts_BasisFun = list(nFreq=200,
+                               MatParam=5/2),
+                               seed=1)

R> # Or equivalent, re-use the same basis functions
R> # and hyper parameters as in the prior we saw
R>
R> modelMAP <- update(modelPrior,newdata = df, method="MAP")

R> summary(modelMAP)
Summary of a Spatial Logistic Gaussian Process (SLGP) model
  Formula      : depth ~ long
  Response     : depth in [40, 680]
  Covariate(s) : long
  Estimation    : MAP
  Basis         : RFF (rank p = 400)
  Length-scales : 0.15, 0.15
  Variance sigma2: 1.124
  Data         : 1000 obs, 605 distinct covariate value(s)

Estimation diagnostics
  Fitting time      : 1.36s (setup 0.81s, estimation 0.55s)
  Coefficient draws : 1
  Log-posterior     : 655.461 (at mode for MAP/Laplace)
  Optimiser         : converged

```

3.2.2 Visualising the results

The estimated conditional density function is displayed using three different representations:

- The default visual in the `plot()` method.
- A colormap showing the predicted density of `depth` as a function of `long`.
- A ribbon plot, which slices the conditional density at selected `long` values to provide a clearer view of the estimated distributions.

```
R> plot(modelMAP, draw = "mean", panels = TRUE)
```

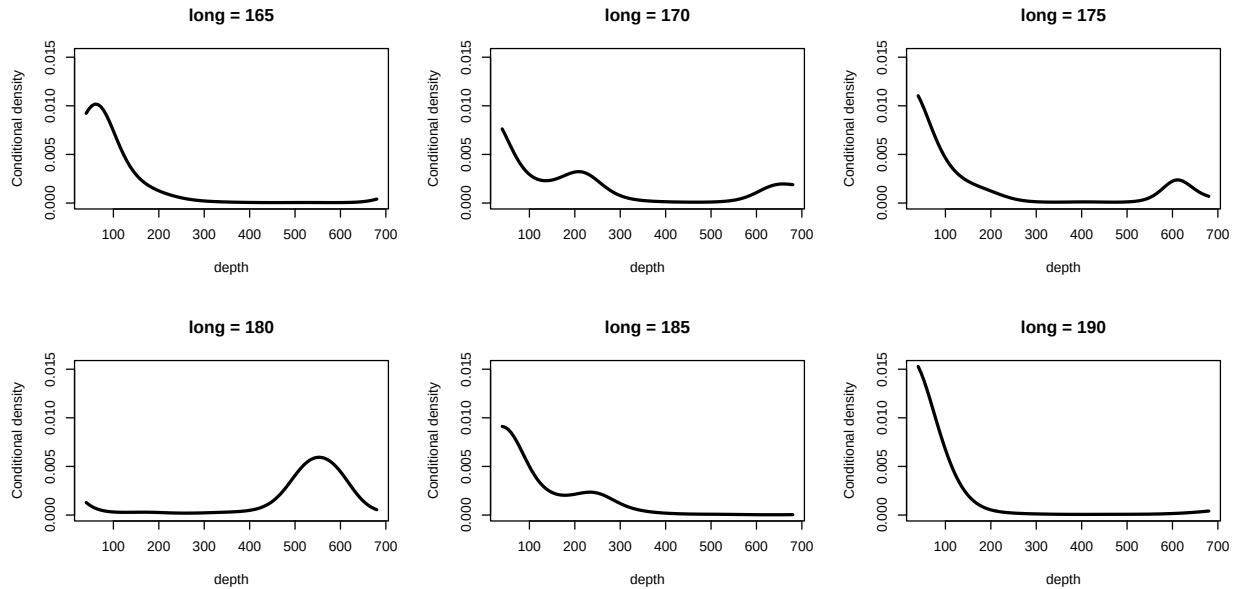


Figure 6: MAP estimate of the conditional density of depth at six longitudes, from the base 'plot()' method.

```
R> dev.copy(pdf, "../Figures/defaultMAP.pdf", width = 10, height = 3.5)
pdf
3
R>
R> dev.off()
pdf
2

R> dfGrid <- data.frame(expand.grid(seq(range_x[1], range_x[2], 101),
+                               seq(range_response[1], range_response[2], 101)))
R> colnames(dfGrid) <- c("long", "depth")
R> pred <- predict(modelMAP, newdata = dfGrid)
R>
R> ggplot() +
+   labs(y = "Hypocentral depth (km)",
+        x = "Longitude (°)") +
+   theme_bw() +
+   geom_raster(data=pred,
+               mapping=aes(x=long, y=depth, fill=pdf_1)) +
+   geom_point(data=df,
+              mapping=aes(x=long, y=depth), alpha = 0.5,
+              pch="x", col="grey") +
+   scale_fill_viridis(option = "viridis",
+                       guide = guide_colorbar(title = "Conditional density of depth",
+                                                barheight = unit(2, units = "mm"),
+                                                barwidth = unit(55, units = "mm"),
+                                                title.position = 'top',
+                                                label.position = "bottom",
+                                                title.hjust = 0.5)) +
+   theme(legend.position = "bottom")
```

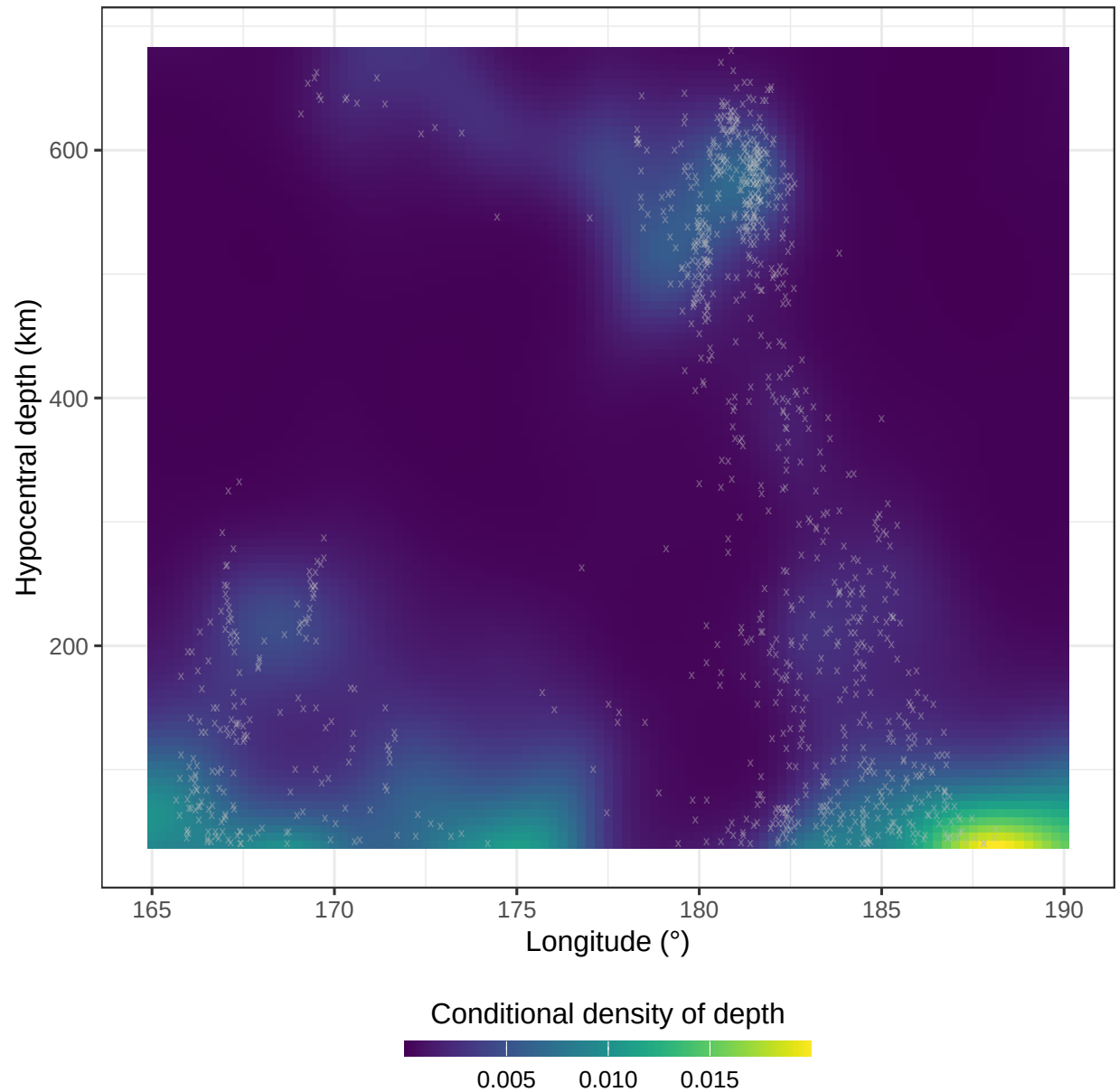


Figure 7: MAP estimate of the whole density field, as a heat map over longitude and depth, with the observed events overlaid.

```
R> library(viridis)
R> dfGrid <- data.frame(expand.grid(seq(range_x[1], range_x[2], 1),
+                               seq(range_response[1], range_response[2], 101)))
R> colnames(dfGrid) <- c("long", "depth")
R> pred <- predict(modelMAP, newdata= dfGrid)
R>
R> scale_factor <- 300
R> ggplot() +
+   labs(x = "Hypocentral depth (km)",
+        y = "Longitude (°)") +
+   theme_bw() +
```

```

+ geom_point(data=df,
+           mapping=aes(x = depth, y = long), alpha = 0.5,
+           pch=20, color = "navy")+
+ geom_ribbon(data=pred,
+           mapping=aes(x=depth, ymax=scale_factor*pdf_1+long,
+                       ymin=long, group=-long, fill=long),
+           col="grey", alpha=0.8)+
+ scale_fill_viridis(option = "plasma",
+                   guide = guide_colorbar(title = "Indexing variable: Longitude",
+                                           barheight = unit(2, units = "mm"),
+                                           barwidth = unit(55, units = "mm"),
+                                           title.position = 'top',
+                                           label.position = "bottom",
+                                           title.hjust = 0.5))+
+ theme(legend.position = "bottom")+
+ coord_flip()

```

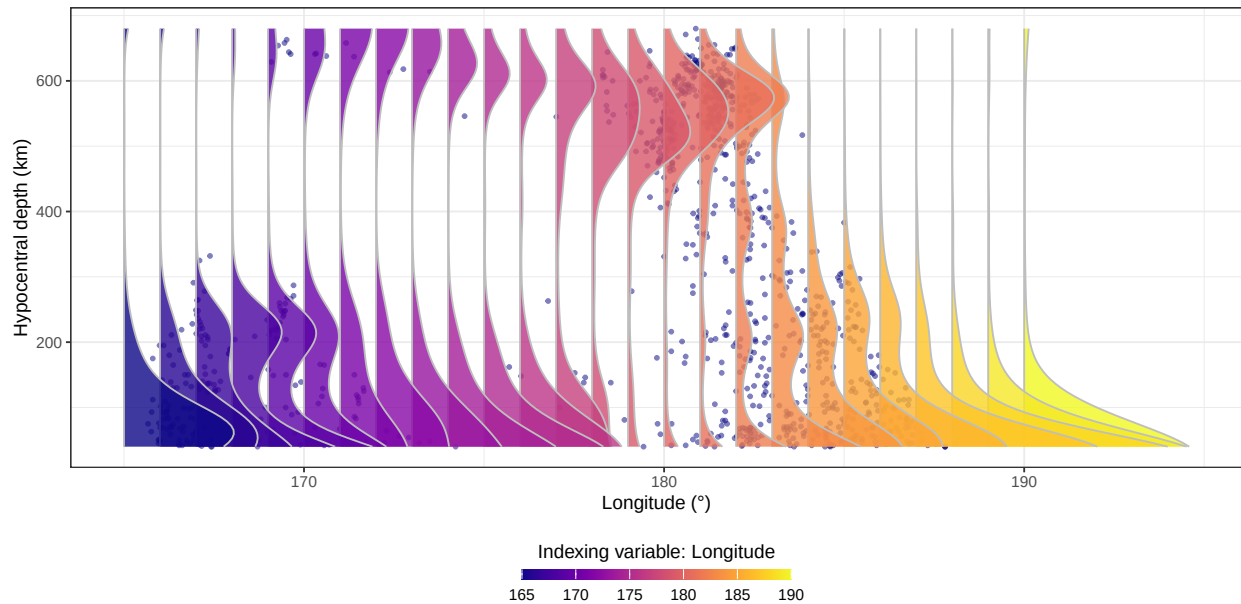


Figure 8: MAP estimate of the conditional density of depth, visualised along slices of longitude, with the observed events overlaid.

```

R> ggsave(paste0("./Figures/ribbonsMAP", ".pdf"), width=10, height=4)

```

The figure reveals how the SLGP captures the structure in the estimated conditional density of **depth** given **long**. It effectively adapts as the shallow mode near the trench gives way to the deep Wadati-Benioff mode along the transect. Notably, the model tracks the bimodal transition region, assigning probability mass to both regimes where they coexist.

3.2.3 Comparing the estimation to data

```

R> selected_values <- c(167, 180, 185)
R> gap <- 0.5
R> df_filtered <- df %>%
+   mutate(interval=findInterval(long, c(0,
+                                       selected_values[1]-gap,

```

```

+               selected_values[1]+gap,
+               selected_values[2]-gap,
+               selected_values[2]+gap,
+               selected_values[3]-gap,
+               selected_values[3]+gap)))]>%
+   filter(interval %in% c(2, 4, 6))>%
+   group_by(interval)>%
+   mutate(category = paste0("long close to ", c("", selected_values[1],
+               "", selected_values[2],
+               "", selected_values[3])[interval],
+               "\nn=", n()))
R> names <- sort(unique(df_filtered$category))
R> dfGrid <- data.frame(expand.grid(selected_values,
+               seq(range_response[1], range_response[2], 101)))
R> colnames(dfGrid) <- c("long", "depth")
R> predMAP <- predict(modelMAP, newdata = dfGrid)
R> colnames(predMAP) <- c("long", "depth", "MAP estimator")
R> predMAP <- predMAP%>%
+   pivot_longer(-c("long", "depth"))
R> predMAP$category <- ifelse(predMAP$long==selected_values[1], names[1],
+               ifelse(predMAP$long==selected_values[2], names[2], names[3]))
R>
R> ggplot(mapping=aes(x = depth)) +
+   geom_histogram(df_filtered,
+               mapping=aes(y=after_stat(density)),
+               position = "identity", breaks = seq(40, 680, 20),
+               fill="darkgrey", col="grey50", lwd=0.2, alpha=0.7) +
+   geom_rug(data=df_filtered, sides = "b", color = "navy", alpha = 0.5)+
+   geom_line(data=predMAP, mapping=aes(y=value, group=name),
+               color = "black", lwd=0.1, alpha=0.5)+
+   geom_line(data=predMAP, mapping=aes(y=value, group=name, col=name), lwd=1.1)+
+   facet_wrap(~ category, scales = "free_y", nrow=1) +
+   labs(x = "Hypocentral depth (km)",
+        y = "Probability density",
+        title = "Binned 'depth' histograms by 'long' (width = 1) VS SLGP-MAP estimators at bins center")
+   theme_bw()+
+   theme(legend.position="bottom",
+         legend.direction = "horizontal",
+         legend.title = element_blank())+
+   coord_cartesian(xlim=range_response,
+                   ylim=c(0, 0.02))

```

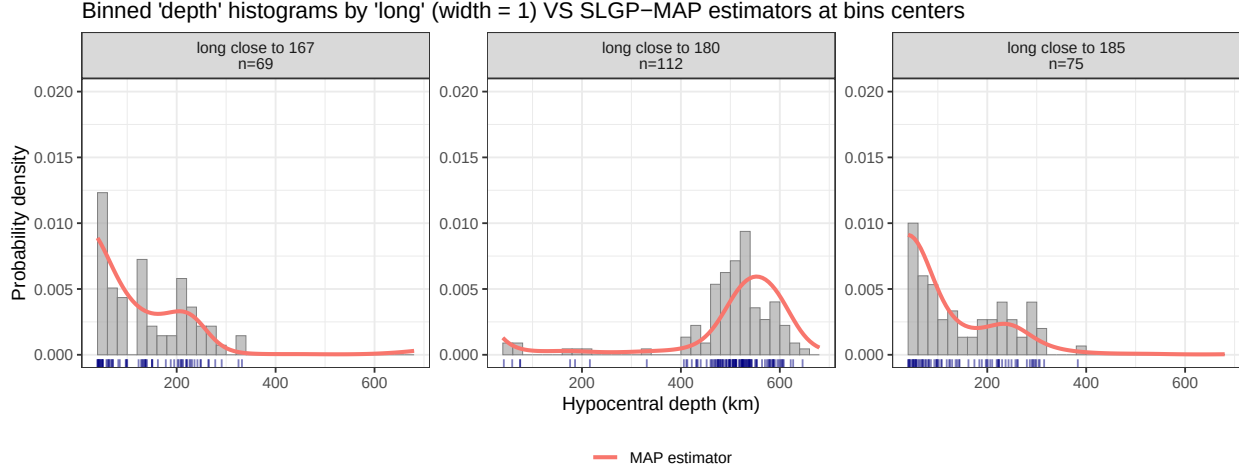


Figure 9: MAP estimate of the conditional density at three longitudes, against the histogram of the events within 0.5 degree of each. The histograms are for illustration only: the SLGP itself requires no binning.

```
R>
R> ggsave(paste0("./Figures/histMAP", ".pdf"), width=10, height=3.5)
```

This other figure directly compares the SLGP MAP estimate to histograms of `depth` at selected `long` values, illustrating how well the model aligns with the observed data. Despite the absence of replicates, the SLGP estimate follows the empirical distribution, adapting to changes in shape - and in the number of modes - across the transect.

3.2.4 Influence of the lengthscale selected

To assess the impact of lengthscale selection on SLGP estimation, we follow the MAP-based strategy described in the paper. We define a prior over the lengthscales, perform posterior evaluations over a regular grid, and compare optimization results under two different data usage regimes:

- Full dataset: model is trained and evaluated on the full data.
- Partial training: model is trained on 25% of the data and evaluated on the remaining 75%.

Concretely, we specify an Inverse-Gamma prior on each lengthscale parameter.

```
R> set.seed(1)
R> newOrder <- sample(seq(nrow(df)))
R>
R> dinvgamma <- function(x, alpha=4.5, beta=0.35) {
+   ifelse(x<=0, 0, (beta^alpha / gamma(alpha)) * x^(-alpha - 1) * exp(-beta / x))
+ }
R> plot(dinvgamma, from=0, to=1)
R> if(file.exists("OptimisingLenQuakes.RData")){
+   load(file="OptimisingLenQuakes.RData")
+ }
+ }else{
+   lengthscale_grid <- seq(0.025, 0.5, 0.025)
+   df_res <- data.frame(expand.grid(lengthscale_grid, lengthscale_grid))
+   colnames(df_res) <- c("l_long", "l_depth")
+   df_res$logPostData <- NaN
+   df_res$logPostData2 <- NaN
+ }
```



```

R>
R>
R> df_res$logPrior <- log(dinvgamma(df_res$l_long))+log(dinvgamma(df_res$l_depth))
R> for(i in which(round(df_res$l_long*100)%5==0&
+               round(df_res$l_depth*100)%5==0)){
+   l1 <- df_res$l_long[i]
+   l2 <- df_res$l_depth[i]
+   if(is.na(df_res$logPostData[i])){
+
+     modelMAPtemp <- update(modelMAP,
+                           newdata = df,
+                           method="MAP",
+                           hyperparams=list(sigma2=1,
+                                             lengthscale=c(l1, l2)),
+                           sigmaEstimationMethod="heuristic")
+     df_res$logPostData[i] <- modelMAPtemp$logPost
+     if(sum(!is.na(df_res$logPost))%10==0){
+       save(df_res, file="OptimisingLenQuakes.RData")
+     }
+   }
+   if(is.na(df_res$logPostData2[i])){
+     modelMAPtemp <- update(modelMAP,
+                           newdata = df[newOrder[1:750],
+                                           c("long", "depth")],
+                           method="MAP",
+                           hyperparams=list(sigma2=1,
+                                             lengthscale=c(l1, l2)),
+                           sigmaEstimationMethod="heuristic")
+     temp <- predict(modelMAPtemp, newdata = df[newOrder[-c(1:750)],
+                                           c("long", "depth") ])
+     df_res$logPostData2[i] <- sum(log(temp$pdf_1))
+     if(sum(!is.na(df_res$logPostData2))%10==0){
+       save(df_res, file="OptimisingLenQuakes.RData")
+     }
+   }
+ }
R> save(df_res, file="OptimisingLenQuakes.RData")
R>

```

The following figure displays the resulting posterior landscape.

```

R> load(file="OptimisingLenQuakes.RData")
R>
R> min_done <- df_res[which.min(-df_res$logPostData2-df_res$logPrior), ]
R>
R> df_res %>%
+   dplyr::filter(!is.na(logPostData2))%>%
+   ggplot(mapping=aes(x=l_long, y=l_depth))+
+   geom_tile(mapping=aes(fill=-logPostData2-logPrior))+
+   geom_contour(mapping=aes(z=-logPostData2-logPrior), col="black", bins=20)+
+   geom_point(data=min_done, col="red")+
+   theme_bw()+
+   scale_fill_viridis(direction=-1,
+                       guide = guide_colourbar(barwidth = 15))+
+   labs(fill="log Posterior")+

```

```
+ theme(legend.position = "bottom",
+       legend.direction="horizontal")+
+ ggtitle("Trained on 75% of dataset, evaluated on remaining 25%")+
+ xlab("Lengthscale for 'long'")+
+ ylab("Lengthscale for 'depth'")
```

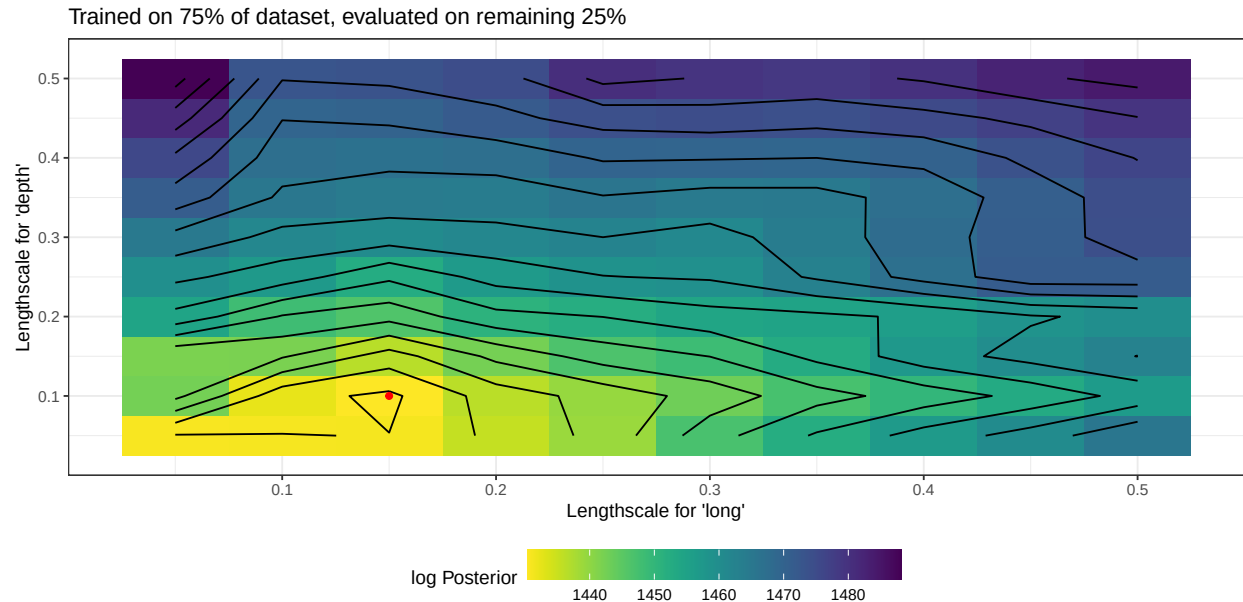


Figure 10: Posterior log-density (up to a constant) over the lengthscales grid (long, depth), under two training strategies. Left: trained and evaluated on the full dataset. Right trained on 25% of the data, evaluated on the remaining 75%. The red dot indicates the optimal lengthscales in each setting.

```
R>
R> ggsave(paste0("./Figures/optimLandscape", ".pdf"), width=5, height=4)
```

We observe that this landscapes is fairly consistent with our proposed heuristic of setting all lengthscales around 10-15% of the ranges.

Now, for a qualitative interpretation, we fit and compare SLGP models with three representative choices of lengthscales:

- Short lengthscales (0.01, 0.01) — inducing highly localized priors,
- Long lengthscales (0.5, 0.5) — yielding overly smooth priors,
- Data-driven MAP estimate from the partial training scheme.

```
R> modelMAP2 <- update(modelPrior,
+                      newdata = df,
+                      method="MAP",
+                      hyperparams=list(sigma2=1,
+                                       lengthscales=c(0.01, 0.01)),
+                      sigmaEstimationMethod="heuristic")
R> modelMAP3 <- update(modelPrior,
+                      newdata = df,
+                      method="MAP",
+                      hyperparams=list(sigma2=1,
+                                       lengthscales=c(0.5, 0.5)),
+                      sigmaEstimationMethod="heuristic")
```

```

R> modelMAP4 <- update(modelPrior,
+                       newdata = df,
+                       method="MAP",
+                       hyperparams=list(sigma2=1,
+                                       lengthscale=c(min_done$l_depth, min_done$l_long)),
+                                       sigmaEstimationMethod="heuristic")
R> gc()
      used (Mb) gc trigger (Mb) max used (Mb)
Ncells 3385134 180.8   6320271 337.6  6320271 337.6
Vcells 9587787  73.2   34972773 266.9  34970966 266.9

```

This results in the following fitted densities:

```

R> dfGrid <- data.frame(expand.grid(seq(range_x[1], range_x[2], 1),
+                                   seq(range_response[1], range_response[2], 101)))
R>
R> colnames(dfGrid) <- c("long", "depth")
R> pred1 <- predict(modelMAP, newdata = dfGrid)
R> pred1$Len <- "Heuristic lengthscale"
R> pred2 <- predict(modelMAP2, newdata = dfGrid)
R> pred2$Len <- "Short lengthscale"
R> pred3 <- predict(modelMAP3, newdata = dfGrid)
R> pred3$Len <- "Long lengthscale"
R> pred4 <- predict(modelMAP4, newdata = dfGrid)
R> pred4$Len <- "Optimised lengthscale"
R> pred <- rbind(pred1, pred2, pred3, pred4)
R> scale_factor <- 100
R> ggplot() +
+   labs(x = "Hypocentral depth (km)",
+        y = "Longitude (°)") +
+   theme_bw()+
+   geom_point(data=df,
+             mapping=aes(x = depth, y = long), alpha = 0.5, color = "navy", pch=20)+
+   geom_ribbon(data=pred,
+             mapping=aes(x=depth, ymax=scale_factor*pdf_1+long,
+                       ymin=long, group=-long, fill=long),
+                       col="grey", alpha=0.8)+
+   scale_fill_viridis(option = "plasma",
+                     guide = guide_colorbar(title = "Indexing variable: Longitude",
+                                             barheight = unit(2, units = "mm"),
+                                             barwidth = unit(55, units = "mm"),
+                                             title.position = 'top',
+                                             label.position = "bottom",
+                                             title.hjust = 0.5))+
+   theme(legend.position = "bottom")+
+   facet_grid(Len~.)+
+   coord_flip()

```

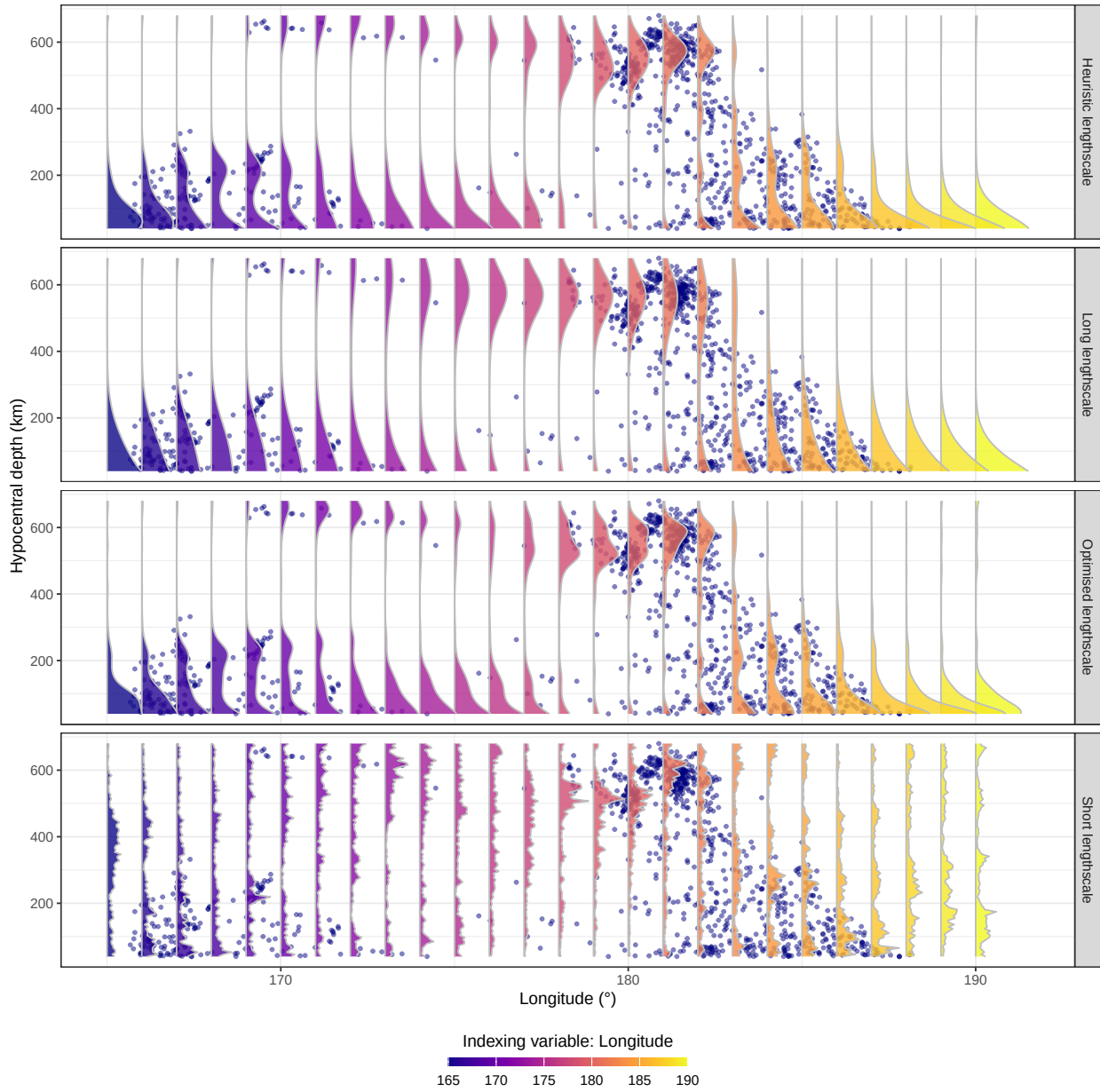


Figure 11: MAP estimates of the density field under four length-scale settings: heuristic, long, optimised and short. Short length-scales overfit the individual events and long ones smooth out the bimodality.

```
R>
R> ggsave(paste0("./Figures/lengthscaleRibbons", ".pdf"),
+         width=10, height=10)
R> dfGrid <- data.frame(expand.grid(selected_values,
+                                   seq(range_response[1], range_response[2], 101)))
R> colnames(dfGrid) <- c("long", "depth")
R>
R> pred1 <- predict(modelMAP, newdata = dfGrid)
R> pred1$Len <- "Heuristic lengthscale"
R> pred2 <- predict(modelMAP2, newdata = dfGrid)
```

```

R> pred2$Len <- "Short lengthscale"
R> pred3 <- predict(modelMAP3, newdata = dfGrid)
R> pred3$Len <- "Long lengthscale"
R> pred4 <- predict(modelMAP4, newdata = dfGrid)
R> pred4$Len <- "Optimised lengthscale"
R> pred <- rbind(pred1, pred2, pred3, pred4)
R>
R> colnames(pred) <- c("long", "depth", "MAP estimator", "Len")
R> pred <- pred%>%
+   pivot_longer(-c("long", "depth", "Len"))
R> pred$category <- ifelse(pred$long==selected_values[1], names[1],
+   ifelse(pred$long==selected_values[2], names[2], names[3]))
R>
R> ggplot(mapping=aes(x = depth)) +
+   geom_histogram(df_filtered,
+   mapping=aes(y=after_stat(density)),
+   position = "identity", breaks = seq(40, 680, 20),
+   fill="darkgrey", col="grey50", lwd=0.2, alpha=0.7) +
+   geom_rug(data=df_filtered, sides = "b", color = "navy", alpha = 0.5)+
+   geom_line(data=pred, mapping=aes(y=value, group=name),
+   color = "black", lwd=0.1, alpha=0.5)+
+   geom_line(data=pred, mapping=aes(y=value, group=name, col=name), lwd=1.1)+
+   facet_grid(Len ~ category, scales = "free_y") +
+   labs(x = "Hypocentral depth (km)",
+   y = "Probability density",
+   title = "Binned 'depth' histograms by 'long' (width = 1) VS SLGP-MAP estimators at bins center")
+   theme_bw()+
+   theme(legend.position="bottom",
+   legend.direction = "horizontal",
+   legend.title = element_blank())+
+   coord_cartesian(xlim=range_response,
+   ylim=c(0, 0.02))

```

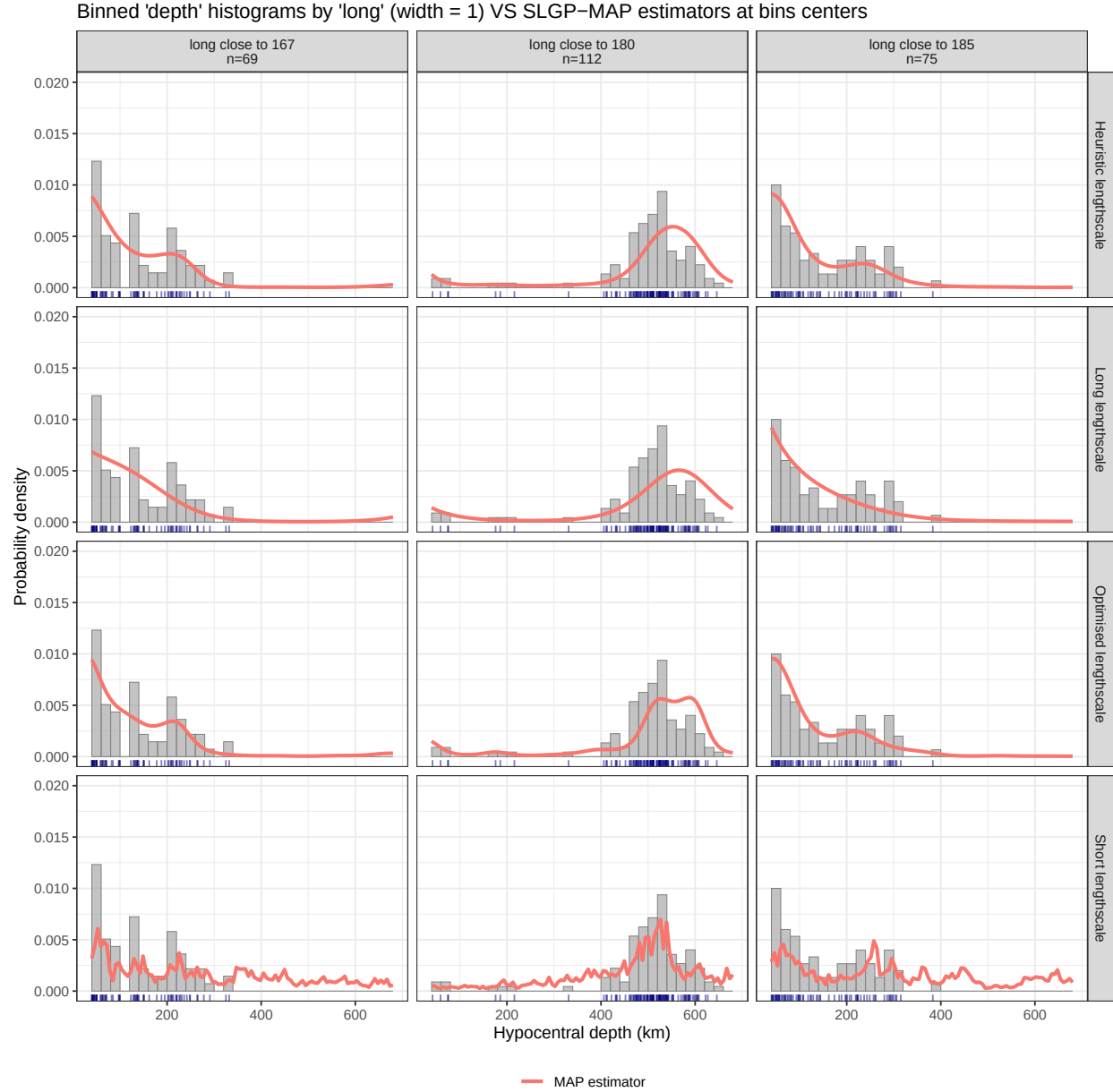


Figure 12: The four length-scale settings at three longitudes, against the local histograms.

```
R>
R> ggsave(paste0("./Figures/lengthscaleHist", ".pdf"), width=10, height=10)
R> rm(modelMAP2, modelMAP3, modelMAP4, pred1, pred2, pred3, pred4); gc()
      used (Mb) gc trigger (Mb) max used (Mb)
Ncells 3416666 182.5   6320271 337.6   6320271 337.6
Vcells 9699478  74.1   34972773 266.9  34970966 266.9
```

We observe that when the lengthscale is too big, while the model captures the general support of the distribution, it fails to capture finer details, such as multi-modalities in certain distributions. This limitation likely arises from using a large lengthscale, which oversmooths small-scale variations. On the opposite using too small a length scale restricts the model's spatial aggregation, limiting the reach of spatial information to a very local scope.

Using a heuristic as before, or the optimised version yield a compromise between the previously observed limitations. For simplicity, we will continue with the lengthscales yielded by the heuristic, set to 15% of the ranges. A more quantitative study on the impact of this hyperparameter on SLGP estimation is presented in a subsequent section.

3.3 Laplace approximation estimate

By integrating the MAP approach with Laplace approximation, we refine our estimation strategy by approximating the posterior distribution with a multivariate Gaussian. This method strikes a balance between the full Bayesian inference of MCMC and the computational efficiency of MAP estimation. By leveraging both the gradient and Hessian of the posterior, it captures essential curvature information, providing a more informed approximation of the posterior landscape.

As for MAP, we could have use two implementations: training from scratch, or re-training a model. We go for the second.

```
R> modellaplace <- update(modelMAP, newdata = df, method="Laplace")
```

```
R> summary(modellaplace)
```

Summary of a Spatial Logistic Gaussian Process (SLGP) model

```
Formula      : depth ~ long
Response     : depth in [40, 680]
Covariate(s) : long
Estimation   : Laplace
Basis        : RFF (rank p = 400)
Length-scales : 0.15, 0.15
Variance sigma2: 1.124
Data         : 1000 obs, 605 distinct covariate value(s)
```

Estimation diagnostics

```
Fitting time      : 1.86s (setup 0.49s, estimation 1.37s)
Coefficient draws : 1000
Log-posterior     : 655.461 (at mode for MAP/Laplace)
Hessian nugget    : none (invertible)
Hessian condition : 121.5
```

`summary()` reports the convergence diagnostics of the sampler (R-hat, effective sample sizes, divergent transitions) and the time spent in each phase. With `diagnostics = TRUE` it also evaluates the fitted field on the data: the mean log predictive density and the probability integral transform of the observations. `newdata` makes this an out-of-sample check.

```
R> summary(modellaplace, diagnostics = TRUE)
```

Warning in `ks.test.default(u, "punif", 0, 1)`: ties should not be present for the one-sample Kolmogorov-Smirnov test

Summary of a Spatial Logistic Gaussian Process (SLGP) model

```
Formula      : depth ~ long
Response     : depth in [40, 680]
Covariate(s) : long
Estimation   : Laplace
Basis        : RFF (rank p = 400)
Length-scales : 0.15, 0.15
Variance sigma2: 1.124
Data         : 1000 obs, 605 distinct covariate value(s)
```

Estimation diagnostics

```
Fitting time      : 1.86s (setup 0.49s, estimation 1.37s)
Coefficient draws : 1000
```

```

Log-posterior      : 655.461 (at mode for MAP/Laplace)
Hessian nugget     : none (invertible)
Hessian condition  : 121.5

```

```

Predictive diagnostics (in-sample, n = 1000)
  Mean log density : -5.7558 per observation
  PIT vs uniform   : KS = 0.03951, p = 0.0881
  Laplace IS-ESS    : 656.5 of 1000 draws
  (in-sample; pass newdata= for an honest assessment)

```

```

R> timing(modelLaplace)
  method  p nobs ndraws setup estimation total n_chains warmup sample
1 Laplace 400 1000  1000  0.49      1.37  1.86      NA      NA      NA

```

For the Laplace fit, the importance-sampling effective sample size measures how well the Gaussian approximation matches the true posterior: a value close to the number of draws means that the two agree, and that the extra cost of MCMC buys little.

Unlike MAP estimation, which provides only a point estimate, the Laplace approximation allows us to visualize uncertainty.

```

R> plot(modelLaplace, draw = c("mean", 1:10), panels = TRUE)

```

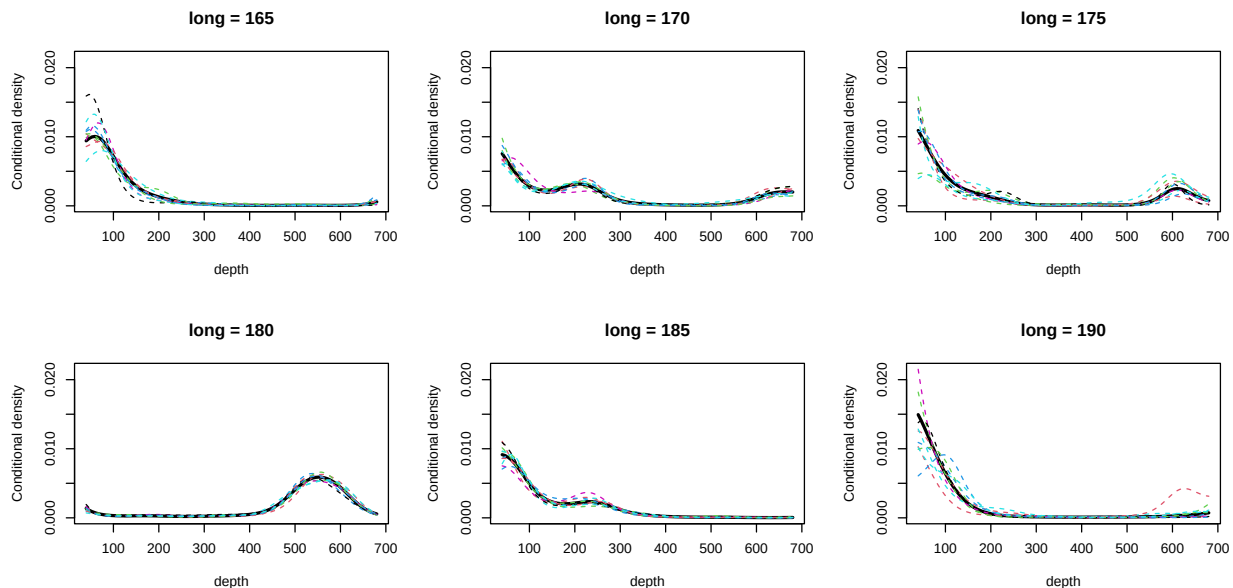


Figure 13: Posterior mean and draws of the conditional density of depth under the Laplace approximation, at six longitudes.

```

R>
R> dev.copy(pdf, "./Figures/defaultLaplace.pdf", width = 10, height = 3.5)
pdf
3
R> dev.off()
pdf
2

```

To highlight the uncertainty quantification, we compare multiple posterior draws from the approximation against histogram data, emphasizing how the estimated conditional density fluctuates. The figure below

illustrates this by overlaying samples from the posterior approximation with binned data, showcasing the range of possible densities rather than focusing solely on the MAP estimate.

```
R>
R> dfGrid <- data.frame(expand.grid(seq(3),
+                               seq(range_response[1], range_response[2], 101)))
R> colnames(dfGrid) <- c("ID", "depth")
R> dfGrid$long <- selected_values[dfGrid$ID]
R> pred <- predict(modelLaplace, newdata = dfGrid)
R> pred$meanpdf <- rowMeans(pred[, -c(1:3)])
R>
R> library(tidyr)
R> # Filter the data: keep values within  $\pm 5$  of the selected ones
R> df_filtered <- df %>%
+   mutate(interval=findInterval(long, c(0,
+                                     selected_values[1]-gap,
+                                     selected_values[1]+gap,
+                                     selected_values[2]-gap,
+                                     selected_values[2]+gap,
+                                     selected_values[3]-gap,
+                                     selected_values[3]+gap)))%>%
+   filter(interval %in% c(2, 4, 6))%>%
+   group_by(interval)%>%
+   mutate(category = paste0("long close to ", c("", selected_values[1],
+                                                "", selected_values[2],
+                                                "", selected_values[3])[interval],
+                             "\nn=", n()))
R> names <- sort(unique(df_filtered$category))
R> pred$category <- names[pred$ID]
R>
R> set.seed(1)
R> selected_cols <- sample(seq(1000), size=10, replace=FALSE)
R> df_plot <- pred %>%
+   dplyr::select(c("long", "depth", "category",
+                   paste0("pdf_", selected_cols)))%>%
+   pivot_longer(-c("long", "depth", "category"))
R>
R>
R> ggplot(mapping=aes(x = depth)) +
+   geom_histogram(df_filtered,
+                 mapping=aes(y=after_stat(density)),
+                 position = "identity", breaks = seq(40, 680, 20),
+                 fill="darkgrey", col="grey50", lwd=0.2, alpha=0.7) +
+   geom_rug(data=df_filtered, sides = "b", color = "navy", alpha = 0.5)+
+   geom_line(data=df_plot, mapping=aes(y=value, group=name),
+             color = "black", lwd=0.1, alpha=0.5)+
+   geom_line(data=pred, mapping=aes(y=meanpdf, group=category), color = "red")+
+   facet_wrap(~ category, scales = "free_y", nrow=1) +
+   labs(x = "Hypocentral depth (km)",
+        y = "Probability density",
+        title = "Binned 'depth' histograms by 'long' (width = 1) VS SLGP-Laplace estimators at bins c
+   theme_bw()+
+   coord_cartesian(xlim=range_response,
+                   ylim=c(0, 0.02))
```

Binned 'depth' histograms by 'long' (width = 1) VS SLGP–Laplace estimators at bins centers

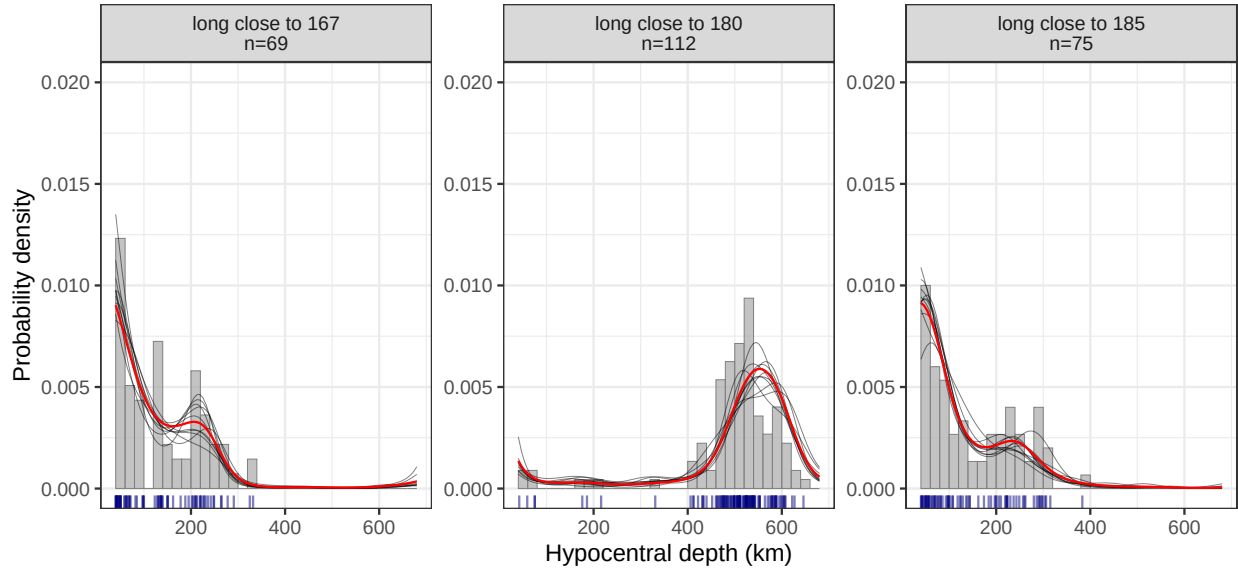


Figure 14: Posterior mean and draws of the conditional density under the Laplace approximation, at three longitudes, against the local histograms.

```
R> ggsave(paste0("./Figures/histLap", ".pdf"), width=10, height=3.5)
R>
```

3.4 MCMC estimate

This method allows us to fully explore the posterior distribution by drawing samples from it. Unlike MAP or Laplace approximation, which provide a single point estimate or a Gaussian approximation, MCMC enables exact Bayesian inference of the underlying density field, given the observed data. The main drawback of this approach being its higher computational cost

As for MAP and Laplace, we could have used all three implementations: training from scratch, re-training a model or manually specifying components. We go for the second again.

```
R> modelMCMC <- update(modelMAP, newdata = df, method="MCMC",
+                       opts = list(stan_chains=2, stan_iter=1000), seed=1)
R>
```

```
R> summary(modelMCMC)
Summary of a Spatial Logistic Gaussian Process (SLGP) model
Formula       : depth ~ long
Response      : depth in [40, 680]
Covariate(s)  : long
Estimation    : MCMC
Basis         : RFF (rank p = 400)
Length-scales : 0.15, 0.15
Variance sigma2: 1.124
Data          : 1000 obs, 605 distinct covariate value(s)
```

```
Estimation diagnostics
Fitting time      : 9m 50s (setup 0.67s, estimation 9m 49s)
Per chain        : warmup 2m 4s, 2m 24s | sampling 2m 25s, 2m 55s
Coefficient draws : 1000
```

```

Log-posterior      : 454.971 (posterior mean)
Chains / iter      : 2 / 500
R-hat              : max 1.003 (0 of 400 above 1.01)
Eff. sample size   : min 1423 (0 of 400 below 400)
Divergences        : 0 max-tredepth hits: 0
Min BFMI           : 0.928

```

MCMC provides a rich representation of uncertainty in the estimated density.

```
R> plot(modelMCMC, draw = c("mean", 1:10), panels = TRUE)
```

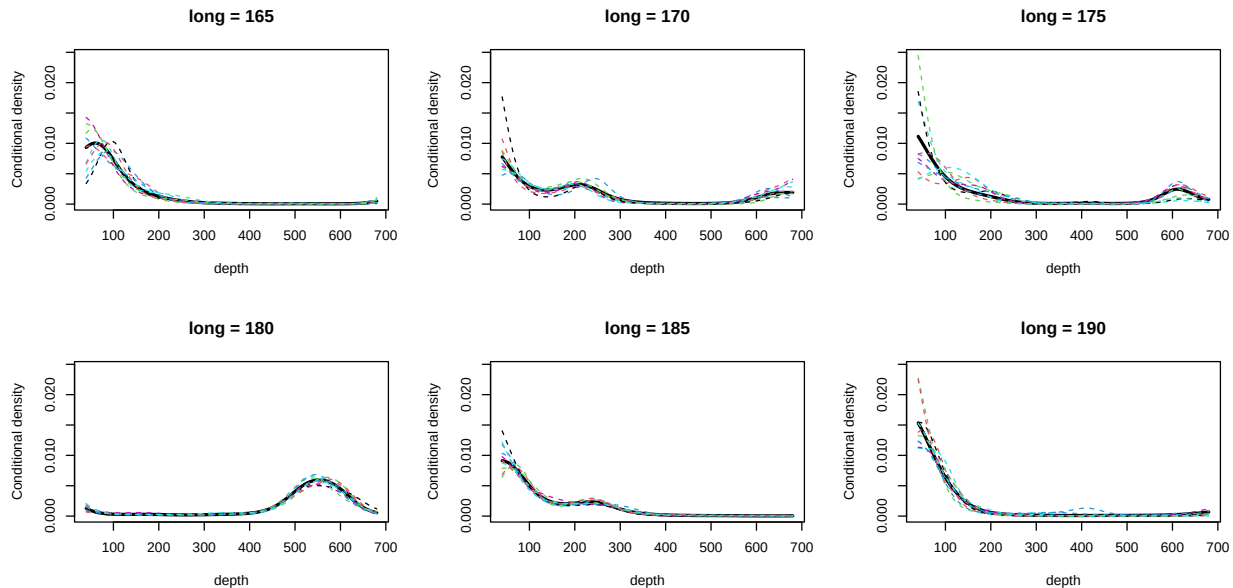


Figure 15: Posterior mean and draws of the conditional density of depth under MCMC, at six longitudes.

```

R> dev.copy(pdf, "../Figures/defaultMCMC.pdf", width = 10, height = 3.5)
pdf
3
R> dev.off()
pdf
2

```

Instead of showing a single estimate, we compare multiple posterior draws against histogram data, similarly to what we did for the Laplace approximation.

```

R> # Define the three selected values
R> pred <- predict(modelMCMC, newdata = dfGrid)
R> pred$meanpdf <- rowMeans(pred[, -c(1:3)])
R> pred$category <- names[pred$ID]
R>
R> df_plot <- pred %>%
+   dplyr::select(c("long", "depth", "category",
+                   paste0("pdf_", selected_cols)))%>%
+   pivot_longer(-c("long", "depth", "category"))
R>
R>
R> ggplot(mapping=aes(x = depth)) +

```

```

+   geom_histogram(df_filtered,
+                 mapping=aes(y=after_stat(density)),
+                 position = "identity", breaks = seq(40, 680, 20),
+                 fill="darkgrey", col="grey50", lwd=0.2, alpha=0.7) +
+   geom_rug(data=df_filtered, sides = "b", color = "navy", alpha = 0.5)+
+   geom_line(data=df_plot, mapping=aes(y=value, group=name),
+             color = "black", lwd=0.1, alpha=0.5)+
+   geom_line(data=pred, mapping=aes(y=meanpdf, group=category), color = "red")+
+   facet_wrap(~ category, scales = "free_y", nrow=1) +
+   labs(x = "Hypocentral depth (km)",
+        y = "Probability density",
+        title = "Binned 'depth' histograms by 'long' (width = 1) VS SLGP-MCMC estimators at bins cent
+   theme_bw()+
+   coord_cartesian(xlim=range_response,
+                   ylim=c(0, 0.02))

```

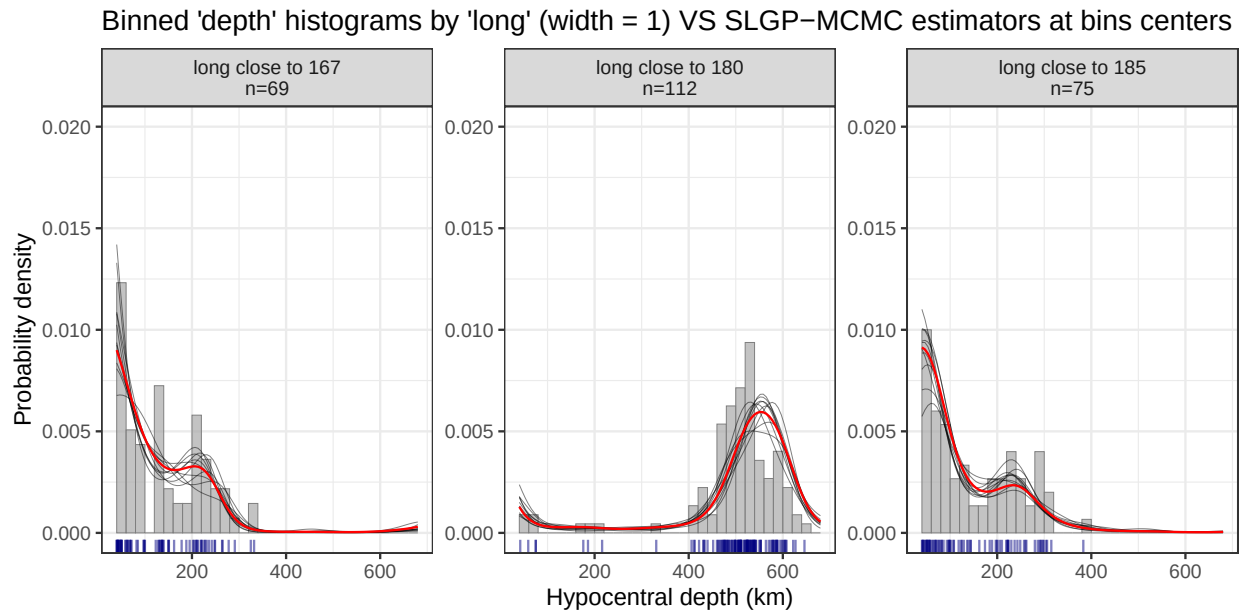


Figure 16: Posterior mean and draws of the conditional density under MCMC, at three longitudes, against the local histograms.

```
R> ggsave(paste0("./Figures/histMCMC", ".pdf"), width=10, height=3.5)
```

3.5 By-products of the estimation

One of the advantages of the SLGP framework is that it predicts entire probability density functions (PDFs) over space. This opens the door to a wide range of nonlinear inferences on the estimated field. In particular, we can compute and visualize functionals of the predicted densities, such as moments (mean, variance, skewness, etc.) or quantiles.

In our current implementation, we provide predictions for both centered and uncentered moments of the estimated PDFs at each location. These derived quantities can themselves be interpreted as spatial fields and are thus informative summaries of the underlying random process.

Importantly, when using probabilistic inference schemes like Laplace approximation or MCMC, the uncertainty in the SLGP predictions naturally propagates to these functionals. As a result, we can also quantify uncertainty

on the functionals. For example, we compute credible intervals for the mean or variance field. This is illustrated in the upcoming figure, where we use the MCMC-trained SLGP model to compute moments.

```
R> dfX <- data.frame(long=seq(range_x[1], range_x[2], 1))
R> # Uncentered moments : the mean
R> predMean <- predict(modelMCMC, type= "moments", newdata = dfX,
+                       power=c(1), centered=FALSE)
R> # Centered moments: variance, Kurtosis and Skewness
R> predVar <- predict(modelMCMC, type= "moments", newdata = dfX,
+                       power=c(2, 3, 4), centered=TRUE)
R>
R>
R> pred <- rbind(predMean, predVar)
R> pred <- pred %>%
+   pivot_longer(-c("long", "power")) %>%
+   pivot_wider(values_from = value, names_from = power) %>%
+   mutate(`3` = `3` / `2`^1.5, # skewness
+          `4` = `4` / `2`^2,   # kurtosis
+          `2` = sqrt(`2`)) %>% # s.d., after standardising
+   pivot_longer(-c("long", "name"), names_to = "power") %>%
+   data.frame()
R>
R> pred$power <- factor(c("Expected value",
+                        "Standard deviation",
+                        "Skewness", "Kurtosis")[as.numeric(pred$power)],
+                      levels=c("Expected value", "Standard deviation",
+                                "Skewness", "Kurtosis"))
R> df_plot <- pred %>%
+   group_by(long, power)%>%
+   summarise(q10 = quantile(value, probs=c(0.1)),
+             q50 = quantile(value, probs=c(0.5)),
+             q90 = quantile(value, probs=c(0.9)),
+             mean = mean(value), .groups="keep")%>%
+   ungroup() # summarise uncertainty
R>
R>
R> ggplot(df_plot, mapping=aes(x = long, group=power)) +
+   geom_ribbon(mapping = aes(ymin=q10, ymax=q90),
+             alpha = 0.25, lty=2, col="black", fill="cornflowerblue")+
+   geom_line(mapping=aes(y=q50))+
+   facet_wrap(~power,
+             scales = "free", nrow=1)+
+   labs(x = "Longitude (°)",
+        y = "Moment value") +
+   theme_bw()+
+   coord_cartesian(xlim=range_x)
```

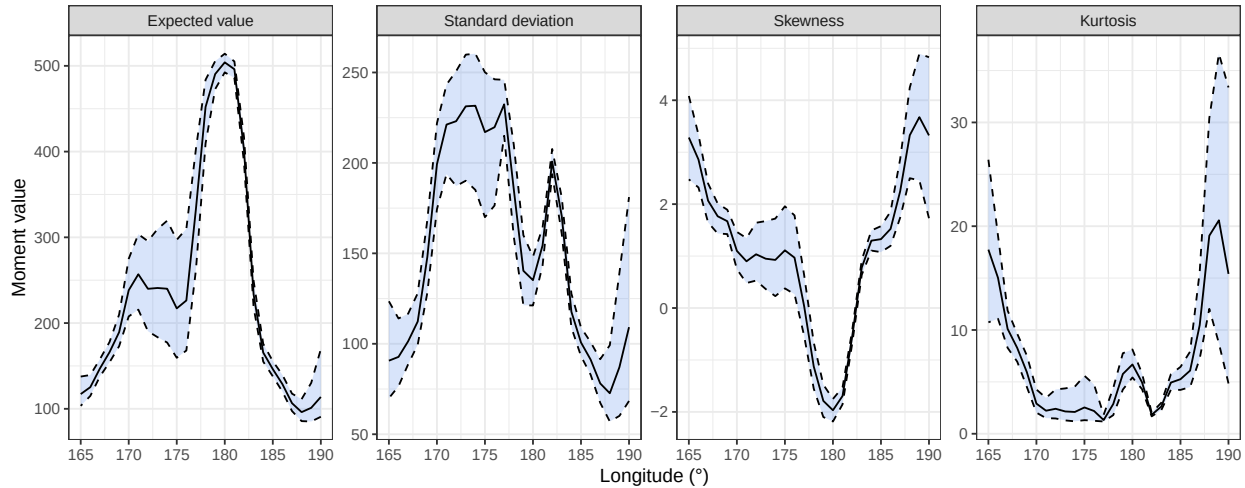


Figure 17: Conditional mean, standard deviation, skewness and kurtosis of depth across longitude, from the MCMC fit: posterior median and 10th-90th percentile band. All four are read off the same fitted object.

```
R> ggsave(paste0("./Figures/MomentsMCMC", ".pdf"), width=8, height=3)
R>
```

We also support the joint prediction of quantiles at arbitrary levels. Because quantiles are derived directly from the estimated densities, they are guaranteed to be consistent and non-crossing - a property that cannot always be ensured in standard quantile regression. This makes them reliable tools for summarizing distributional shape (e.g., asymmetry, spread) at each location. We conduct a sanity check by comparing SLGP-based quantile estimates with those obtained using classical quantile regression, as implemented in the `quantreg` package, and also empirical quantiles on a rolling window.

```
R> probsL <- c(5, 25, 50, 75, 95)/100
R> dfX <- data.frame(long = seq(range_x[1], range_x[2], 1))
R>
R> # SLGP prediction
R> pred <- predict(modelMAP, type = "quantiles",
+               newdata = dfX,
+               probs = probsL)
R> pred$probs <- factor(paste0("Quantile: ", 100*pred$probs, "%"),
+               levels = paste0("Quantile: ", 100*probsL, "%"))
R>
R> # Comparison 1: classical quantile regression (cubic polynomial)
R> library(quantreg)
R> long_center <- mean(df$long)
R> long_scale <- sd(df$long)
R> df$long2 <- (df$long-long_center)/long_scale
R> dfX$long2 <- (dfX$long-long_center)/long_scale
R>
R> fit_q <- rq(depth ~ poly(long2, degree=3, raw=TRUE),
+               data = df, tau = probsL, method="br")
R> pred_q <- predict(fit_q, newdata = dfX)
R> pred$qQuantileReg <- c(t(pred_q))
R>
R> # Comparison 2: empirical quantiles on bins, kept only when n >= 10
R> bin_w <- 1
R> min_n <- 10
```

```

R>
R> emp <- do.call(rbind, lapply(dfX$long, function(x0) {
+   in_win <- df$depth[df$long >= x0 & df$long <= x0 + bin_w]
+   ok <- length(in_win) >= min_n
+   data.frame(long = x0,
+               n = length(in_win),
+               probs = probsL,
+               q = if (ok) as.numeric(quantile(in_win, probs = probsL))
+               else rep(NA_real_, length(probsL)))
+ })))
R> emp$probs <- factor(paste0("Quantile: ", 100*emp$probs, "%"),
+                     levels = paste0("Quantile: ", 100*probsL, "%"))
R>
R> plot_slgp <- ggplot(pred, mapping = aes(x = long)) +
+   geom_line(mapping = aes(y = qSLGP_1, lty = "Model (SLGP or QR)",
+                           col = probs, group = probs), lwd = 0.75) +
+   geom_line(data = emp,
+             mapping = aes(y = q, col = probs, group = probs,
+                           lty = "Empirical"),
+             lwd = 0.9) +
+   labs(x = "Longitude [°E]", y = "Hypocentre depth [km]",
+        col = "Quantile levels", lty = "Quantile estimation method") +
+   theme_bw() +
+   coord_cartesian(xlim = range_x, ylim = range_response) +
+   scale_linetype_manual(values = c("Empirical" = 2,
+                                    "Model (SLGP or QR)" = 1))
R>
R> plot_qr <- ggplot(pred, mapping = aes(x = long)) +
+   geom_line(mapping = aes(y = qQuantileReg, lty = "Model (SLGP or QR)",
+                           col = probs, group = probs), lwd = 0.75) +
+   geom_line(data = emp,
+             mapping = aes(y = q, col = probs, group = probs,
+                           lty = "Empirical"),
+             lwd = 0.9) +
+   labs(x = "Longitude [°E]", y = "Hypocentre depth [km]",
+        col = "Quantile levels", lty = "Quantile estimation method") +
+   theme_bw() +
+   coord_cartesian(xlim = range_x, ylim = range_response) +
+   scale_linetype_manual(values = c("Empirical" = 2,
+                                    "Model (SLGP or QR)" = 1))
R>
R> plot_hist <- ggplot(df, mapping = aes(x = long)) +
+   geom_histogram(col = "navy", fill = "grey", alpha = 0.3,
+                 breaks = seq(range_x[1], range_x[2], 1)) +
+   theme_minimal() +
+   labs(x = NULL, y = "Sample\ncount")
R>
R> library(ggpubr)
R> p_with_marginals <- ggarrange(
+   plot_hist+ labs(title = "SLGP vs empirical quantiles"),
+   plot_hist+
+     labs(title = "Quantile regression from quantreg vs empirical quantiles"),
+   plot_slgp,
+   plot_qr,

```

```

+   ncol = 2,
+   nrow = 2,
+   heights = c(1.5, 4),
+   common.legend = TRUE,
+   legend = "bottom",
+   align = "hv"
+ )
R>
R>
R> print(p_with_marginals)

```

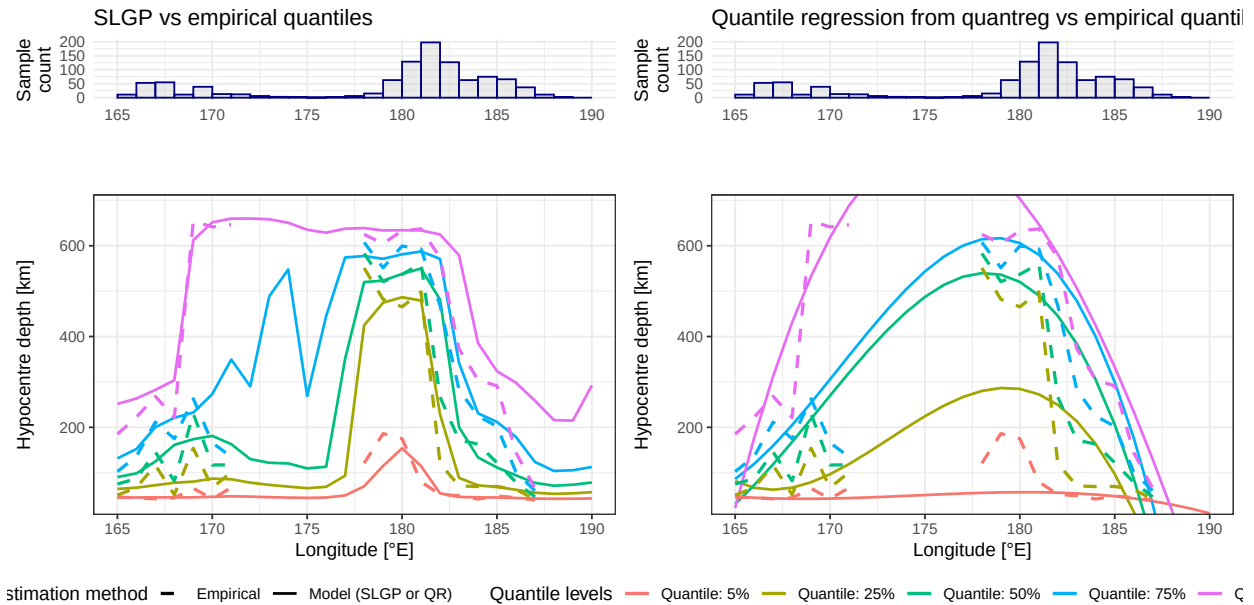


Figure 18: Simultaneous quantile prediction using an SLGP model (MAP), compared to quantile regression and empirical quantiles on bins with at least 10 events.

```

R> ggsave(p_with_marginals, file = "./Figures/QuantilesMAP2.pdf", width = 8, height = 5, scale=1.5)

```

There are disagreement between the methods. The three approaches mostly agree in the data-rich part of the domain, but differences arise where the data are sparse or the conditional distribution is bimodal. It seems the SLGP quantiles track the rolling-window empirical quantiles more faithfully there. The classical quantile regression curves depart markedly from the empirical estimates, and, more fundamentally, the quantreg estimates cross one another in places. This is not a matter of tuning: any method that estimates quantiles marginally, one level at a time and without an underlying density, can violate monotonicity across levels.

The SLGP quantiles, being obtained through the estimated density at each location, they are non-crossing by construction and remain ordered everywhere.

We can also get probabilistic prediction of these quantiles.

```

R> # SLGP prediction
R> probsL <- c(5, 25, 50, 75, 95) / 100
R>
R> pred <- predict(modelMCMC, type = "quantiles", newdata = dfX, probs = probsL)
R>
R> df_plot <- pred %>%
+   pivot_longer(-c("long", "probs")) %>%

```



```

+   group_by(long, probs) %>%
+   summarise(q10 = quantile(value, 0.1), q50 = quantile(value, 0.5),
+             q90 = quantile(value, 0.9), .groups = "drop") %>%
+   mutate(probs = factor(paste0("Quantile: ", 100 * probs, "%"),
+                         levels = paste0("Quantile: ", 100 * probsL, "%")))
R>
R> plot_slgp <- ggplot(df_plot, aes(x = long)) +
+   geom_ribbon(aes(ymin = q10, ymax = q90, fill = probs, group = probs),
+             alpha = 0.2, colour = NA) +
+   geom_line(aes(y = q50, col = probs, group = probs, lty = "SLGP"),
+            lwd = 0.75) +
+   geom_line(data = emp, aes(y = q, col = probs, group = probs, lty = "Empirical"),
+            lwd = 0.9) +
+   labs(x = "Longitude [°E]", y = "Hypocentre depth [km]",
+        col = "Quantile levels", fill = "Quantile levels",
+        lty = "Quantile estimation method") +
+   theme_bw() +
+   coord_cartesian(xlim = range_x, ylim = range_response) +
+   scale_linetype_manual(values = c("Empirical" = 2, "SLGP" = 1))
R>
R> p_with_marginals <- ggarrange(
+   plot_hist + labs(title = "SLGP posterior quantiles vs empirical quantiles"),
+   plot_hist + labs(title = "Quantile regression vs empirical quantiles"),
+   plot_slgp, plot_qr,
+   ncol = 2, nrow = 2,
+   heights = c(1.5, 4),
+   common.legend = TRUE,
+   legend = "bottom",
+   align = "hv"
+ )
R>
R> print(p_with_marginals)

```

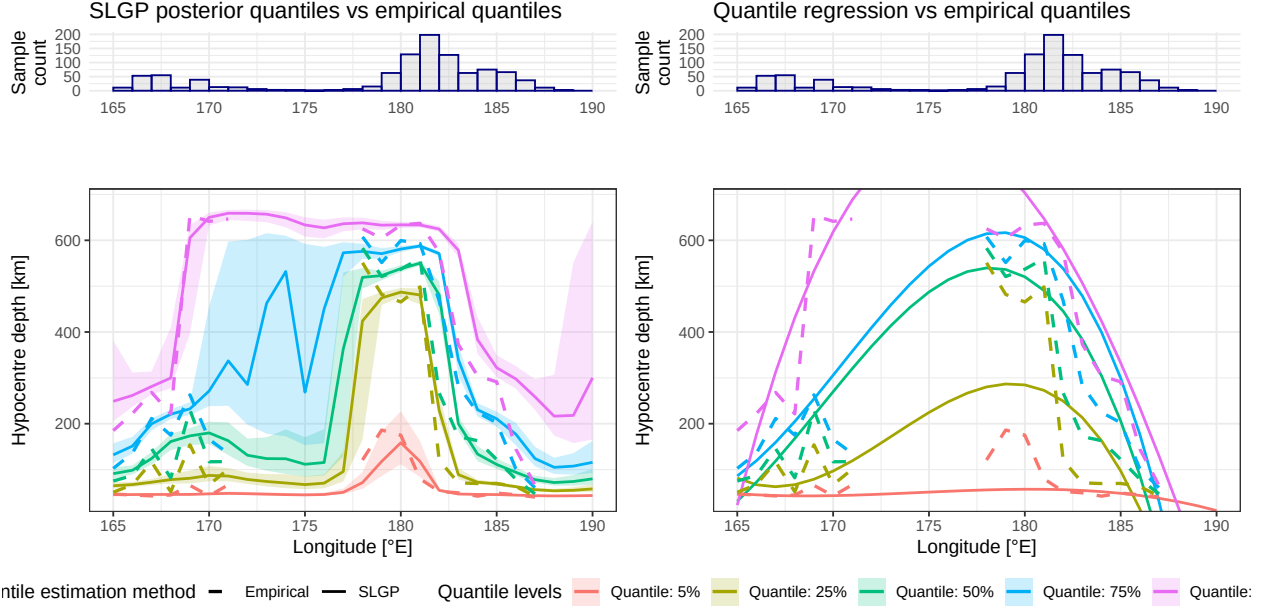


Figure 19: Posterior quantile prediction using an SLGP model estimated by MCMC, compared with quantile regression and empirical quantiles in bins with at least 10 events. Shaded bands represent posterior uncertainty for the SLGP quantiles.

```
R>
R> ggsave("./Figures/QuantilesMCMC2.pdf",
+         p_with_marginals, width = 8, height = 5, scale = 1.5)
R>
```

We can see that the uncertainty on the quantiles is larger where data are sparse.

3.6 Asymptotic behaviours

To assess the quality of SLGP-based density field estimation, we first define a reference field using the MAP estimate, which serves as the ground truth. We then generate synthetic data from this reference field and compare different estimation setups to evaluate their impact on accuracy.

3.6.1 Experiment description

We explore three main experimental factors, and for each setting we replicate the experiment 10 times with several seeds to account for randomness:

- 1. **Number of Basis Functions** : This experiment evaluates the impact of the number of Random Fourier Features (RFFs). A larger number of basis functions is expected to improve accuracy but increase computational cost. This setup allows us to study the approximation error induced by using a finite number of basis functions and assess the trade-off between speed and expressiveness.
- 2. **Length-scale Misspecification**: This experiment, we test the sensitivity of SLGP estimation to various specification of the length-scale hyperparameter. We train models using different length-scales. We consider length-scale that are too small: 0.01, 0.05; well-specified: 0.15; or too large: 0.25, 0.5. This analysis highlights the consequences of hyper-parameter selection.
- 3. **Sampling Schemes (Experiment 3)** This experiment investigates how the spatial structure of the data affects the quality of estimation. We vary the sampling scheme across different types: Uniform random sampling for the index; points restricted to regular grids (with 5, 11, or 21 points); "Hole-y" uniform random sampling with two excluded longitude bands, mimicking corridors left

uncovered by the seismic network. These variations reveal the method's ability to generalise in poorly observed regions.

This analysis provides practical insights into how SLGP estimation behaves under different conditions, guiding model selection for real applications.

```
R> modelRef <- modelMAP
R>
R> n_list <- c(10, 25, 50,
+             100, 250, 500,
+             1000, 2500, 5000,
+             10000, 25000, 50000,
+             100000)
R>
R> config1 <- expand.grid(1, c(25, 50, 100, 200, 300),
+                         seq(25), 0, 0.15, "Unif")
R> config2 <- expand.grid(2, 200,
+                         1, seq(25), c(0.01, 0.05, 0.15, 0.25, 0.5), "Unif")
R> config3 <- expand.grid(3, 200,
+                         1, seq(25), 0.15, c("Unif", "Clustered",
+                                             "Clustered2", "Clustered3",
+                                             "Hole-y"))
R> colnames(config1) <-
+   colnames(config2) <-
+   colnames(config3) <- c("Scenario", "nFreq", "SeedFreq",
+                           "SeedSamples", "lengthscale", "Sampling")
R> config1$SeedSamples <- config1$SeedFreq
R> config <- rbind(config1, config2, config3)
R> config <- config[, c("Scenario", "Sampling", "SeedSamples",
+                       "nFreq", "SeedFreq", "lengthscale")]
+ 
```

3.6.2 Evaluation of the performances

For each tested configuration, we compute the estimation error, comparing the inferred density field to the reference field. This allows us to measure the effect of different modeling choices. To assess the similarity between estimated and true distributions, we compute several integrated distances based on both the probability density functions (PDFs) and the cumulative distribution functions (CDFs).

The considered distances are defined for a distribution P_1 that admits a pdf f_1 and cdf F_1 and a distribution P_2 that admits a pdf f_2 and cdf F_2 as:

- Hellinger Distance $d_H(f_1, f_2) = \sqrt{\frac{1}{2} \int \left(\sqrt{f_1(t)} - \sqrt{f_2(t)} \right)^2 dt}$
- Total Variation Distance $TV(f_1, f_2) = \frac{1}{2} \int |f_1(t) - f_2(t)| dt$
- Kullback–Leibler Divergence $KL(f_1, f_2) = \int f_1(t) \log \left(\frac{f_1(t)}{f_2(t)} \right) dt$
- Wasserstein-1 Distance $W_1(F_1, F_2) = \int |F_1(t) - F_2(t)| dt$
- Kolmogorov–Smirnov Distance: $KS(F_1, F_2) = \sup_t |F_1(t) - F_2(t)|$
- Cramér–von Mises Distance: $CvM(F_1, F_2) = \int (F_1(t) - F_2(t))^2 f_1(t) dt$ for the f_1 -weighted version
 $CvM_{\text{sym}}(F_1, F_2) = \int (F_1(t) - F_2(t))^2 dt$ for the unweighted version.

These metrics are evaluated across a grid of values and provide different perspectives on the quality of the density estimation.

3.6.3 Code for the experiment

We generate multiple synthetic datasets from the reference field to evaluate model performance under each experimental scenario. To reduce memory usage, data generation is performed in slices.

```
R> n <- max(n_list)
R>
R> for(strat in unique(config$Sampling)){
+   for(SeedSamples in sort(unique(config$SeedSamples))){
+     #cat("Strategy ", strat, " Seed ", SeedSamples, "\n")
+     title <- paste0("./res/samp_", strat, "_num_", SeedSamples, ".Rdata")
+     samp <- data.frame()
+     if(strat == "Hole-y"){
+       title2 <- paste0("./res/samp_", "Unif",
+         "_num_", SeedSamples, ".Rdata")
+       load(title2)
+       samp <- samp[abs(samp$long-172.5)>=2.5 & abs(samp$long-185)>=1, ]
+       samp <- samp[1:50000, ]
+       save(samp, file=title)
+     }
+     if(!(file.exists(title))){
+       for(slice in seq(25)){
+         cat("Strategy ", strat, " Seed ",
+           SeedSamples, "slice", slice, "/25\n")
+         current_seed <- SeedSamples*1000+slice
+         set.seed(current_seed)
+         if(strat == "Unif"){
+           newX <- data.frame(long=runif(n/25, range_x[1], range_x[2]))
+           nsamp=1
+         }
+         if(strat == "Clustered"){
+           x <- sample(seq(range_x[1], range_x[2],, 21),
+             size=n/25, replace=T)
+           t <- table(x)
+           newX <- data.frame(long=as.numeric(names(t)))
+           nsamp <- as.vector(t)
+         }
+         if(strat == "Clustered2"){
+           x <- sample(seq(range_x[1], range_x[2],, 11),
+             size=n/25, replace=T)
+           t <- table(x)
+           newX <- data.frame(long=as.numeric(names(t)))
+           nsamp <- as.vector(t)
+         }
+         if(strat == "Clustered3"){
+           x <- sample(seq(range_x[1], range_x[2],, 5),
+             size=n/25, replace=T)
+           t <- table(x)
+           newX <- data.frame(long=as.numeric(names(t)))
+           nsamp <- as.vector(t)
+         }
+         temp <- simulate(modelMAP,
+           newdata = newX,
+           seed=current_seed*2,
+           nsim=nsamp,
```

```

+             interpolateBasisFun = "WNN")
+       samp <- rbind(samp, temp[sample(seq(nrow(temp))), ])
+     }
+     save(samp, file=title)
+     gc()
+   }
+ }
+ }
R>

```

Next, we fit SLGP models to these synthetic datasets using varying hyperparameters and configurations to assess how different modeling choices impact estimation quality.

```

R> dfGrid <- data.frame(expand.grid(seq(range_x[1], range_x[2],, 101),
+             seq(range_response[1], range_response[2],, 101)))
R> colnames(dfGrid) <- c("long", "depth")
R> dt <- diff(sort(unique(dfGrid$depth))[1:2])
R> predRef <- predict(modelMAP, newdata = dfGrid)
R> predRefCDF <- predict(modelMAP, newdata = dfGrid, type = "cdf")
R> predRefCDF <- round(predRefCDF, 15)
R> df_compute <- predRef
R> colnames(df_compute)[3] <- "TruePDF"
R> df_compute$TrueCDF <- predRefCDF$cdf_1
R> df_compute$TruePDFsafe <- df_compute$TruePDF + 1e-15
R> rm(predRefCDF, predRef)
R>
R> pb <- txtProgressBar(min = 0, max = nrow(config), style = 3)
R>
R>
R> for(i in seq(nrow(config))){
+   Scenario <- config$Scenario[i]
+   nFreq <- config$nFreq[i]
+   SeedFreq <- config$SeedFreq[i]
+   SeedSamples <- config$SeedSamples[i]
+   lengthscale <- config$lengthscale[i]
+   Sampling <- config$Sampling[i]
+
+   title <- paste0("./res/samp_", Sampling, "_num_",
+     SeedSamples, ".Rdata")
+   if(file.exists(title)){
+
+     load(file=title)
+
+     set.seed(SeedFreq)
+     title2 <- paste0("./res/SLGP_len_", 100*lengthscale,
+       "_nFreq_", nFreq,
+       "_rep_", SeedFreq, ".Rdata")
+     if(!file.exists(title2)){
+       modelPrior <- slgp(depth~long,
+         data=samp[1:n_list[1], ],
+         method="none",
+         basisFunctionsUsed = "RFF",
+         interpolateBasisFun="WNN",
+         hyperparams = list(lengthscale=c(lengthscale,
+           lengthscale),

```

```

+                                     sigma2=1),
+                                     sigmaEstimationMethod = "heuristic",
+                                     predictorsLower= c(range_x[1]),
+                                     predictorsUpper= c(range_x[2]),
+                                     responseRange= range_response,
+                                     opts_BasisFun = list(nFreq=nFreq,
+                                     MatParam=5/2),
+                                     seed=SeedFreq)
+ save(modelPrior, file=title2)
+ cat("\nCreated and saved model ", title2, "\n")
+ }else{
+   load(title2)
+ }
+ title3 <- paste0("./res/strat_", Sampling,
+                 "_SLGP_nFreq_", nFreq,
+                 "_seedFreq_", SeedFreq,
+                 "_len_", lengthscale*100, "%",
+                 "_rep_", SeedSamples, ".Rdata")
+
+ if(!file.exists(title3)){
+   d_list <- data.frame()
+   if(Sampling=="Hole-y"){
+     n_list2 <- n_list[1:(length(n_list)-1)]
+   }else{
+     n_list2 <- n_list
+   }
+   for(j in seq_along(n_list2)){
+     start_time <- Sys.time()
+     cat(".")
+     n <- n_list2[j]
+     modelCurrent <- update(modelPrior, newdata = samp[1:n, ], method="MAP",
+                           sigmaEstimationMethod = "heuristic")
+     predTemp <- predict(modelCurrent, newdata = dfGrid)
+     predTempCDF <- predict(modelCurrent, type = "cdf", newdata = dfGrid)
+     predTempCDF <- round(predTempCDF, 15)
+     df_compute$estPDF <- predTemp$pdf_1
+     df_compute$estPDFsafe <- df_compute$estPDF + 1e-15
+     df_compute$estCDF <- predTempCDF$cdf_1
+
+     # ggplot(df_compute, mapping=aes(x=long, y=depth, fill=TruePDF-estPDF))+
+     #   geom_raster()+theme_bw()+scale_fill_viridis()+#limits = c(-0.12, 0.12))+
+     #   labs(fill="pdf")
+
+     distances <- df_compute %>%
+       group_by(long)%>%
+       summarise(H=sqrt(0.5* dt*
+                       sum((sqrt(estPDF) - sqrt(TruePDF))^2)),
+                 TV = 0.5 * sum(abs(estPDF - TruePDF)) * dt,
+                 KL_TE = sum(TruePDFsafe *
+                             log(TruePDFsafe / estPDFsafe)) * dt,
+                 KL_ET = sum(estPDFsafe *
+                             log(estPDFsafe / TruePDFsafe)) * dt,
+                 W1 = sum(abs(TrueCDF - estCDF)) * dt,
+                 KS = max(abs(TrueCDF - estCDF)),

```

```

+           CvM = sum((TrueCDF - estCDF)^2 * TruePDF) * dt,
+           CvMsym = sum((TrueCDF - estCDF)^2) * dt)%>%
+           data.frame()
+           distances$n <- n
+           end_time <- Sys.time()
+           diff_minutes <- as.numeric(difftime(end_time, start_time, units = "mins"))
+           distances$time <- diff_minutes
+
+           d_list <- rbind(d_list, distances)
+       }
+
+       save(d_list, file=title3)
+       cat(title3, "(",round(sum(d_list$time)/101, 2), "m)\n")
+       setTxtProgressBar(pb, i) # Update progress
+       gc()
+   }
+ }
+ }
R> close(pb)
R>
R>

```

Finally, we combine all results into a single data frame, which is then ready for further analysis and visualization.

```

R> df_res <- data.frame()
R>
R> for(i in seq(nrow(config))){
+   Scenario <- config$Scenario[i]
+   nFreq <- config$nFreq[i]
+   SeedFreq <- config$SeedFreq[i]
+   SeedSamples <- config$SeedSamples[i]
+   lengthscale <- config$lengthscale[i]
+   Sampling <- config$Sampling[i]
+   title3 <- paste0("./res/strat_", Sampling,
+                     "_SLGP_nFreq_", nFreq,
+                     "_seedFreq_", SeedFreq,
+                     "_len_", lengthscale*100, "%",
+                     "_rep_", SeedSamples, ".Rdata")
+
+   if(file.exists(title3)){
+     load(file=title3)
+
+     temp <- data.frame(Scenario=Scenario,
+                        nFreq=nFreq,
+                        SeedFreq=SeedFreq,
+                        SeedSamples=SeedSamples,
+                        lengthscale=lengthscale,
+                        Sampling=Sampling)
+     if(!is.null(d_list$nFreq)){ d_list$nFreq <- NULL}
+     temp <- cbind(temp, d_list)
+     df_res <- rbind(df_res, temp)
+   }
+ }

```

```

R> df_res <- df_res %>%
+   group_by(Scenario, nFreq, SeedFreq, SeedSamples, lengthscale, Sampling)%>%
+   mutate(time=ifelse(var(time)<=1e-15, NA, time)) %>%
+   ungroup()%>%
+   data.frame()
R>

```

3.6.4 Results of the experiment

3.6.4.1 Impact of the number of basis functions The following figure summarizes the effect of the number of basis functions (nFreq) on the quality of SLGP density estimation. For Scenario 1, we compute several integrated distances between the estimated density and the reference field across a range of training sample sizes. The shaded ribbons represent the 25th to 75th percentile range across multiple repetitions, while the solid line shows the mean value.

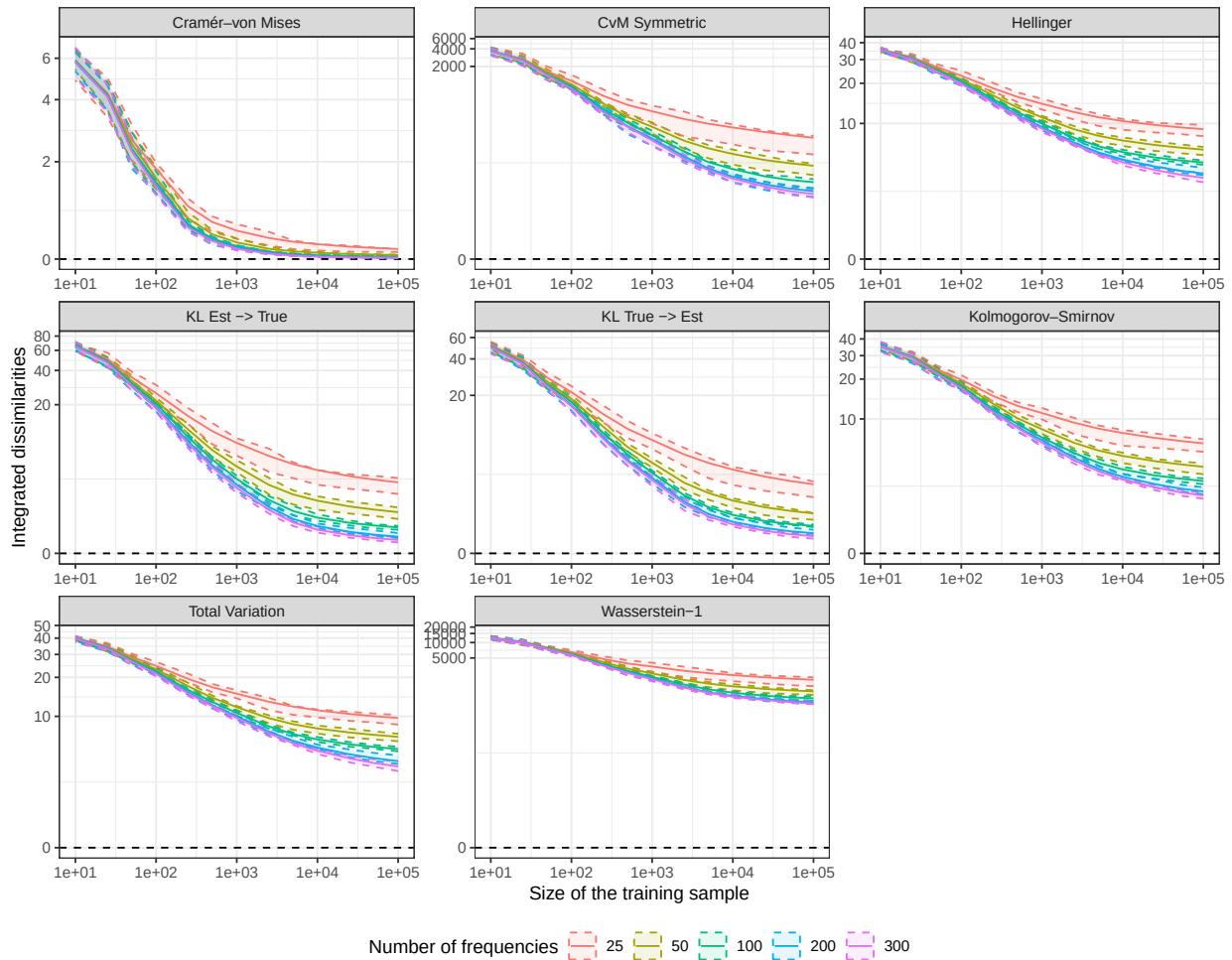


Figure 20: Effect of the number of basis functions on SLGP density estimation accuracy (Scenario 1).

As expected, increasing the number of Random Fourier Features improves the accuracy of the estimation. As we increase nFreq, the estimates converge: beyond a certain threshold (e.g., nFreq = 200, the well-specified setting), additional basis functions provide negligible improvements.

Interestingly, while the Kullback–Leibler divergence and Cramér–von Mises distance show rapid convergence

toward zero, other metrics such as the Hellinger distance, Total Variation distance, Wasserstein-1 distance, and Kolmogorov–Smirnov distance improve more gradually, highlighting differences in sensitivity between the considered dissimilarity measures. Specifically, KL and CvM are known to be more sensitive to overall distributional differences, particularly in the bulk of the distribution, whereas Hellinger, TV, and Wasserstein-1 capture local discrepancies and shape variations more finely, which shows here.

3.6.4.2 Impact of the length-scale selection We investigate the impact of length-scale selection on the quality of SLGP density estimation. In this experiment, we use a uniformly sampled design and keep the number of basis functions fixed at 200, consistent with the reference field. We then vary the sample size and evaluate the effect of different length-scale hyperparameters on the integrated distributional dissimilarities.

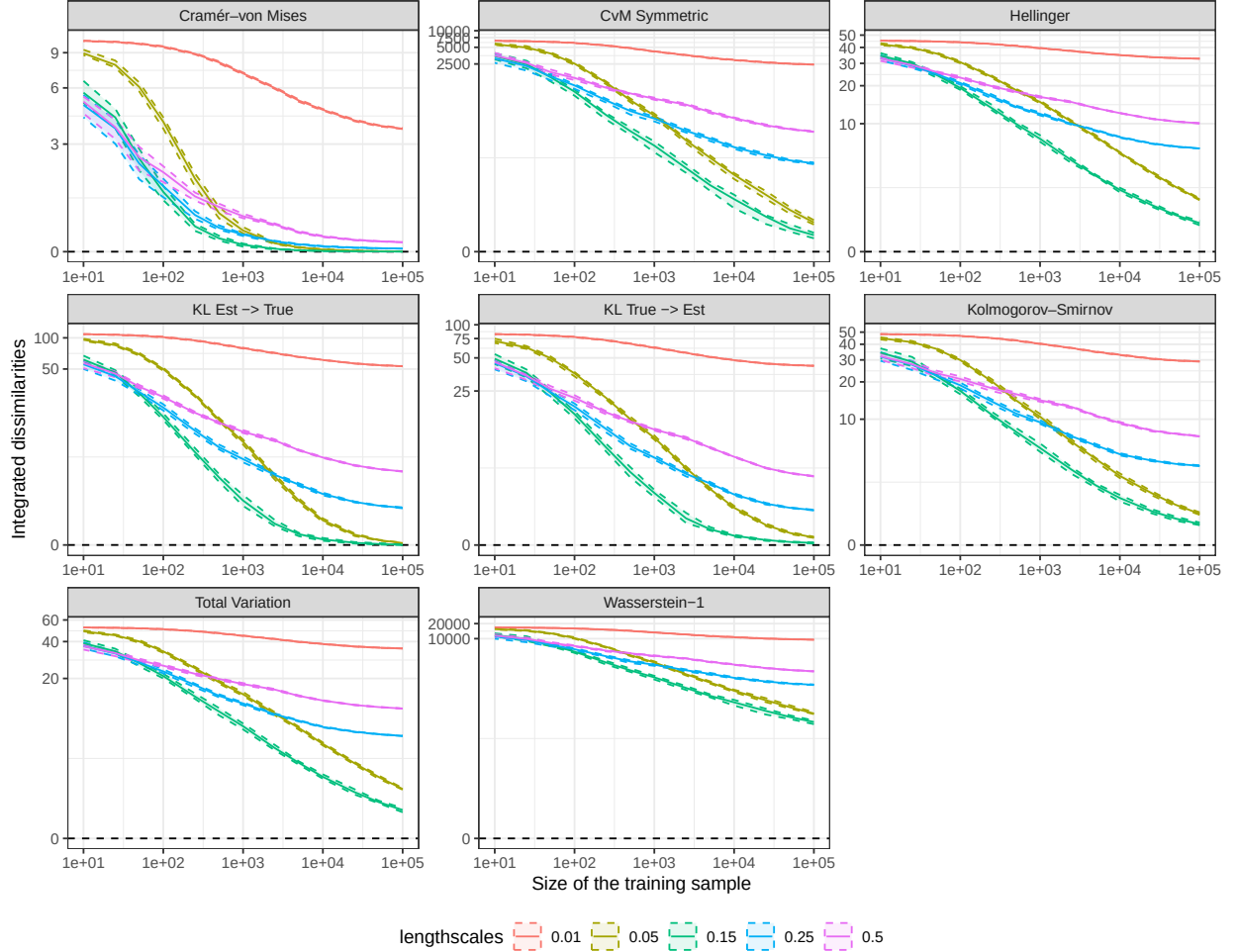


Figure 21: Effect of the length-scale specification on SLGP density estimation accuracy (Scenario 2).

As expected, the best performance across all dissimilarities is achieved using the true lengthscale or values close to it. Deviations from this value lead to slower convergence, with lengthscales that are too small performing worst. This behavior arises because small lengthscales reduce the model’s ability to generalise, resulting in overly localized predictions that fail to capture broader trends in the data.

These results align with our previous observations when experimenting with small and large length-scales, confirming that selecting a reasonable, data-informed length-scale is crucial for reliable SLGP estimation.

3.6.4.3 Effect of the sampling scheme In this experiment, we study how the choice of sampling strategy for `long` impacts the quality of field reconstruction.

Unlike the previous analysis, here we look at localised errors as well as integrated ones: instead of summarising performance across the entire domain, we also examine how the estimation error varies as a function of the location in `.` This setting highlights whether some regions are systematically poorly estimated under certain sampling designs.

We consider the following sampling schemes:

- Very Sparse Grid: Points sampled uniformly on a regular grid of 5 points spanning the transect.
- Medium Grid: Points sampled uniformly on a regular grid of 11 points.
- Fine Grid: Points sampled uniformly on a regular grid of 21 points.
- Uniform : Points sampled uniformly at random over the whole transect.
- Uniform with holes: Points sampled uniformly outside two excluded longitude bands, mimicking unmonitored corridors.

For all scenarios, we use 200 basis functions and a well-specified length-scale, consistent with the reference setting. By comparing the estimated densities across these designs, we can assess the effect of sample coverage and spacing on the accuracy of SLGP predictions.

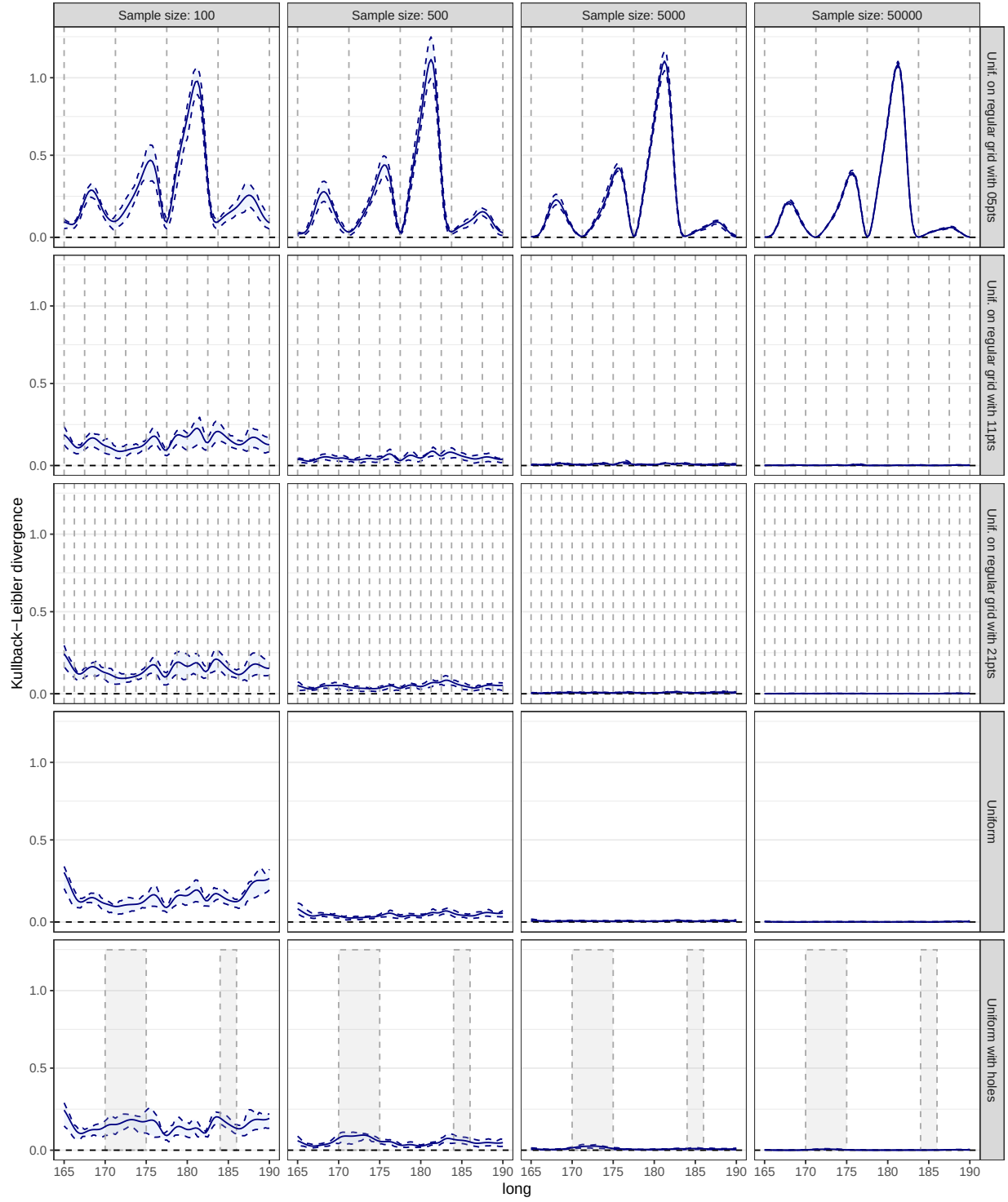


Figure 22: Effect of the sampling localisation on SLGP density estimation accuracy (Scenario 3, Kullback-Leibler divergence).

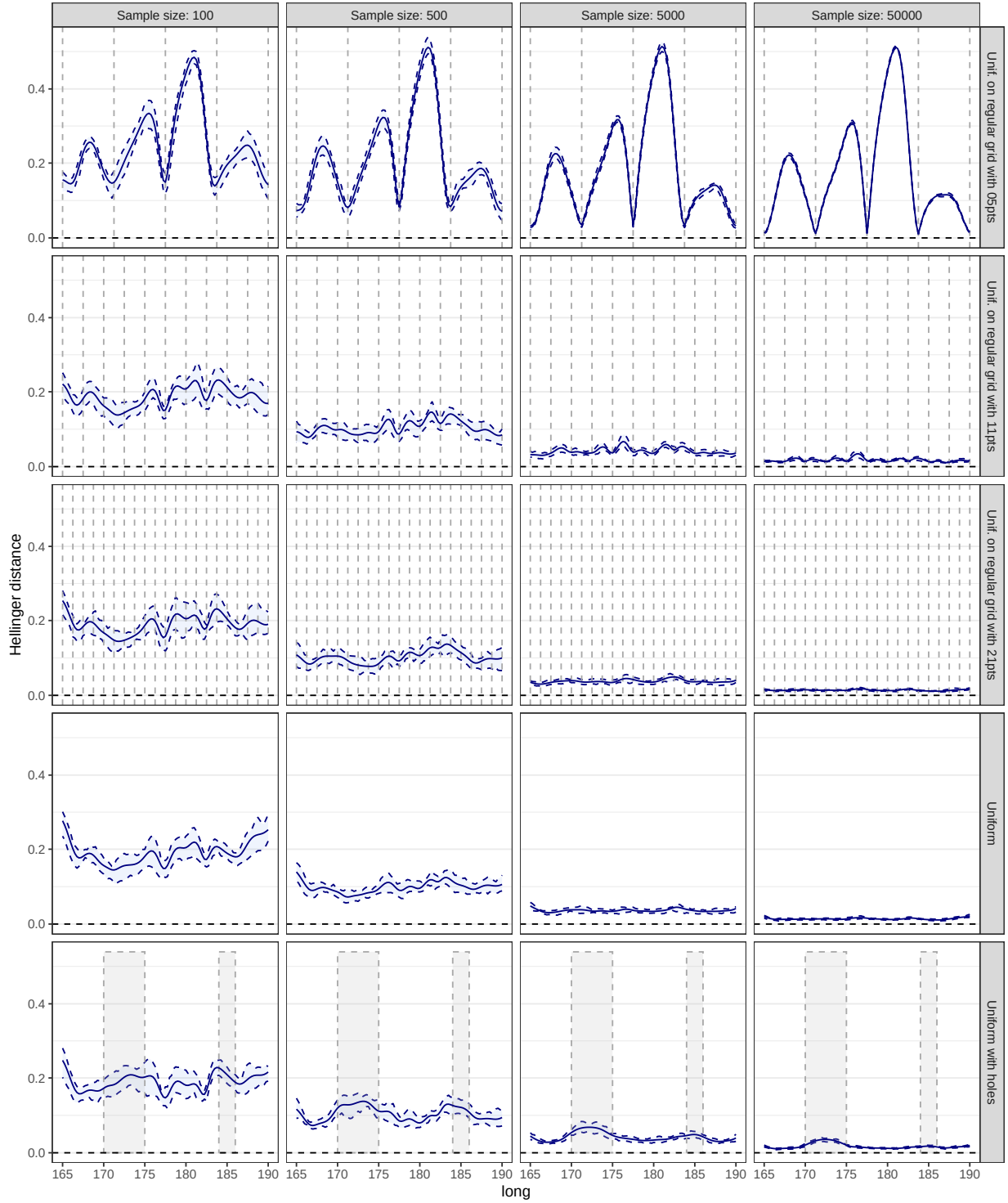


Figure 23: Effect of the sampling localisation on SLGP density estimation accuracy (Scenario 3, Hellinger distance).

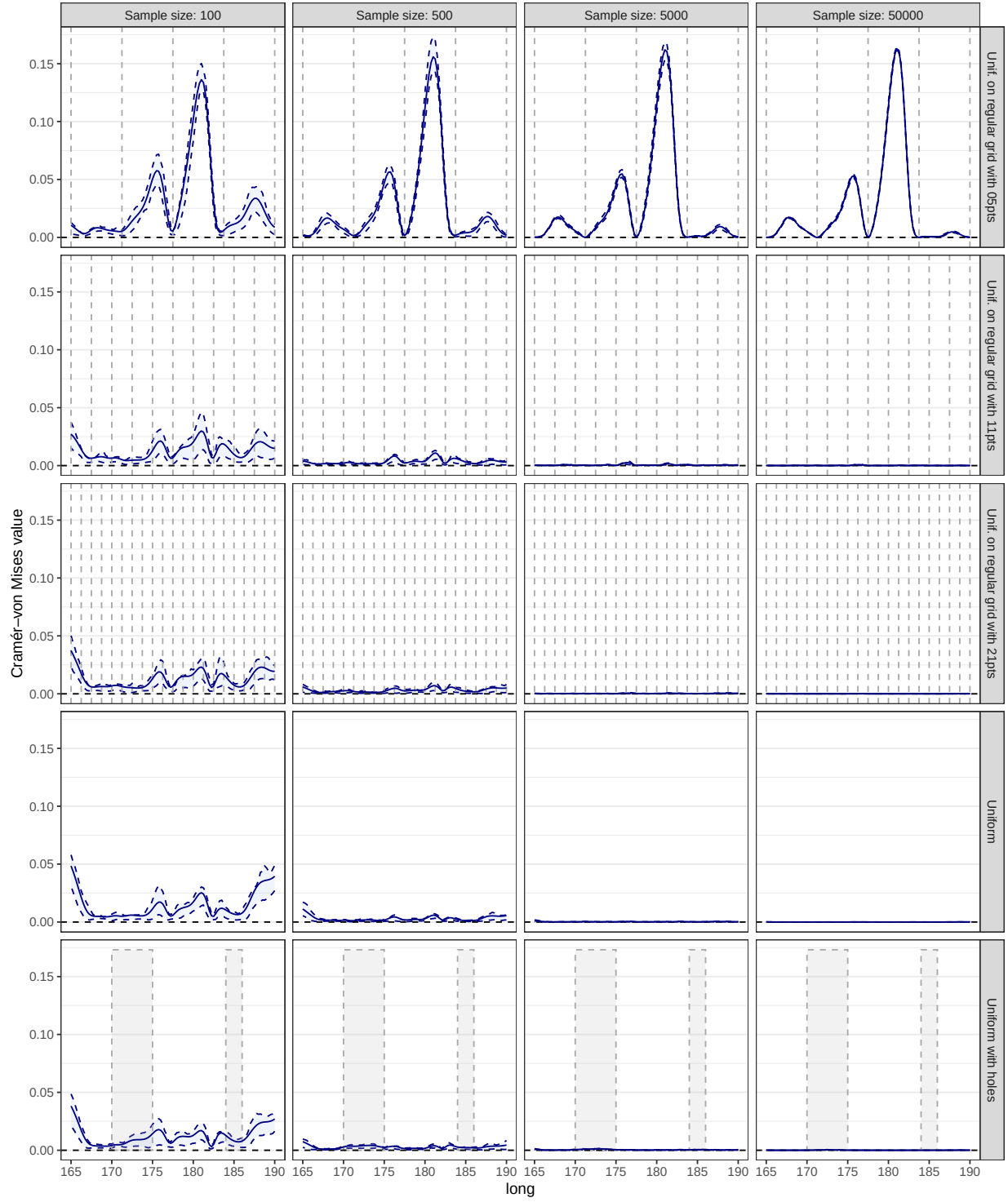


Figure 24: Effect of the sampling localisation on SLGP density estimation accuracy (Scenario 3, Cramér-von Mises).

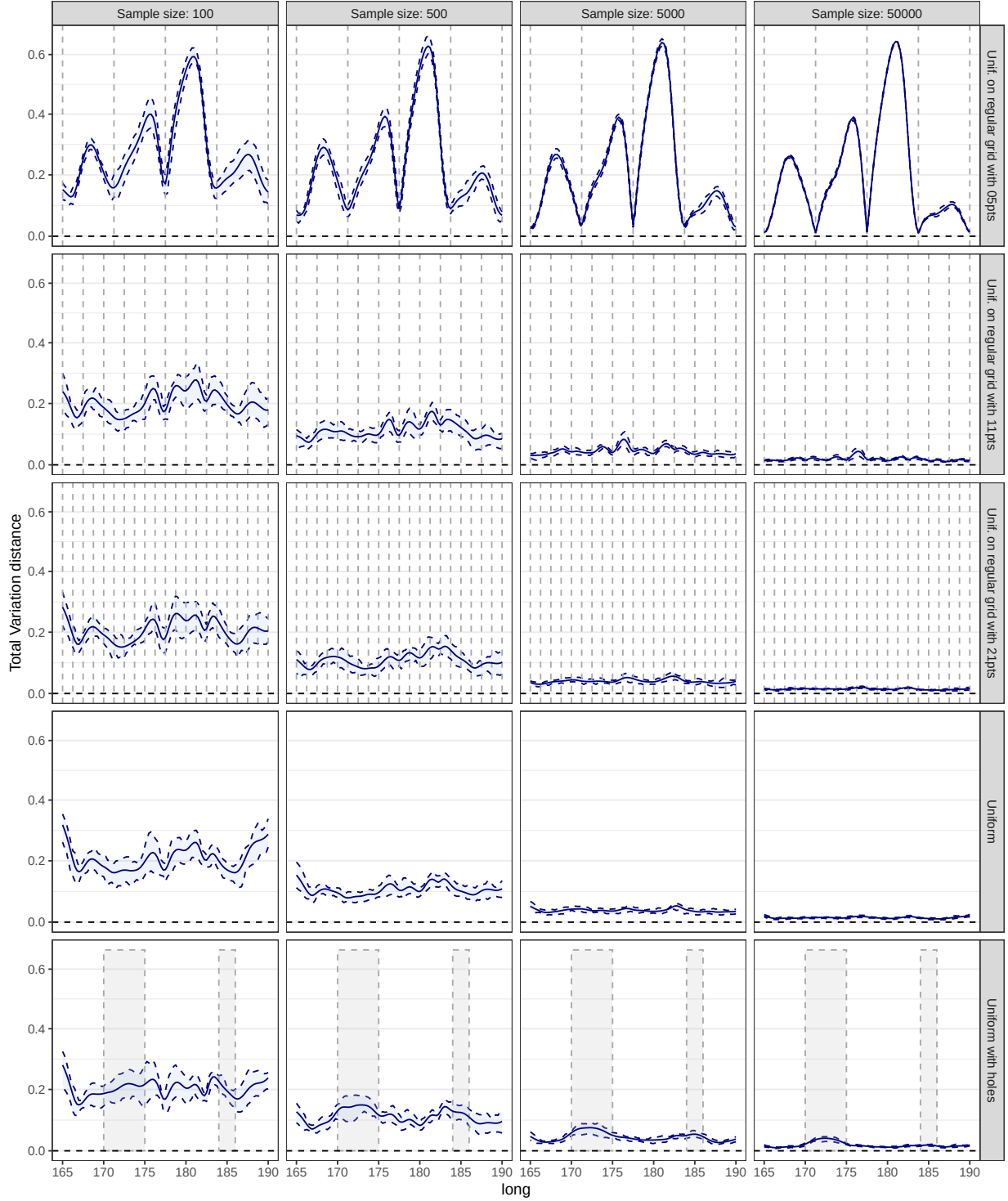


Figure 25: Effect of the sampling localisation on SLGP density estimation accuracy (Scenario 3, Total Variation distance).

As expected, sampling strategies with gaps in the `long` space (such as the Uniform with holes and the Very Sparse Grid) lead to worse local performance in the poorly sampled regions. In these areas, the SLGP is unable to generalise, producing higher estimation errors.

For the Very Sparse Grid, increasing the number of training points does not improve the estimation beyond a certain threshold due to the limited coverage of the domain. To facilitate comparison, we also provide a figure excluding this design, allowing the other sampling strategies to be visualized on a sensible scale.

We also look at the integrated dissimilarities with varying sample sizes.

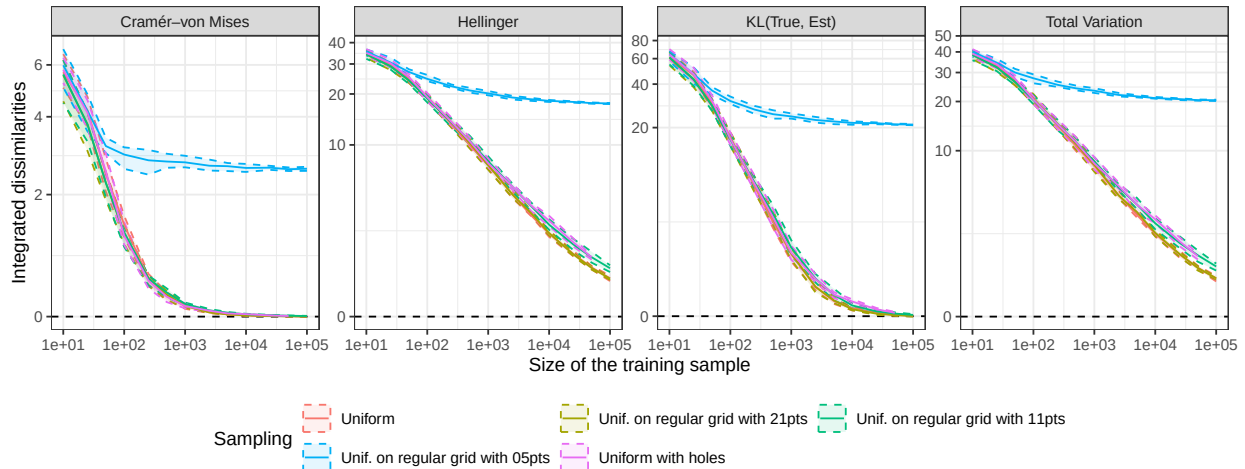


Figure 26: Effect of the sampling localisation on SLGP density estimation accuracy (Scenario 3).

These results clearly illustrate the impact of localization: dense coverage in some areas does not compensate for missing information elsewhere.

This analysis suggests a practical guideline: if the goal is to learn the full field rather than targeting specific locations, it is preferable to distribute sampling points evenly across the space rather than accumulating them at selected covariates. However if the goal is to refine knowledge at prescribed locations, intensifying sampling in the area leads to better local predictions.

3.7 Computational cost

The experiments above measure how well the SLGP estimates the field, let us now check what it costs. A fit has two phases, and `timing()` reports them separately:

- **setup** : normalising the data, pre-computing the quadrature and evaluating the basis functions at the nodes,
- **estimation** : the optimisation or the sampling itself.

Two choices drive the cost: the *integral approximation scheme* (and the number of observations), which decides how many normalising integrals the model works with, and the *estimation method*, which decides how many times they are evaluated.

3.7.1 Design

We cross the sample size and the rank with each of the two choices in turn, around a baseline of `interpolateBasisFun = "WNN"` and `method = "MAP"`:

factor	values
sample size <code>n</code>	100, 1000, 10000
rank <code>p = 2 nFreq</code>	50, 100, 200, 500
scenario 1: integral scheme, at MAP	<code>nothing</code> , NN, WNN

factor	values
scenario 2: estimation method, at WNN	MAP, Laplace, MCMC

The length-scale is held at 0.15 and the kernel at Matérn 5/2, which is the well-specified setting for these synthetic data. Each configuration is replicated 10 times, with a different sample and a different draw of random Fourier frequencies, so the spread shown below includes the variability of the basis and not only that of the data.

```
R> timing_n      <- c(100, 1000, 10000)
R> timing_sch    <- c("nothing", "NN", "WNN")
R> timing_meth   <- c("MAP", "Laplace", "MCMC")
R> timing_reps   <- seq(10)
R> timings_nfreq <- c(25, 50, 100, 250)
R>
R> config1 <- expand.grid(n = timing_n,
+                        scheme = timing_sch,
+                        method = "MAP",
+                        rep     = timing_reps,
+                        nfreq   = timings_nfreq,
+                        stringsAsFactors = FALSE)
R> config2 <- expand.grid(n = timing_n,
+                        scheme = "WNN",
+                        method = timing_meth,
+                        rep     = timing_reps,
+                        nfreq   = timings_nfreq,
+                        stringsAsFactors = FALSE)
R> config_time <- unique(rbind(config1, config2))
R> nrow(config_time)
[1] 600
```

The loop below is not run when knitting: it takes several hours, dominated by the MCMC fits. It checkpoints every ten fits and resumes where it stopped.

```
R> timing_file <- "./res/timings.Rdata"
R> checkpoint_every <- 10
R>
R> if (!file.exists(timing_file)) {
+   t_list <- data.frame()
+ }else{
+   load(timing_file)
+ }
R> if(nrow(t_list)< nrow(config_time)){
+   pb <- txtProgressBar(min = 0, max = nrow(config_time), style = 3)
+   since_save <- 0
+   for (i in seq(nrow(t_list)+1, nrow(config_time))) {
+     cfg <- config_time[i, ]
+     load(file = paste0("./res/samp_Unif_num_", cfg$rep, ".Rdata"))
+     set.seed(cfg$rep)
+     fit <- slgp(depth ~ long,
+               data = samp[1:cfg$n, ],
+               method = cfg$method,
+               basisFunctionsUsed = "RFF",
+               interpolateBasisFun = cfg$scheme,
+               hyperparams = list(lengthscale = rep(0.15, 2),
```



```

+                                     sigma2 = 1),
+                                     sigmaEstimationMethod = "heuristic",
+                                     predictorsLower = range_x[1], predictorsUpper = range_x[2],
+                                     responseRange = range_response,
+                                     opts_BasisFun = list(nFreq = cfg$nfreq, MatParam = 5 / 2),
+                                     seed = cfg$rep)
+
+   tm <- timing(fit)
+   tm$ndistinct <- nrow(unique(samp[1:cfg$n, "long", drop = FALSE]))
+   tm$scheme <- cfg$scheme
+   tm$rep <- cfg$rep
+   t_list <- rbind(t_list, tm)
+
+   cat(sprintf(
+     "\n[%3d/%3d] %-7s %-7s n=%6d p=%4d setup %6.2fs est %7.2fs",
+     i, nrow(config_time), cfg$method, cfg$scheme, tm$nobs, tm$p,
+     tm$setup, tm$estimation))
+   setTxtProgressBar(pb, i)
+   rm(fit); gc()
+   since_save <- since_save + 1L
+   if (since_save >= checkpoint_every) {
+     save(t_list, file = timing_file)
+     since_save <- 0L
+     cat(sprintf("\n    ... checkpoint saved (%d rows)\n", nrow(t_list)))
+   }
+ }
+ close(pb)
+ save(t_list, file = timing_file)
+ }

R> load("./res/timings.Rdata") # t_list: one row per fit, from timing()

```

3.7.2 The integral approximation scheme

```

R>
R> rank_levels <- paste("p =", sort(unique(t_list$p)))
R>
R> df_tot <- t_list %>%
+   filter(method == "MAP") %>%
+   mutate(scheme = factor(scheme, levels = c("nothing", "NN", "WNN")),
+          p_lab = factor(paste("p =", p), levels = rank_levels))
R>
R> ggplot(df_tot, aes(x = nobs, y = total, colour = scheme, fill = scheme)) +
+   stat_summary(fun = median, geom = "line", linewidth = 0.8) +
+   stat_summary(fun = median, geom = "point", size = 1.5) +
+   stat_summary(fun.min = function(z) quantile(z, 0.1),
+                fun.max = function(z) quantile(z, 0.9),
+                geom = "ribbon", alpha = 0.2, colour = NA) +
+   facet_wrap(~ p_lab, nrow = 1) +
+   scale_x_log10(breaks = c(100, 1000, 10000),
+                labels = c("100", "1000", "10000")) +
+   scale_y_log10() +
+   labs(x = "Training sample size", y = "Total fitting time [s]",
+        colour = "Integral approximation scheme",
+        fill = "Integral approximation scheme") +

```

```
+ theme_bw() + theme(legend.position = "bottom")
```

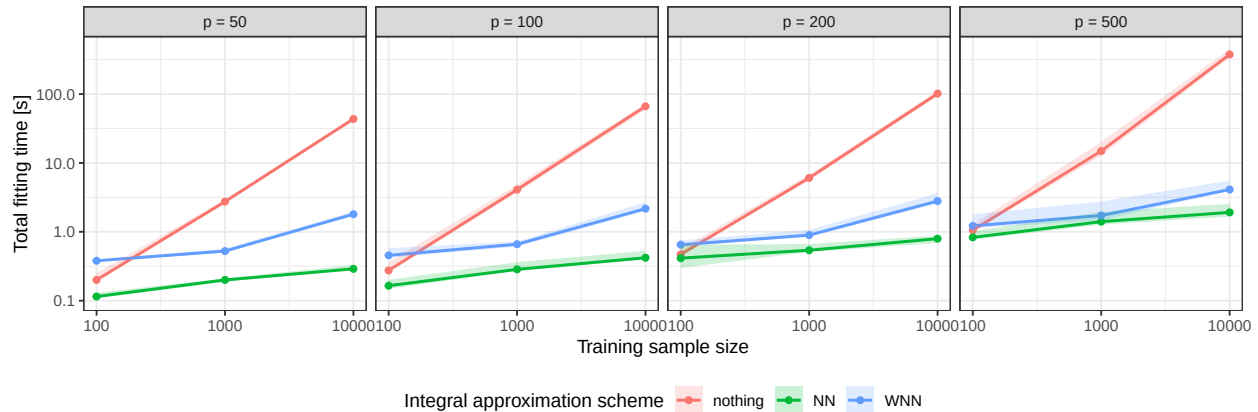


Figure 27: Total time of a MAP fit against the sample size, for the three integral approximation schemes and four ranks. Lines are medians over 10 replicates, ribbons the 10th-90th percentiles. Both axes are logarithmic.

```
R> ggsave("./Figures/timingTotalScheme.pdf", width = 10, height = 3.5)
R> ## speed-up of the interpolated schemes over the exact one, total time
R> t_list %>%
+   filter(method == "MAP") %>%
+   group_by(nobs, p, scheme) %>%
+   summarise(total = median(total), .groups = "drop") %>%
+   pivot_wider(names_from = scheme, values_from = total) %>%
+   mutate(speedup_NN = round(nothing / NN, 1),
+          speedup_WNN = round(nothing / WNN, 1)) %>%
+   data.frame()
  nobs  p  NN  WNN nothing speedup_NN speedup_WNN
1   100 50 0.115 0.380   0.200         1.7         0.5
2   100 100 0.165 0.455   0.275         1.7         0.6
3   100 200 0.415 0.650   0.470         1.1         0.7
4   100 500 0.830 1.220   1.055         1.3         0.9
5  1000 50 0.200 0.525   2.745        13.7         5.2
6  1000 100 0.285 0.660   4.095        14.4         6.2
7  1000 200 0.540 0.895   6.055        11.2         6.8
8  1000 500 1.405 1.725  14.880         10.6         8.6
9 10000 50 0.290 1.800  43.480       149.9        24.2
10 10000 100 0.420 2.165  66.445       158.2        30.7
11 10000 200 0.795 2.795 101.550       127.7        36.3
12 10000 500 1.920 4.110 375.975       195.8        91.5
```

The exact scheme evaluates one normalising integral per distinct covariate value, and under uniform sampling there are as many of these as observations. At $n = 10000$ a MAP fit with the exact scheme takes between 44 s ($p = 50$) and 376 s ($p = 500$), against 0.3 to 1.9 s for NN and 1.8 to 4.1 s for WNN: the interpolated schemes are 130 to 200 times faster than the exact one for NN, and 24 to 91 times for WNN. The gap widens with the rank, since the exact scheme multiplies a matrix whose height grows with n by a vector whose length grows with p .

The saving is not only about the pre-computation. The matrix of basis values the optimiser works with has one block of quadrature nodes per distinct covariate value under `nothing`, but at most `nDiscret` blocks under NN and WNN, so every gradient evaluation is proportionally cheaper too. At $n = 10000$ and $p = 50$, the exact scheme spends 23 s in setup and 20 s in estimation, against 1.0 s and 0.8 s for WNN: the interpolated

schemes pay off by the same factor in both phases.

NN and WNN do not cost the same. WNN is 1.2 to 6 times slower than NN, the gap being widest at large n and small p . WNN combines every observation with the vertices of the grid cell that contains it, a step whose cost grows with n ; NN assigns each observation to a single node and aggregates the rest through multiplicities, so its cost barely moves between $n = 100$ and $n = 10000$. WNN remains the recommended default, since interpolating between nodes is what keeps the approximate likelihood smooth in the covariate. NN is the choice when a fast fit on a large sample matters more than that smoothness.

3.7.3 The estimation method

We now fix the scheme to WNN and compare the three estimation methods.

```
R> n_levels <- paste("n =", sort(unique(t_list$nobs)))
R>
R> df_ratio <- t_list %>%
+   filter(scheme == "WNN") %>%
+   mutate(ratio = estimation / total,
+          method = factor(method, levels = c("MAP", "Laplace", "MCMC")),
+          n_lab = factor(paste("n =", nobs), levels = n_levels))
R>
R> ggplot(df_ratio, aes(x = nobs, y = total, colour = method, fill = method)) +
+   stat_summary(fun = median, geom = "line", linewidth = 0.8) +
+   stat_summary(fun = median, geom = "point", size = 1.5) +
+   stat_summary(fun.min = function(z) quantile(z, 0.1),
+               fun.max = function(z) quantile(z, 0.9),
+               geom = "ribbon", alpha = 0.2, colour = NA) +
+   facet_wrap(~ paste("p =", p), nrow = 1) +
+   scale_x_log10() +
+   scale_y_log10() +
+   labs(x = "Training sample size", y = "Total fitting time [s]",
+        colour = "Estimation method", fill = "Estimation method") +
+   theme_bw() + theme(legend.position = "bottom")
```

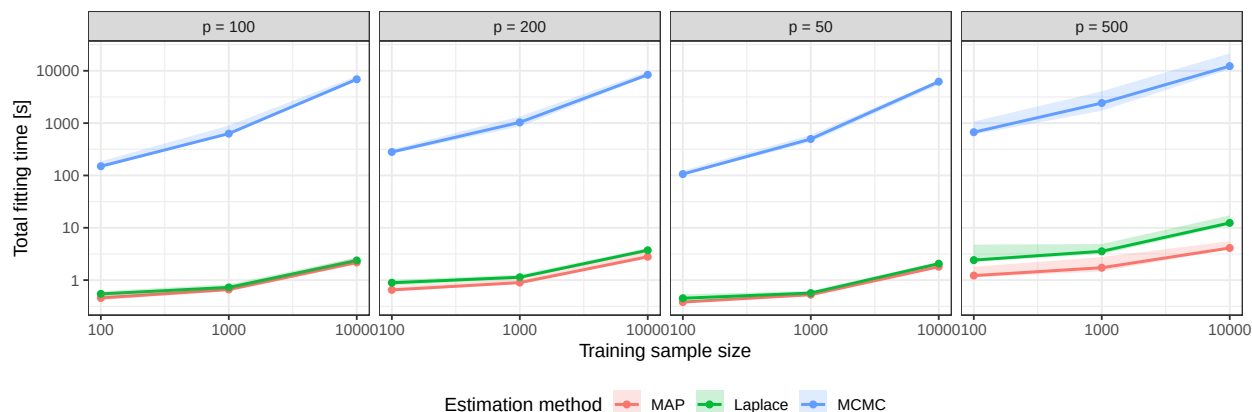


Figure 28: Total fitting time of MAP, Laplace and MCMC against the sample size, at four ranks, for the WNN scheme. Medians and 10th-90th percentiles over 10 replicates; logarithmic axes.

```
R> ggsave("./Figures/timingMethod.pdf", width = 10, height = 3.5)
```

The share of the total time spent in the estimation phase shows where the cost of each method lies.

```
R> n_levels <- paste("n =", sort(unique(t_list$nobs)))
```

```

R>
R> df_ratio <- t_list %>%
+   filter(scheme == "WNN") %>%
+   mutate(ratio = estimation / total,
+           method = factor(method, levels = c("MAP", "Laplace", "MCMC")),
+           n_lab = factor(paste("n =", nobs), levels = n_levels))
R>
R> ggplot(df_ratio, aes(x = p, y = ratio, colour = method, fill = method)) +
+   geom_hline(yintercept = 1, linetype = "dashed", colour = "grey40") +
+   stat_summary(fun = median, geom = "line", linewidth = 0.8) +
+   stat_summary(fun = median, geom = "point", size = 1.5) +
+   stat_summary(fun.min = function(z) quantile(z, 0.1),
+               fun.max = function(z) quantile(z, 0.9),
+               geom = "ribbon", alpha = 0.2, colour = NA) +
+   facet_wrap(~ n_lab, nrow = 1) +
+   scale_x_log10(breaks = sort(unique(t_list$p))) +
+   scale_y_log10() +
+   labs(x = "Rank p", y = "Estimation time / Total time",
+        colour = "Estimation method", fill = "Estimation method") +
+   theme_bw() + theme(legend.position = "bottom")

```

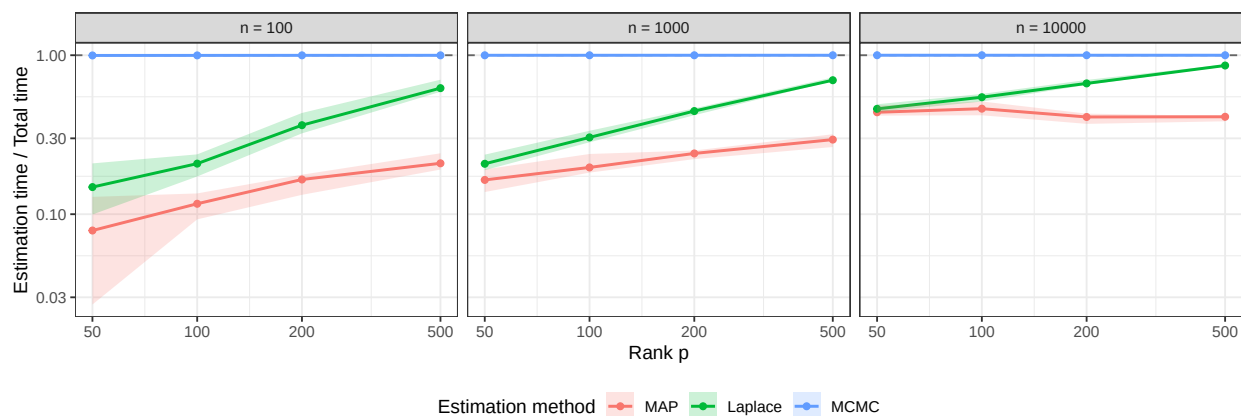


Figure 29: Share of the total fitting time spent in the estimation phase, against the rank, for the three methods. A value of one means that the pre-computation is negligible; one half, that it weighs as much as the estimation.

```

R> ggsave("./Figures/timingRatioMethod.pdf", width = 10, height = 3.5)

```

For MCMC the share is one throughout: the sampler dwarfs everything else. For MAP it lies between 0.08 and 0.44, that is, setup costs 1.3 to 11.7 times as much as the optimisation itself, most of all for small samples and small ranks. The pre-computation (and hence the choice of integral scheme) is therefore the dominant cost of a point estimate. Laplace sits in between, from 0.15 to 0.86: the Hessian it computes at the mode takes longer and longer as the rank grows.

```

R> ## estimation time of each method, relative to MAP
R> t_list %>%
+   filter(scheme == "WNN") %>%
+   group_by(method, nobs, p) %>%
+   summarise(estimation = median(estimation), .groups = "drop") %>%
+   pivot_wider(names_from = method, values_from = estimation) %>%
+   mutate(Laplace_vs_MAP = round(Laplace / MAP, 1),
+          MCMC_vs_MAP = round(MCMC / MAP, 0)) %>%

```

```

+   data.frame()
+   nobs    p Laplace    MAP      MCMC Laplace_vs_MAP MCMC_vs_MAP
1    100   50  0.070 0.030   105.810          2.3       3527
2    100  100  0.110 0.055   149.575          2.0       2720
3    100  200  0.310 0.110   280.275          2.8       2548
4    100  500  1.455 0.265   670.195          5.5       2529
5   1000   50  0.120 0.085   496.905          1.4       5846
6   1000  100  0.225 0.130   629.350          1.7       4841
7   1000  200  0.475 0.220  1033.405          2.2       4697
8   1000  500  2.445 0.520  2422.195          4.7       4658
9  10000   50  0.930 0.785  6197.960          1.2       7895
10 10000  100  1.280 1.010  6883.290          1.3       6815
11 10000  200  2.475 1.165  8400.315          2.1       7211
12 10000  500 10.745 1.645 12277.915          6.5       7464
R>
R> ## growth in the rank, as a slope on the log-log scale
R> t_list %>%
+   filter(scheme == "WNN") %>%
+   group_by(method, nobs) %>%
+   summarise(slope_p = round(coef(lm(log(estimation) ~ log(p)))[2], 2),
+             .groups = "drop") %>%
+   data.frame()
+   method  nobs slope_p
1 Laplace   100    1.45
2 Laplace  1000    1.33
3 Laplace 10000    1.07
4    MAP    100    1.01
5    MAP  1000    0.80
6    MAP 10000    0.32
7   MCMC    100    0.84
8   MCMC  1000    0.72
9   MCMC 10000    0.36

```

At $p = 500$, the estimation phase of Laplace costs 4.7 to 6.5 times that of MAP: the extra work is a single evaluation of the analytic Hessian at the mode, where a finite-difference Hessian would require of the order of $2p$ gradient evaluations. The estimation phase of MCMC costs 2500 to 7500 times that of MAP, that is from 11 minutes ($n = 100$) to 3.4 hours ($n = 10000$) for a single fit with the default four chains of 2000 iterations, against 1.5 to 11 seconds for Laplace.

Warmup accounts for half of the MCMC time in every configuration, exactly its share of the iterations, so adaptation is not the bottleneck: each iteration simply costs several gradient evaluations, and MCMC needs thousands of iterations where an optimiser needs tens.

3.7.4 In short

For a point estimate, the integral scheme matters far more than the rank: at $n = 10000$ the exact scheme is two orders of magnitude slower than the interpolated ones. Prefer WNN by default, and NN when speed on a large sample matters more than smoothness in the covariate.

For uncertainty quantification, Laplace provides a Gaussian approximation of the posterior for two to three times the total cost of a MAP fit, and should be the default. MCMC remains the reference, but at 500 to 3000 times the total cost of a MAP fit (can take hours rather than seconds at $n = 10000$) it is worth running when the posterior is expected to depart from Gaussianity, which the importance-sampling efficiency reported by `summary(fit, diagnostics = TRUE)` can be used to check.

4 A uni-dimensional index and discrete response application

The SLGP is not restricted to continuous responses. Because the normalising integral is computed by quadrature, aligning the quadrature nodes with a discrete support turns the model into an estimator of conditional probability mass functions. The `quakes` catalogue contains a natural discrete response: the event magnitude `mag`, which is reported to the nearest tenth and therefore takes values on the regular grid $\{4.0, 4.1, \dots, 6.4\}$.

This variable also illustrates a second point. The dataset only records events of magnitude at least 4.0: smaller events are not reliably detected by the seismic network, so they are absent from the data. The observed distribution of `mag` is thus the distribution of magnitude **conditional on exceeding the completeness threshold** of 4.0. We use this as an opportunity to demonstrate the SLGP trend: rather than simply restricting the response support to $[4.0, 6.5]$, we model the full range $[0, 6.5]$ and encode the detection mechanism as prior knowledge through a trend function that suppresses probability mass below the threshold.

This makes explicit which part of the fitted field is governed by data (above 4.0) and which is prior-driven extrapolation (below it).

```
R>
R> range_x <- c(165, 190)
R> mag_levels <- seq(4.0, 6.4, by = 0.1) # observed discrete support
R>
R>
R> scatter_mag <- ggplot(df, aes(x = long, y = mag)) +
+   geom_point(size = 1, alpha = 0.4, colour = "navy") +
+   coord_cartesian(xlim = range_x, ylim=c(0, 6.4)) +
+   labs(x = "Longitude [°E]", y = "Magnitude",
+        title = "Observed magnitudes") +
+   theme_bw()
R>
R> df$long_bin <- cut(df$long, breaks = seq(165, 190, by = 2.5))
R> hist_mag <- ggplot(df, aes(x = mag)) +
+   geom_histogram(aes(y = after_stat(density)),
+                  breaks = seq(0.0, 6.5, by = 0.1),
+                  fill = "darkgrey", colour = "grey50", linewidth = 0.2) +
+   facet_wrap(~ long_bin, nrow = 2) +
+   labs(x = "Magnitude", y = "Probability density",
+        title = "Magnitude distribution by longitude band") +
+   theme_bw()
R>
R> ggarrange(scatter_mag, hist_mag, ncol = 2, widths = c(0.35, 0.65))
```

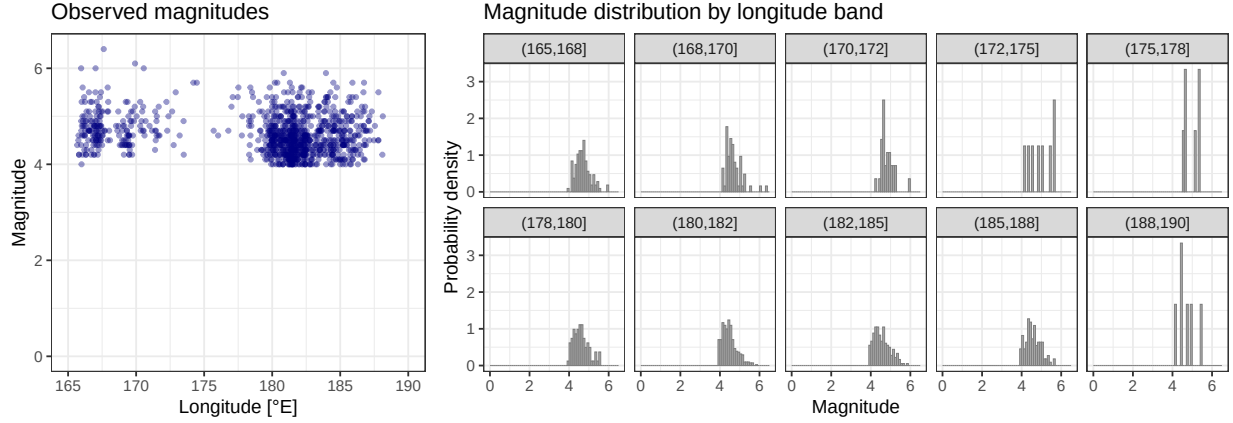


Figure 30: A visual representation of the event magnitudes depending on the longitude in the ‘quakes’ catalogue.

```
R> ggsave("./Figures/scatterDiscrete.pdf", width=10, height=3.5)
```

4.1 A trend to encode the completeness threshold

The trend enters the SLGP as an additive term on the latent Gaussian process, $Z_{x,t} = m(x,t) + \sigma \sum_j f_j(x,t) \varepsilon_j$, so it shifts the prior mean of the log-density: before any data are seen, the prior expects the density to be proportional to $e^{m(x,t)}$. To suppress magnitudes below the completeness threshold we use a trend that is zero above 4.0 and decreases linearly below it, so that e^m is negligible on $[0, 4.0)$ and unconstrained above:

$$m(\text{magnitude}) = \begin{cases} -10, & \text{magnitude} < 4, \\ 0, & \text{magnitude} \geq 4. \end{cases}$$

The trend is supplied to `slgp()` as a function of a data frame containing the response and covariate columns. One implementation detail matters: the trend is evaluated on the response in its original units here, so the breakpoint is the physical value 4.0.

```
R> range_mag_full <- c(0, 6.5) # FULL modelled range, including the unobserved part
R> K_full <- 66
R>
R> trend_fun <- function(df){return(ifelse(df$mag < 4, -10, 0))}
```

To see what the trend does, we compare draws from the SLGP prior with and without it. Without a trend the prior density draws place mass across the whole range, including the unobservable $[0, 4.0)$; with the completeness trend every prior draw is a priori near-zero below 4.0 and free above it (figure below).

```
R> common <- list(formula = mag ~ long, data = df, method = "Prior",
+               basisFunctionsUsed = "RFF", interpolateBasisFun = "WNN",
+               hyperparams = list(lengthscale = c(0.05, 0.15), sigma2 = 1),
+               sigmaEstimationMethod = "heuristic",
+               predictorsLower = range_x[1], predictorsUpper = range_x[2],
+               responseRange = range_mag_full,
+               nIntegral = K_full,
+               opts_BasisFun = list(nFreq = 250, MatParam = 5/2), seed = 1,
+               discrete = TRUE, opts = list(ndraws = 3))
R>
R> priorNoTrend <- do.call(slgp, common)
R> priorTrend <- do.call(slgp, c(common, list(trend = trend_fun)))
R>
```

```
R> plot(priorNoTrend, draw = c(1:3), panels = TRUE)
```

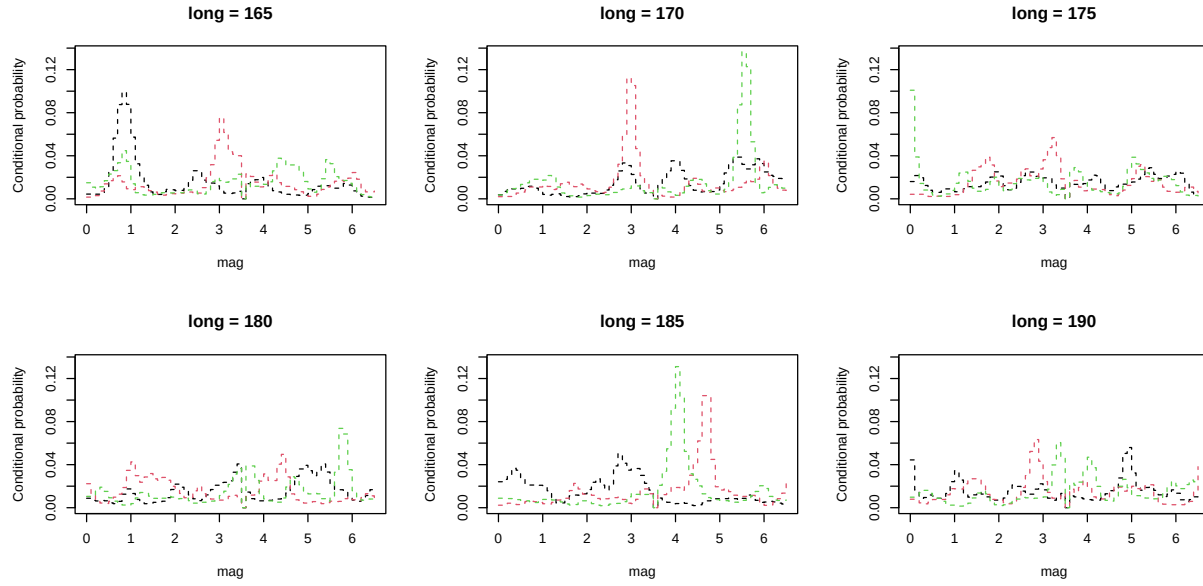


Figure 31: Three draws from the SLGP prior on the discrete magnitude support, without a trend (top) and with the completeness trend (bottom). The trend suppresses prior mass below the threshold of 4.0 (dashed line) before any data are seen.

```
R> plot(priorTrend, draw = c(1:3), panels = TRUE)
```

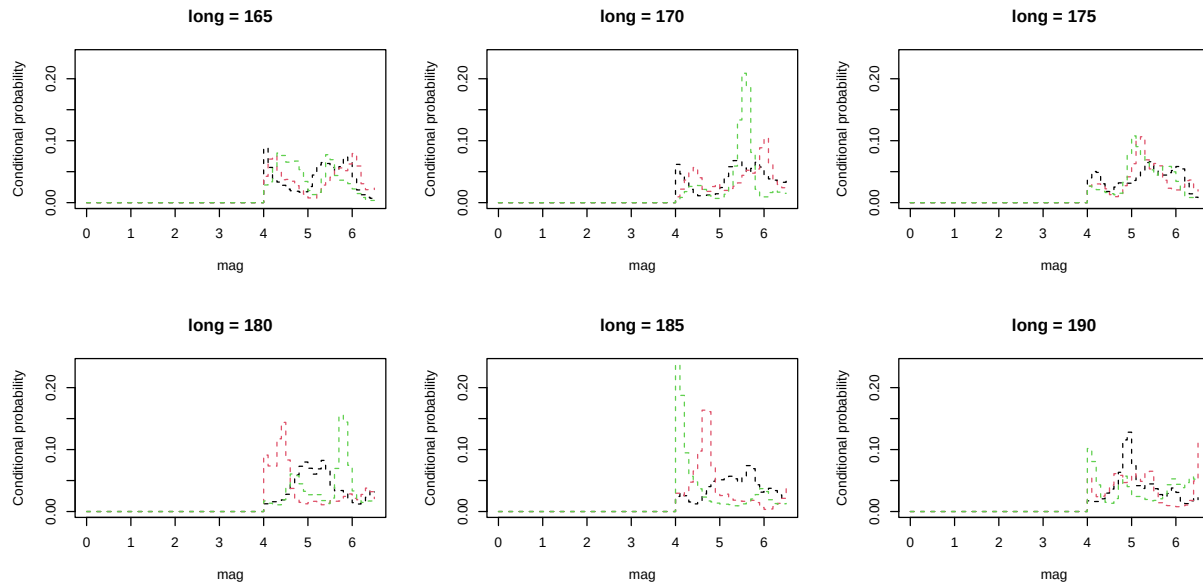


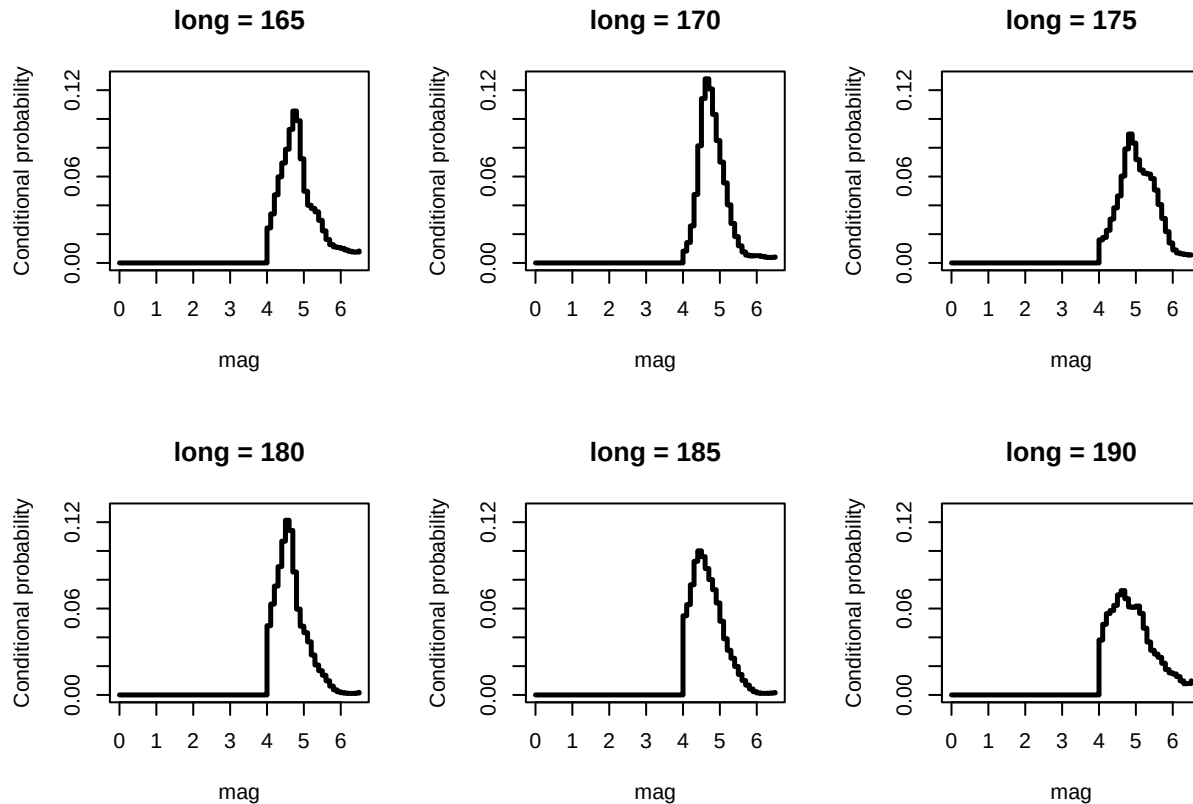
Figure 32: Three draws from the SLGP prior on the discrete magnitude support, without a trend (top) and with the completeness trend (bottom). The trend suppresses prior mass below the threshold of 4.0 (dashed line) before any data are seen.

4.2 Fitting and checking the model

We now fit the model by MAP on the full range with the completeness trend, and compare the estimated pmf to the empirical frequencies at several longitudes. The fitted mass is essentially zero below \$4.0 (as imposed by the trend).

```
R> modelMAGdisc <- update(priorTrend, newdata=df, method="MAP")
```

```
R> plot(modelMAGdisc, draw = "mean", panels = TRUE)
```



```
R> dev.copy(pdf, "./Figures/defaultMAPDiscrete.pdf", width = 10, height = 3.5)
```

```
pdf
```

```
3
```

```
R> dev.off()
```

```
pdf
```

```
2
```

```
R> selected_values <- c(167, 181, 185)
```

```
R> gap <- 1.0
```

```
R> df_filtered <- df %>%
```

```
+   mutate(interval=findInterval(long, c(0,
+                                     selected_values[1]-gap,
+                                     selected_values[1]+gap,
+                                     selected_values[2]-gap,
+                                     selected_values[2]+gap,
+                                     selected_values[3]-gap,
+                                     selected_values[3]+gap)))%>%
+   filter(interval %in% c(2, 4, 6))%>%
+   group_by(interval)%>%
```

```

+   mutate(category = paste0("long close to ", c("", selected_values[1],
+   "", selected_values[2],
+   "", selected_values[3])[interval],
+   "\nn=", n()))
R> names <- sort(unique(df_filtered$category))
R> dfGrid <- data.frame(expand.grid(selected_values,
+   seq(range_mag_full[1], range_mag_full[2],, K_full)))
R> colnames(dfGrid) <- c("long", "mag")
R> predMAP <- predict(modelMAGdisc, newdata = dfGrid, discrete=TRUE, nIntegral = K_full)
R> colnames(predMAP) <- c("long", "mag", "MAP estimator")
R> predMAP <- predMAP%>%
+   pivot_longer(-c("long", "mag"))
R> predMAP$category <- ifelse(predMAP$long==selected_values[1], names[1],
+   ifelse(predMAP$long==selected_values[2], names[2], names[3]))
R>
R> ggplot(mapping=aes(x = mag)) +
+   geom_histogram(df_filtered,
+   mapping=aes(y = after_stat(density)/10, group=category),
+   closed = "left", binwidth = 0.1, boundary = 0,
+   fill="darkgrey", col="grey50", lwd=0.2, alpha=0.7) +
+   geom_rug(data=df_filtered, sides = "b", color = "navy", alpha = 0.5)+
+   geom_step(data=predMAP, mapping=aes(y=value, group=name, col=name), lwd=1.1)+
+   facet_wrap(~ category, scales = "free_y", nrow=1) +
+   labs(x = "Magnitude (-)",
+   y = "Probability density",
+   title = "Binned 'mag' histograms by 'long' (width = 2) VS SLGP-MAP estimators at bins centers
+   theme_bw()+
+   theme(legend.position="bottom",
+   legend.direction = "horizontal",
+   legend.title = element_blank())+
+   coord_cartesian(xlim=range_mag_full,
+   ylim=c(0, 0.15))

```

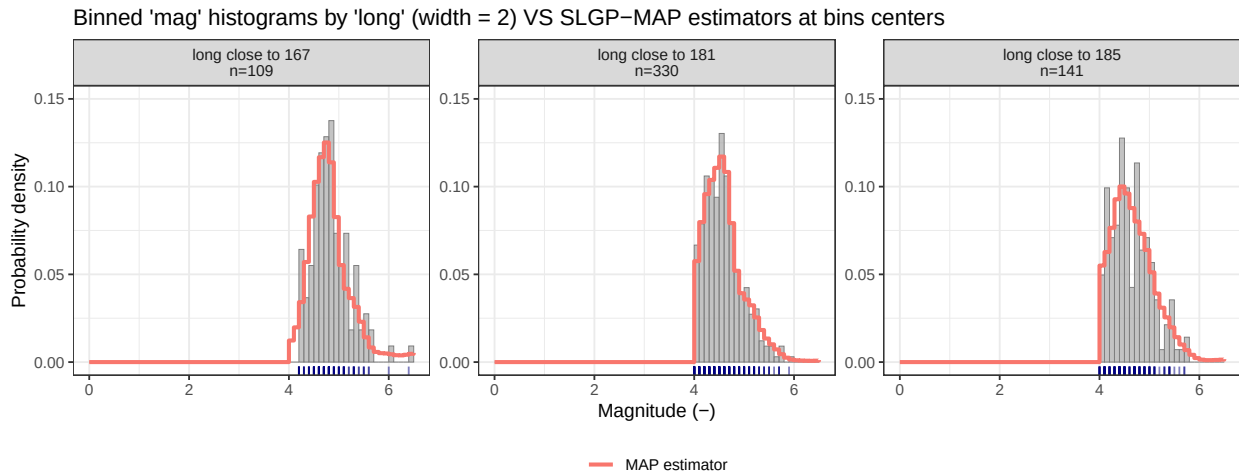


Figure 33: MAP estimate of the conditional probability mass function of magnitude at three longitudes, against the empirical frequencies of the events within 1 degree of each.

R>

```

R> ggsave(paste0("./Figures/histMAPdisc", ".pdf"), width=10, height=3.5)

Calibration can be assessed, and when we do, we consider only where the data are informative, i.e. on the
observed range [4.0, 6.5]. [explain what we do here, are we really comparing the right things? ]

R> obs_freq <- as.numeric(table(factor(round(df$mag, 1), levels = mag_levels))) / nrow(df)
R>
R> # average predicted pmf over the observed longitudes
R> gridPool <- expand.grid(mag = mag_levels, long = df$long)
R> poolPred <- predict(modelMAGdisc, newdata = gridPool, type = "density",
+                       nIntegral = K_full, discrete = TRUE)
R> poolPred$prob <- rowMeans(poolPred[, grep("^pdf_", colnames(poolPred)), drop = FALSE])
R> pred_freq <- tapply(poolPred$prob, poolPred$mag, mean)
R> calib <- data.frame(mag = mag_levels, observed = obs_freq,
+                     predicted = as.numeric(pred_freq[as.character(mag_levels)]))
R>
R> ggplot(calib, aes(x = observed, y = predicted)) +
+   geom_abline(slope = 1, intercept = 0, linetype = 2, colour = "navy") +
+   geom_point(colour = "grey", size = 1.5) +
+   coord_equal() +
+   labs(x = "Observed frequency", y = "SLGP-predicted frequency",
+        title = "Discrete calibration on the observed range") +
+   theme_bw()

```

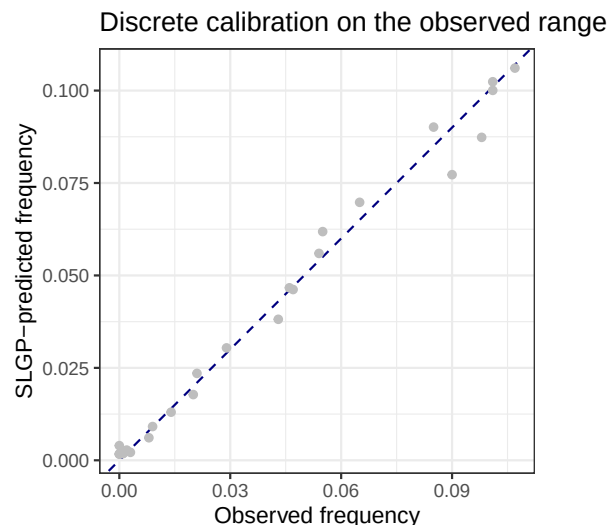


Figure 34: Observed versus SLGP-predicted marginal frequencies of each magnitude level on the observed range, pooled over the dataset Points near the diagonal indicate a well-calibrated discrete fit.

```

R> ggsave("./Figures/discreteCalib.pdf", width = 5, height = 3.6)

```

5 A bi-dimensional index and continuous response application

Everything so far has indexed the depth distribution by longitude alone. That one-dimensional setting is convenient for slice-based figures, but it is also within reach of many conditional density estimators. The distinctive strength of the SLGP is that nothing in its construction is tied to a scalar index: the model estimates a field of densities over an index domain of any dimension, from scattered and replicate-free data, at no change to the interface.

We now exploit the full epicentre location (`long`, `lat`) as a two-dimensional index and estimate the conditional distribution of `depth` over the region. This is a regime where per-cell histograms become untenable, since binning a two-dimensional index with 1000 scattered events leaves most cells empty, yet the SLGP applies unchanged.

To obtain an honest assessment of the fit rather than a purely visual one, we adopt a train/test split: the model is trained on a random 75% of the events and validated on the remaining 25%. The figure below shows the two subsets over the region.

```
R> range_response <- c(40, 680) # Can use range(df$depth), or user defined range as we do here
R> range_xlong <- c(165, 190) # Can use range(df$long), or user defined range as we do here
R> range_xlat <- c(-40, -10) # Can use range(df$lat), or user defined range as we do here
R>
R> set.seed(1)
R> id_train <- sample(seq_len(nrow(df)), size = floor(0.75 * nrow(df)), replace = FALSE)
R>
R> df_train <- df[id_train, ]
R> df_test <- df[-id_train, ]
R>
R>
R> map_plot <- ggplot(mapping = aes(x = long, y = lat, fill = depth)) +
+   geom_point(data = df_train, mapping = aes(col = "Train", pch = "Train"),
+             size = 1.5, alpha = 0.8) +
+   geom_point(data = df_test, mapping = aes(col = "Test", pch = "Test"),
+             size = 2, alpha = 0.8) +
+   scale_fill_viridis_c(name = "Hypocentre\ndepth [km]",
+                       limits = range_response, option = "viridis") +
+   scale_shape_manual(values = c("Train" = 21, "Test" = 22)) +
+   scale_color_manual(values = c("Train" = "grey", "Test" = "red")) +
+   coord_cartesian(xlim = range_xlong, ylim = range_xlat) +
+   labs(x = "Longitude [°E]", y = "Latitude [°]",
+        title = "Observed hypocentre depths",
+        col = "Data split", shape = "Data split") +
+   theme_bw()
R> map_plot
```

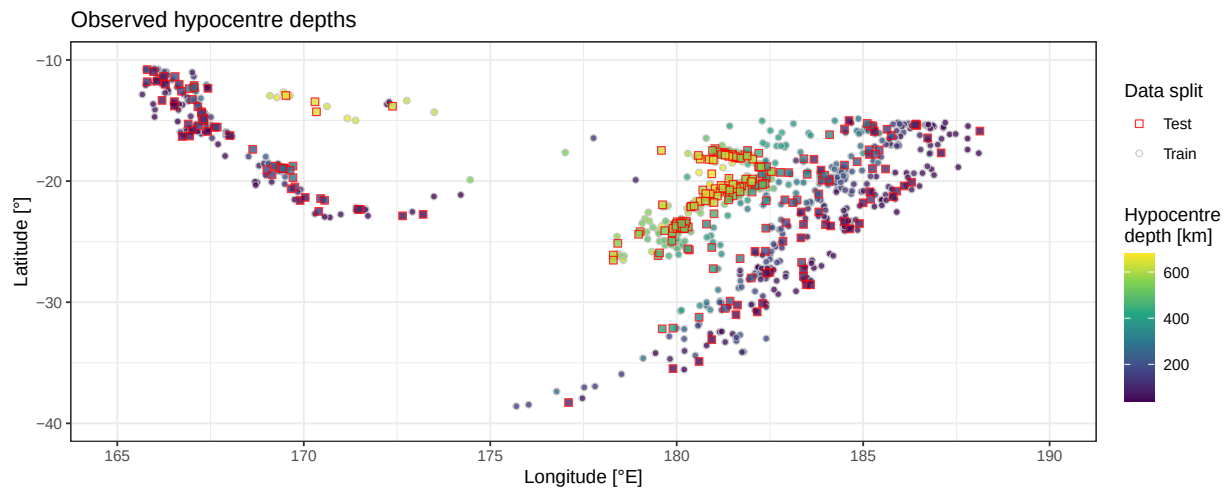


Figure 35: Observed hypocentre depths over the Fiji-Tonga region, split into a 75% training set (grey) and a 25% held-out test set (red). Colour encodes depth.

Fitting the two-dimensional model requires only that the index-related arguments grow accordingly: the formula becomes `depth ~ long + lat`, `predictorsLower` and `predictorsUpper` are length two, and the length-scale vector has one entry per index dimension plus one for the response. The estimation scheme, basis and quadrature are otherwise identical to the one-dimensional case.

```
R> modelMAP2D <- slgp(depth ~ long + lat,
+                     data = df_train,
+                     method = "MAP",
+                     basisFunctionsUsed = "RFF",
+                     interpolateBasisFun = "WNN",
+                     hyperparams = list(lengthscale = rep(0.15, 3), sigma2 = 1),
+                     sigmaEstimationMethod = "heuristic",
+                     predictorsLower = c(range_xlong[1], range_xlat[1]),
+                     predictorsUpper = c(range_xlong[2], range_xlat[2]),
+                     responseRange = range_response,
+                     opts_BasisFun = list(nFreq = 250, MatParam = 5/2),
+                     seed = 1)
```

```
R> summary(modelMAP2D)
Summary of a Spatial Logistic Gaussian Process (SLGP) model
  Formula      : depth ~ long + lat
  Response     : depth in [40, 680]
  Covariate(s) : long, lat
  Estimation   : MAP
  Basis        : RFF (rank p = 500)
  Length-scales : 0.15, 0.15, 0.15
  Variance sigma2: 0.6967
  Data         : 750 obs, 748 distinct covariate value(s)
```

```
Estimation diagnostics
  Fitting time   : 22.1s (setup 15.5s, estimation 6.63s)
  Coefficient draws : 1
  Log-posterior   : 634.894 (at mode for MAP/Laplace)
  Optimiser      : converged
```

5.1 Out-of-sample calibration

Before inspecting the fitted field, we check that it is *calibrated* on data it has never seen. Because the SLGP returns a full predictive distribution at every location, we can apply the probability integral transform (PIT): for each held-out event we evaluate the predictive CDF at the observed depth. If the model is well calibrated, these PIT values should be approximately uniform on $[0, 1]$.

We assess this both visually (with a PIT histogram and a uniform quantile-quantile plot) and with two complementary goodness-of-fit tests against the uniform. The Kolmogorov-Smirnov statistic is the largest vertical gap between the empirical and uniform CDFs, and so reacts to a localised departure, while the Cramér-von Mises statistic integrates the squared gap over $[0, 1]$, and so reacts to a diffuse, systematic one. Passing both is therefore a stronger check than passing either.

```
R> pit_pred <- predict(modelMAP2D, type = "cdf",
+                      newdata = df_test[, c("long", "lat", "depth")])
R> u <- pit_pred$cdf_1
R>
R> ks.test(u, "punif", 0, 1)
```

Asymptotic one-sample Kolmogorov-Smirnov test

```
data: u
```

```

D = 0.062536, p-value = 0.2822
alternative hypothesis: two-sided
R> library(goftest)
R> cvm.test(u, null = "punif")

Cramer-von Mises test of goodness-of-fit
Null hypothesis: uniform distribution
Parameters assumed to be fixed

data: u
omega2 = 0.34671, p-value = 0.1003
R>
R> pit_hist <- ggplot(pit_pred, aes(x = cdf_1)) +
+   geom_histogram(breaks = seq(0, 1, by = 0.1),
+   fill = "darkgrey", colour = "grey40", linewidth = 0.3) +
+   geom_hline(yintercept = nrow(df_test) / 10,
+   linetype = 2, linewidth = 0.8, colour = "navy") +
+   scale_x_continuous(breaks = seq(0, 1, by = 0.2), limits = c(0, 1)) +
+   labs(x = "PIT", y = "Count", title = "PIT histogram") +
+   theme_bw()
R>
R> n_pit <- length(u)
R> qq_df <- data.frame(
+   theoretical = (seq_len(n_pit) - 0.5) / n_pit,   # uniform plotting positions
+   empirical   = sort(u)
+ )
R> pit_qq <- ggplot(qq_df, aes(x = theoretical, y = empirical)) +
+   geom_abline(slope = 1, intercept = 0, linetype = 2, colour = "navy") +
+   geom_point(size = 1, alpha = 0.6, colour = "grey20") +
+   coord_equal(xlim = c(0, 1), ylim = c(0, 1)) +
+   labs(x = "Theoretical uniform quantile", y = "Empirical PIT quantile",
+   title = "Uniform Q-Q plot") +
+   theme_bw()
R>
R> ggarrange(pit_hist, pit_qq, ncol = 2, widths = c(0.5, 0.5))

```

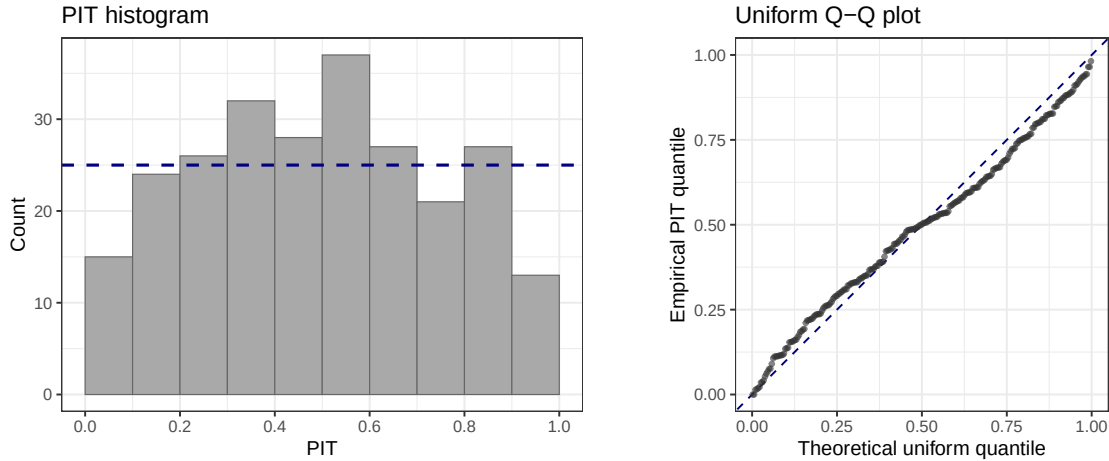


Figure 36: Out-of-sample calibration of the two-dimensional SLGP. Left: PIT histogram of the held-out depths (dashed line: expected count under uniformity). Right: uniform Q-Q plot of the PIT values (points on the diagonal indicate calibration).

```
R> ggsave("./Figures/pit2D.pdf", width = 9, height = 3.0)
```

Both tests fail to reject uniformity at the 5% and 10% level (Kolmogorov-Smirnov: $D = 0.062413$, $p = 0.2844$, and Cramér-von Mises: $\omega^2 = 0.33872$, $p = 0.1055$), though the Cramér-von Mises p-value is borderline, indicating a mild, diffuse rather than localised departure from perfect calibration.

The PIT histogram is reasonably close to flat and the points of the Q-Q plot lie near the diagonal, confirming that the fitted field is well calibrated on held-out data: its predictive distributions are neither systematically too narrow nor too wide. This is a stronger statement than visual agreement with binned histograms, since it is computed entirely out of sample and summarises calibration across the whole region.

Now that we validated quantitatively, we can also check visual agreement with histograms.

5.2 Reading the fitted field at selected locations

Since the SLGP predicts an entire distribution of `depth` at each `(long, lat)`, visualisation is less direct than in the one-dimensional case. We first inspect the fitted conditional densities at a handful of arbitrary bins, then turn to whole-region summary maps.

For the empirical reference we define a small set of spatial rectangles, retaining their train/test counts, and compare the SLGP density at each rectangle centre with the pooled histogram of datapoints falling inside it (Figure~??).

```
R> long_min <- c(166, 180, 178, 182, 180, 178)
R> long_max <- c(170, 184, 182, 186, 184, 182)
R> lat_min <- c(-15, -20, -25, -25, -30, -35)
R> lat_max <- c(-10, -15, -20, -20, -25, -30)
R>
R> bins <- data.frame(bin_id = seq_along(long_min),
+                     long_min, long_max, lat_min, lat_max) %>%
+   mutate(bin_name = paste0("Bin (", letters[bin_id], ")"))
R>
R> df_train$set <- "Train"; df_test$set <- "Test"
R> df_split <- bind_rows(df_train, df_test) %>%
+   mutate(set = factor(set, levels = c("Train", "Test")))
R>
R> df_selected <- do.call(rbind, lapply(seq_len(nrow(bins)), function(i) {
```

```

+   tmp <- df_split %>%
+     filter(long > bins$long_min[i], long <= bins$long_max[i],
+            lat > bins$lat_min[i], lat <= bins$lat_max[i])
+   tmp$bin_id <- bins$bin_id[i]; tmp$bin_name <- bins$bin_name[i]
+   tmp
+ }))
R>
R> bin_counts <- df_selected %>%
+   count(bin_id, bin_name, set) %>%
+   tidyr::pivot_wider(names_from = set, values_from = n, values_fill = 0)
R>
R> bins <- bins %>%
+   left_join(bin_counts, by = c("bin_id", "bin_name")) %>%
+   mutate(facet_label = paste0(bin_name, "\n",
+                               "long in (", long_min, ", ", long_max, "]\n",
+                               "lat in (", lat_min, ", ", lat_max, "]\n",
+                               "nTrain = ", Train, ", nTest = ", Test))
R>
R> df_selected <- df_selected %>%
+   left_join(bins %>% select(bin_id, facet_label), by = "bin_id") %>%
+   mutate(facet_label = factor(facet_label, levels = bins$facet_label))
R>
R> hist_plot <- ggplot(df_selected, aes(x = depth, fill = set, colour = set)) +
+   geom_histogram(aes(y = after_stat(density)),
+                 breaks = seq(range_response[1], range_response[2], by = 20),
+                 position = "identity", alpha = 0.35, linewidth = 0.3) +
+   facet_wrap(~ facet_label, nrow = 2, scales = "free_y") +
+   scale_fill_manual(values = c("Train" = "grey50", "Test" = "navy")) +
+   scale_colour_manual(values = c("Train" = "grey30", "Test" = "navy")) +
+   labs(x = "Hypocentre depth [km]", y = "Probability density",
+        fill = NULL, colour = NULL) +
+   coord_cartesian(xlim = range_response) +
+   theme_bw() + theme(legend.position = "bottom")
R>
R> map_plot2 <- map_plot +
+   geom_rect(data = bins,
+            aes(xmin = long_min, xmax = long_max,
+               ymin = lat_min, ymax = lat_max),
+            inherit.aes = FALSE, fill = NA, colour = "black", linewidth = 0.8) +
+   geom_label(data = bins,
+            aes(x = (long_min + long_max) / 2, y = (lat_min + lat_max) / 2,
+               label = bin_name),
+            inherit.aes = FALSE, fontface = "bold", size = 3,
+            fill = "white", alpha = 0.8)
R>
R> ggarrange(map_plot2, hist_plot, ncol = 2, nrow = 1, widths = c(0.45, 0.55))

```

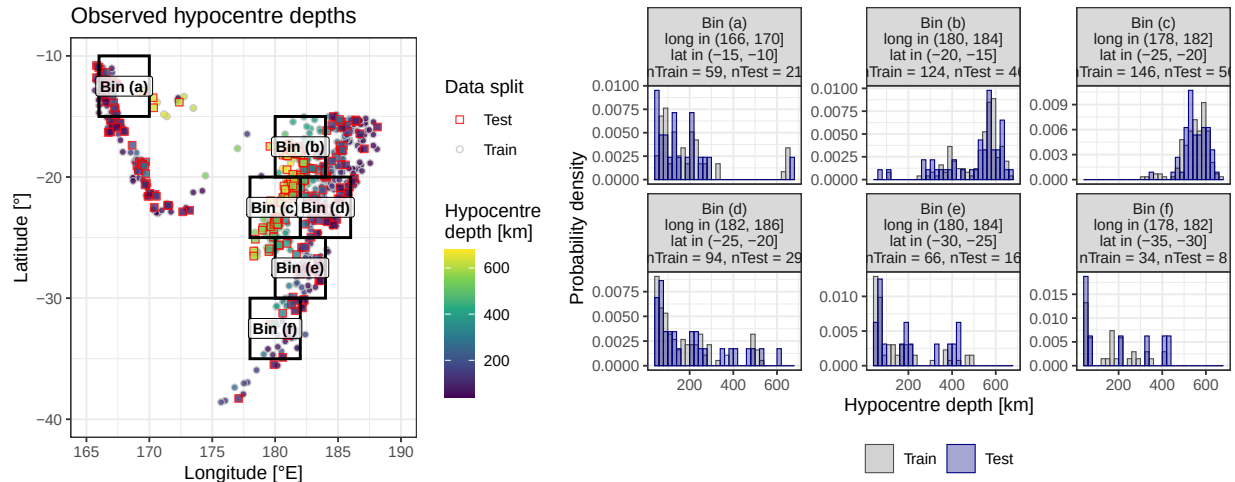



Figure 37: Selected spatial rectangles over the region (left) and, for each, the pooled empirical depth distribution split by train/test (right). Rectangles are labelled (a)-(f).

```
R> ggsave("./Figures/scatter2D.pdf", width = 10, height = 4, scale=1.1)

R> bin_centers <- bins %>%
+   mutate(long = (long_min + long_max) / 2, lat = (lat_min + lat_max) / 2)
R>
R> depth_grid <- seq(range_response[1], range_response[2], length.out = 101)
R> dfGrid <- merge(bin_centers %>% select(bin_id, bin_name, facet_label, long, lat),
+   data.frame(depth = depth_grid))
R>
R> predMAP <- predict(modelMAP2D, newdata = dfGrid %>% select(long, lat, depth))
R> predMAP <- cbind(dfGrid, predMAP %>% select(starts_with("pdf_")))
R>
R>
R> hist_plotMAP <- ggplot(mapping = aes(x = depth)) +
+   geom_histogram(data = df_selected, aes(y = after_stat(density),
+   fill = set, colour = set),
+   breaks = seq(range_response[1], range_response[2], by = 20),
+   position = "identity", alpha = 0.35, linewidth = 0.3) +
+   geom_rug(data = df_selected, sides = "b", colour = "navy", alpha = 0.5) +
+   geom_line(data = predMAP,
+   aes(y = pdf_1, group = "MAP estimator", col = "MAP estimator"),
+   linewidth = 1.1) +
+   scale_fill_manual(values = c("Train" = "grey50", "Test" = "navy")) +
+   scale_colour_manual(values = c("Train" = "grey30", "Test" = "navy",
+   "MAP estimator" = "tomato"))+
+   guides(colour = guide_legend(
+   override.aes = list(fill = c("grey50", "navy", NA),
+   colour = c("grey30", "navy", "tomato"),
+   linetype = c(0, 0, 1))),
+   fill = "none")+
+   facet_wrap(~ facet_label, scales = "free_y", nrow = 2) +
+   labs(x = "Hypocentre depth [km]", y = "Probability density",
+   title = "Empirical depth distributions vs SLGP estimates at bin centres") +
+   coord_cartesian(xlim = range_response) +
+   theme_bw() +
```

```

+   theme(legend.position = "bottom", legend.direction = "horizontal",
+         legend.title = element_blank())
R> ggsave(hist_plotMAP, file="./Figures/scatter2DMAPa.pdf", width = 10, height = 4)
R>
R> ggarrange(map_plot2, hist_plotMAP, ncol = 2, nrow = 1, widths = c(0.45, 0.55))

```

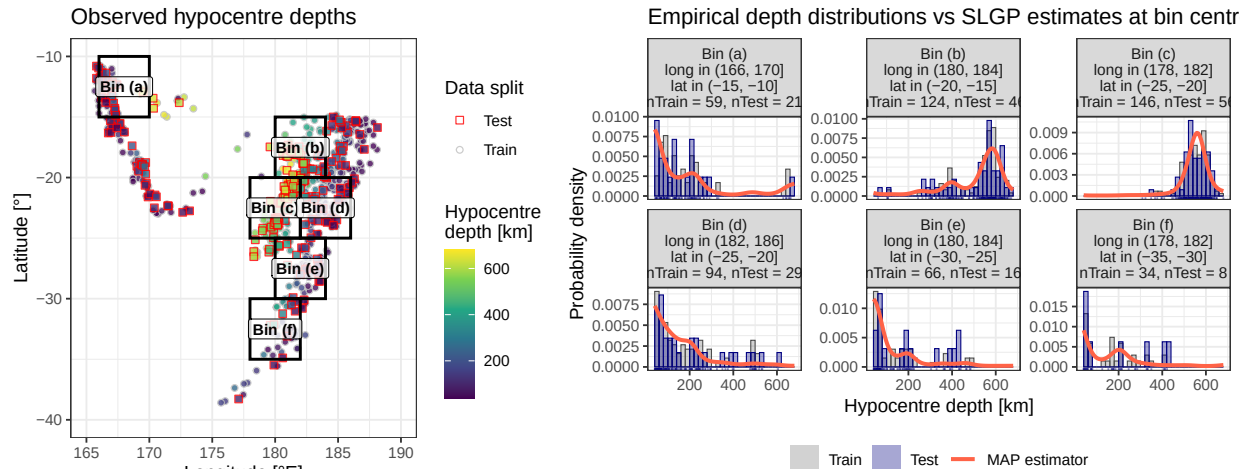


Figure 38: Empirical depth distributions (grey) versus the SLGP conditional density evaluated at the centre of each rectangle (red).

```

R> ggsave("./Figures/scatter2DMAPc.pdf", width = 10, height = 4, scale=1.1)
R>

```

The agreement is good in the well-sampled rectangles (a)-(c) and looser in (d)-(f). Part of this discrepancy is an artefact of the comparison rather than a failure of the model: the histogram pools every observation inside a fairly large and arbitrary rectangle, whereas the SLGP curve is evaluated at the single point at its centre. If the conditional distribution varies appreciably across the rectangle, the two objects need not coincide even for a perfectly specified model.

We could test this by shrinking each rectangle toward its centre and recomputing the histogram from the reduced region, giving a more local empirical reference, but for several bins this would leave too few observations. Instead we exploit the fact that the SLGP is defined everywhere: we evaluate the fitted density not only at each rectangle centre but also at its corners and edge midpoints (Figure~??). The spread of these curves measures how much the fitted conditional distribution varies within the rectangle.

```

R> eval_locations <- bind_rows(
+   bins %>% transmute(bin_id, facet_label, location = "Center",
+                     long = (long_min + long_max)/2, lat = (lat_min + lat_max)/2),
+   bins %>% transmute(bin_id, facet_label, location = "SW", long = long_min, lat = lat_min),
+   bins %>% transmute(bin_id, facet_label, location = "SE", long = long_max, lat = lat_min),
+   bins %>% transmute(bin_id, facet_label, location = "NW", long = long_min, lat = lat_max),
+   bins %>% transmute(bin_id, facet_label, location = "NE", long = long_max, lat = lat_max),
+   bins %>% transmute(bin_id, facet_label, location = "N",
+                     long = (long_min + long_max)/2, lat = lat_max),
+   bins %>% transmute(bin_id, facet_label, location = "S",
+                     long = (long_min + long_max)/2, lat = lat_min),
+   bins %>% transmute(bin_id, facet_label, location = "E",
+                     long = long_max, lat = (lat_min + lat_max)/2),
+   bins %>% transmute(bin_id, facet_label, location = "W",
+                     long = long_min, lat = (lat_min + lat_max)/2))

```

```

R>
R> dfGrid_locations <- merge(eval_locations, data.frame(depth = depth_grid))
R> pred_locations <- predict(modelMAP2D, type = "density",
+                           newdata = dfGrid_locations %>% select(long, lat, depth))
R> pred_locations <- cbind(dfGrid_locations,
+                          pred_locations %>% select(starts_with("pdf_"))) %>%
+   rename(value = pdf_1) %>%
+   mutate(location_type = case_when(
+     location == "Center" ~ "Center",
+     location %in% c("N", "S", "E", "W") ~ "Edge midpoint",
+     TRUE ~ "Corner"))
R>
R> ggplot(mapping = aes(x = depth)) +
+   geom_histogram(data = df_selected, aes(y = after_stat(density)),
+                 breaks = seq(40, 680, by = 20),
+                 fill = "darkgrey", colour = "grey50",
+                 linewidth = 0.2, alpha = 0.7) +
+   geom_rug(data = df_selected, sides = "b", colour = "navy", alpha = 0.4) +
+   geom_line(data = pred_locations %>% filter(location != "Center"),
+            aes(y = value, group = interaction(location, facet_label),
+              linetype = location_type, col = "MAP estimator"),
+            linewidth = 0.95, alpha = 0.75) +
+   geom_line(data = pred_locations %>% filter(location == "Center"),
+            aes(y = value, group = "Center", linetype = "Center",
+              col = "MAP estimator"), linewidth = 1.2) +
+   facet_wrap(~ facet_label, nrow = 2, scales = "free_y") +
+   scale_linetype_manual(values = c("Center" = 1, "Edge midpoint" = 2, "Corner" = 3)) +
+   labs(x = "Hypocentre depth [km]", y = "Probability density",
+        linetype = "SLGP evaluation location") +
+   coord_cartesian(xlim = range_response) +
+   theme_bw() +
+   theme(legend.position = "bottom", legend.direction = "horizontal",
+        legend.title = element_blank())

```

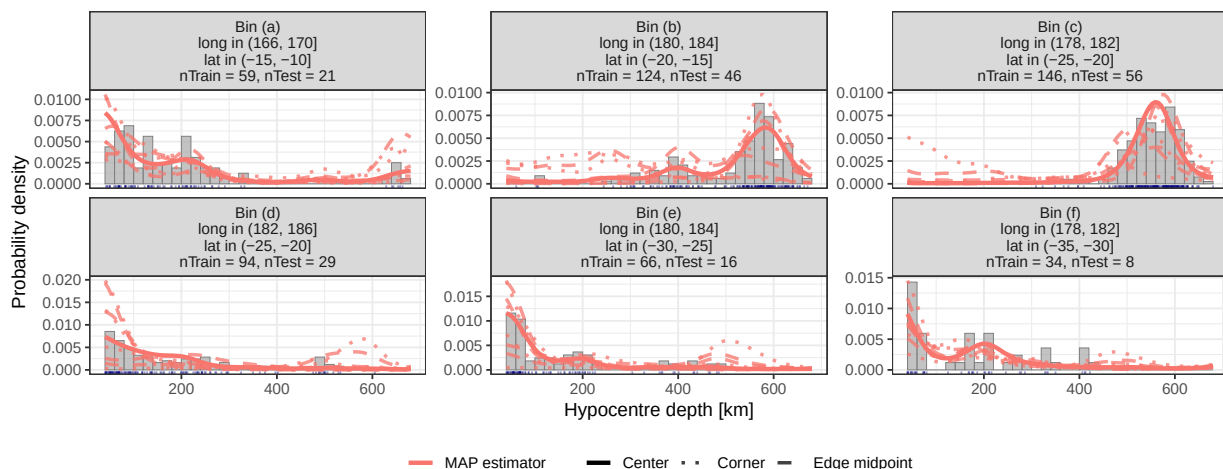


Figure 39: SLGP conditional depth densities evaluated across each rectangle (centre, edge midpoints, corners), against the pooled empirical histogram. A wide spread of curves indicates within-rectangle spatial heterogeneity of the conditional distribution.

```
R> ggsave(paste0("./Figures/histMAPdiscB", ".pdf"), width=10, height=3.5)
```

The curves confirm the interpretation: within a single rectangle the SLGP density can shift substantially between centre, edges and corners, precisely in the bins where the pooled histogram disagreed with the centre estimate. The earlier discrepancies are therefore consistent with genuine within-rectangle spatial heterogeneity captured by the model, rather than a systematic lack of fit and they illustrate why aggregating a spatially varying field into arbitrary bins, a practice the SLGP is designed to avoid, can be misleading.

5.3 Whole-region summary maps

Evaluating a scalar functional of the fitted distribution on a dense grid turns the density field into an interpretable map. Because the SLGP provides the entire conditional distribution at each location, any such functional (moments, quantiles, or exceedance probabilities) is available from the same fitted object. Figure~?? maps the conditional mean and standard deviation of `depth`.

```
R> grid2D <- expand.grid(long = seq(range_xlong[1], range_xlong[2], length.out = 50),
+                         lat  = seq(range_xlat[1], range_xlat[2], length.out = 50))
R>
R> pred_mean <- predict(modelMAP2D, type = "moments", newdata = grid2D,
+                       power = c(1), centered = FALSE)
R> pred_sd   <- predict(modelMAP2D, type = "moments", newdata = grid2D,
+                       power = c(2), centered = TRUE)
R>
R> pan_mean <- ggplot() +
+   geom_raster(data = pred_mean, aes(x = long, y = lat, fill = mSLGP_1),
+             interpolate=TRUE) +
+   geom_point(data = df_train, aes(x = long, y = lat, fill = depth),
+             inherit.aes = FALSE, shape = 21, size = 1.4,
+             colour = "grey20", alpha = 0.65, stroke = 0.2) +
+   scale_fill_viridis_c(name = "Predicted mean\ndepth [km]",
+                       limits = range_response, option = "viridis") +
+   coord_cartesian(xlim = range_xlong, ylim = range_xlat) +
+   labs(x = "Longitude [°E]", y = "Latitude [°]", title = "Conditional mean") +
+   theme_bw()
R>
R> pan_sd <- ggplot() +
+   geom_raster(data = pred_sd, aes(x = long, y = lat, fill = sqrt(mSLGP_1)),
+             interpolate=TRUE) +
+   geom_point(data = df_train, aes(x = long, y = lat),
+             inherit.aes = FALSE, shape = 21, size = 1.4,
+             colour = "grey20", fill = "white", alpha = 0.65, stroke = 0.2) +
+   scale_fill_viridis_c(name = "Standard dev.\ndepth [km]",
+                       limits = c(0, range_response[2] / 2), option = "viridis") +
+   coord_cartesian(xlim = range_xlong, ylim = range_xlat) +
+   labs(x = "Longitude [°E]", y = "Latitude [°]",
+        title = "Conditional standard deviation") +
+   theme_bw()
R>
R> ggarrange(pan_mean, pan_sd, ncol = 2, nrow = 1, widths = c(0.5, 0.5))
```

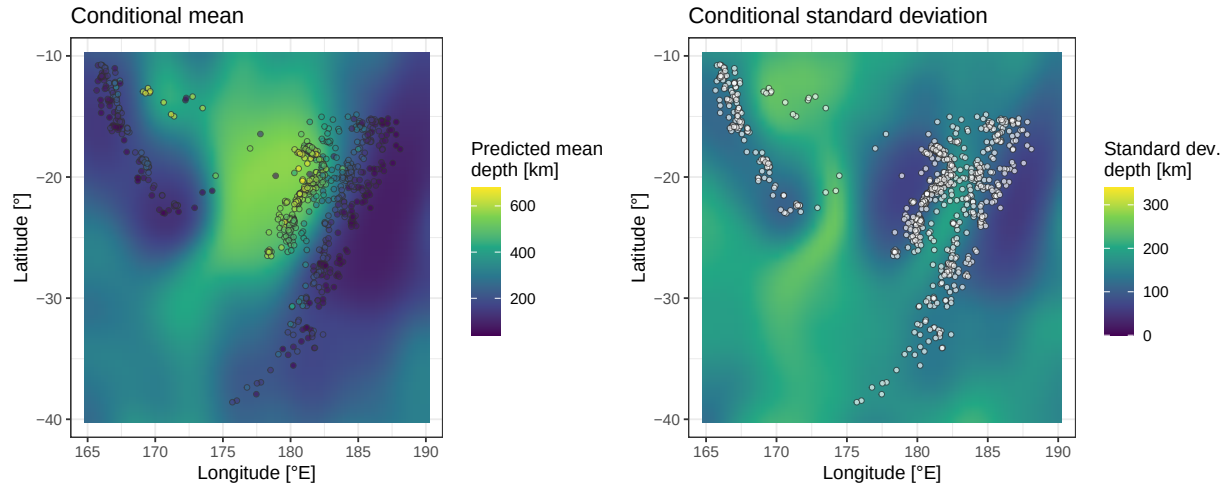


Figure 40: Conditional mean (left) and standard deviation (right) of hypocentre depth over the region, from the two-dimensional SLGP. Points are the observed epicentres; the mean map is coloured by observed depth on the same scale.

```
R> ggsave("./Figures/pred2D.pdf", width = 10, height = 4)
```

The mean map traces the deepening of seismicity from the shallow trench in the east towards the deep intraslab events in the west, while the standard-deviation map is largest in the transition zone, where shallow and deep events coexist and the conditional distribution is most dispersed.

A complementary and informative view retains the full distributional information through spatially varying exceedance probabilities.

Since the SLGP estimates the entire conditional distribution at each location, the probability $P(\text{depth} > u \mid \text{long}, \text{lat})$ of an event deeper than a threshold u is read directly from the predictive CDF, for any threshold. Figure-?? maps this probability at $u = 300$ -km, isolating the deep Wadati-Benioff seismicity.

```
R> threshold <- 300
R> grid2D_cdf <- grid2D
R> grid2D_cdf$depth <- threshold
R> pred_cdf <- predict(modelMAP2D, type = "cdf", newdata = grid2D_cdf)
R> grid2D_cdf$exceed <- 1 - pred_cdf$cdf_1
R>
R> df_train2 <- df_train %>%
+   mutate(depth_class = ifelse(depth > threshold,
+                               "Observed depth > 300 km",
+                               "Observed depth <= 300 km"))
R>
R> ggplot() +
+   geom_raster(data = grid2D_cdf,
+               aes(x = long, y = lat, fill = exceed),
+               interpolate = TRUE) +
+   geom_point(data = df_train2,
+              aes(x = long, y = lat, shape = depth_class, colour = depth_class),
+              inherit.aes = FALSE,
+              size = 1.7,
+              stroke = 0.6) +
+   scale_shape_manual(name = "Observed events",
+                      values = c("Observed depth > 300 km" = 0,
+                                "Observed depth <= 300 km" = 1)) +
```

```

+   scale_colour_manual(name = "Observed events",
+                       values = c("Observed depth > 300 km" = "grey40",
+                                   "Observed depth <= 300 km" = "grey80")) +
+   scale_fill_viridis_c(name = paste0("P(depth > ", threshold, " km)"),
+                       limits = c(0, 1),
+                       option = "plasma") +
+   coord_cartesian(xlim = range_xlong,
+                   ylim = range_xlat) +
+   labs(x = "Longitude [°E]",
+        y = "Latitude [°]",
+        title = paste0("Probability of an event deeper than ", threshold, " km")) +
+   theme_bw() +
+   theme(legend.position = "right",
+         legend.key = element_rect(fill = "white", colour = NA))

```

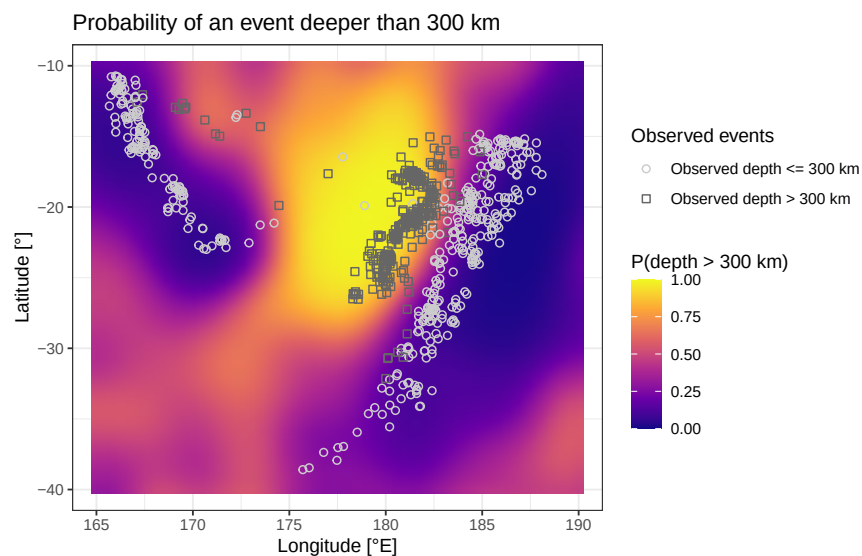


Figure 41: Posterior probability of a deep event, $P(\text{depth} > 300 \text{ km} \mid \text{long}, \text{lat})$, from the two-dimensional SLGP. Observed epicentres are overlaid. The map isolates the deep intraslab seismicity of the subducting plate.

```

R>
R> ggsave("./Figures/exceed2D.pdf", width = 6, height = 4)

```

This single map summarises a genuinely spatial, distribution-valued object: at every location it reports not a point prediction but a probability derived from the full conditional law, and it does so continuously across the region, including where events are sparse. We stress the reading is most reliable where the epicentres are dense: in the poorly sampled corners of the domain the field is an extrapolation of the smooth GP prior and should be interpreted with the corresponding caution.

This kind of output is a coherent field of conditional distributions over a two-dimensional spatial index, from scattered and replicate-free data, with out-of-sample calibration. It is, to our knowledge, not directly available from existing conditional density estimation software, and it is the setting for which the SLGP was designed.

5.4 Simulating from the fitted field

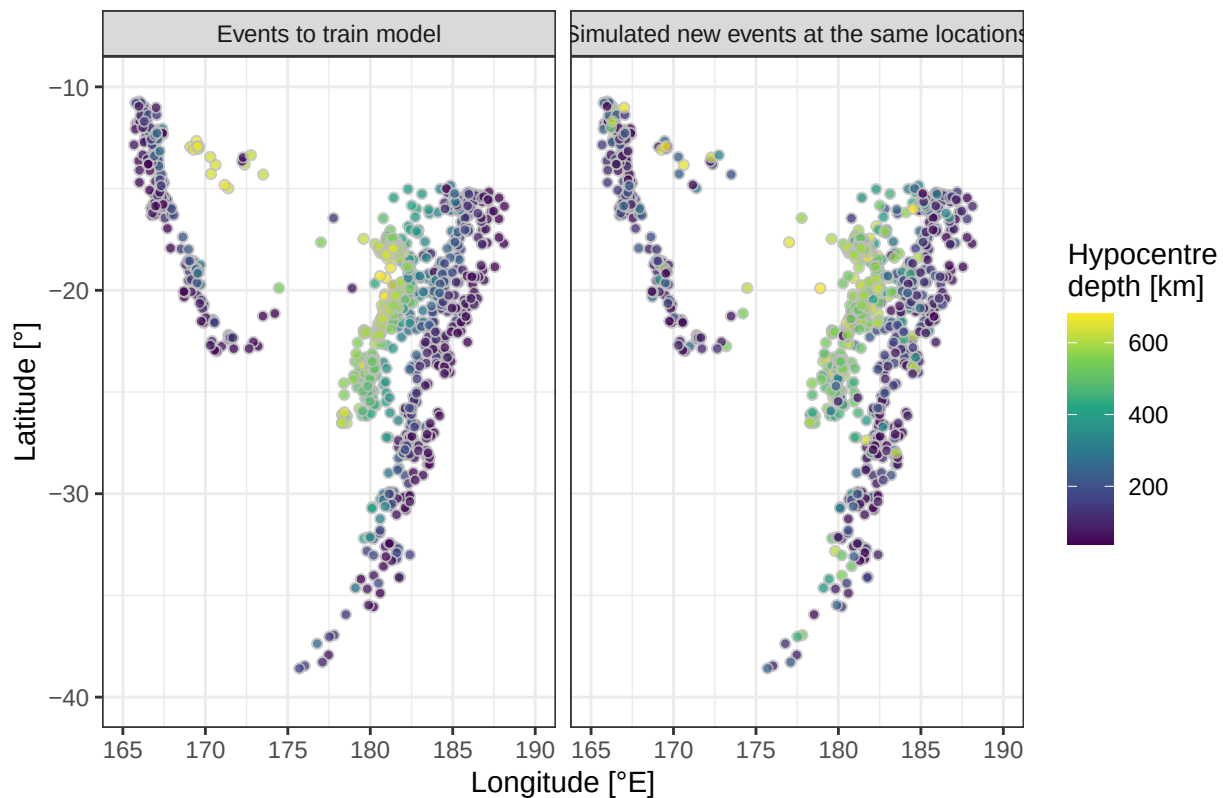
The fitted object is a full generative model, not only a density evaluator. `simulate()` draws synthetic responses at chosen locations by inverse-transform sampling from the predictive CDF, which is useful for propagating distributional uncertainty into downstream computations.

```

R> newX <- df[, c("long", "lat")]
R> sims <- simulate(modelMAP2D, nsim = 1, newdata = newX, seed=0)
R>
R> df_obs <- df %>%
+   mutate(event_type = "Events to train model")
R>
R> df_sim <- sims %>%
+   mutate(event_type = "Simulated new events at the same locations")
R>
R> df_plot <- bind_rows(df_obs, df_sim) %>%
+   mutate(event_type = factor(event_type,
+                               levels = c("Events to train model",
+                                           "Simulated new events at the same locations")))
R>
R> ggplot(df_plot, aes(x = long, y = lat, fill = depth)) +
+   geom_point(size = 1.5, alpha = 0.8, pch = 21, col = "grey", alpha=0.8) +
+   scale_fill_viridis_c(name = "Hypocentre\ndepth [km]",
+                         limits = range_response,
+                         option = "viridis") +
+   coord_cartesian(xlim = range_xlong, ylim = range_xlat) +
+   facet_wrap(~ event_type, nrow = 1) +
+   ggtitle("Observed and simulated events from the fitted SLGP") +
+   labs(x = "Longitude [°E]",
+        y = "Latitude [°]") +
+   theme_bw()

```

Observed and simulated events from the fitted SLGP



R>

```
R>  
R> ggsave("Figures/simulate.pdf", width = 8, height = 4)
```

References

- Cliff Frohlich. *Deep Earthquakes*. Cambridge University Press, Cambridge, 2006.
- Bryan Isacks, Jack Oliver, and Lynn R. Sykes. Seismology and the new global tectonics. *Journal of Geophysical Research*, 73(18):5855–5899, 1968. doi: 10.1029/JB073i018p05855.
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2025. URL <https://www.R-project.org/>.